Hanoi University of Science and Technology
School of Information and Communication Technology

IT3180E — Introduction to Software Engineering

AI-Driven Smart Home Energy Optimizer (SHEO)

Final Project Report

A capability-driven IoT energy-management system with explainable WHEN–THEN automation and Vietnamese-dong bill-saving estimation

Group	⟨*group name*⟩
Team leader	⟨*name — student ID*⟩
Members	⟨*name — ID*⟩, ⟨*name — ID*⟩, ⟨*name — ID*⟩, ⟨*name — ID*⟩, ⟨*name — ID*⟩ *(six members in total)*
Instructor	⟨*instructor name*⟩

Hanoi, June 22, 2026

Abstract

This report documents the design and verification of the **AI-Driven Smart Home Energy Optimizer (SHEO)**, a capability-driven Internet-of-Things energy-management system built for the course *IT3180E — Introduction to Software Engineering*. SHEO models a single apartment building: a building owner oversees every unit while each resident monitors the real-time power and cumulative energy of their own unit's devices, controls them through a generic interface that is generated from a declarative *capability schema*, learns daily usage habits, and turns those habits into explainable WHEN–THEN automation rules. Its defining feature, and the priority the client emphasised during elicitation, is that every rule is priced: the system estimates and displays, in Vietnamese dong (VND), the electricity bill a unit would save by adopting a suggested rule, and tracks the saving actually achieved within the current billing cycle.

The system is implemented as a three-tier, layered application — a React single-page client over a FastAPI service layer and a SQLite data tier — with a deterministic mock device simulator standing in for physical hardware, and is covered by 65 automated tests.

Rather than describing only the product, the report is organised to demonstrate the full software-engineering lifecycle taught across lectures C01–C09: software-engineering context and stakeholders, development process, project management, requirements engineering, architecture and detailed design, user-interface design, software construction, configuration management, and verification, validation, and maintenance. Each chapter names the relevant course concepts and traces them to concrete evidence in the SHEO project.

Contents

List of Figures

Chapter 1

Introduction

1.1 Project Overview and Motivation

This report documents the design and construction of the **AI-Driven Smart Home Energy Optimizer** (hereafter **SHEO**), a software system developed for the course IT3180E, *Introduction to Software Engineering*. SHEO models a single apartment building and helps each unit's resident understand and reduce its electricity bill, while the building owner oversees the building as a whole. It performs four closely related tasks. First, it *monitors* the real-time power and cumulative energy of every device in a unit. Second, it offers *capability-driven control*: rather than hard-coding a separate screen for each appliance, the system describes each device type by a declarative schema of its controls and telemetry, and the same interface renders and validates commands for a plug, a bulb, a fan, an air-conditioner, or a sensor without per-device code. Third, it *learns daily habits* from the observed usage and proposes human-readable automation rules of the form **WHEN ... THEN ...**, each of which the user can read, edit, accept, or dismiss. Fourth—and this is the headline contribution that the client singled out—it *estimates and displays the Vietnamese-dong saving* that each rule is expected to deliver, shown *before* the rule is accepted, and then reports the saving actually accrued over the current billing cycle.

The motivation is concrete and topical. Residential electricity costs in Vietnam follow a tiered tariff in which the marginal price rises with consumption, so even small, habitual waste compounds into a noticeably larger monthly bill. Residents are willing to automate their appliances, but they are reluctant to trust automation they cannot understand or whose benefit they cannot quantify. SHEO therefore treats *explainability and monetary visibility* as first-class requirements: every suggestion is a sentence a non-expert can read, and every saving figure reduces to one transparent formula expressed in VND. The project's purpose is to demonstrate that a modest, fully working system, engineered with discipline, can deliver this value end-to-end.

1.2 Software Engineering Context

SHEO is, before anything else, an exercise in software engineering, and the report is structured to make that discipline visible. The course adopts the canonical definition introduced in Lecture C01 [1] (*Overview of Software Engineering*): software engineering is the body of *methodologies, techniques, and tools* integrated into the software production and operation process in order to create software *of the desired quality*. The same lecture frames software engineering as a layered activity [7, 8]: a foundational *quality focus* supports a *process*, which is enacted through *methods*, which are in turn supported by *tools*. This report is organised along exactly that layering—the process and management chapters establish the process layer, the requirements, design, and construction chapters supply the methods, and the configuration-management and quality-assurance chapters anchor the quality focus.

A second principle from C01 governs the report throughout: the strict separation of *requirements* from *design*—of *what* the system must do from *how* it does it. Accordingly, the requirements chapter states only the observable behaviour the system must exhibit, while the design and construction chapters describe the architecture, components, and code that realise that behaviour. Keeping the two apart is what makes the chain from requirement to design to code to test auditable, which is itself a quality property the course grades.

1.3 The Client and the Stakeholders

Lecture C01 insists on a distinction that is easy to blur in everyday speech but essential to requirements work: the **Client** (the party for whom the software is built and who funds it), the **Customer** (the party who purchases or selects the software for use within an organisation), and the **User** (the person who actually operates the software day to day). C01 notes that customer satisfaction is the single most important measure of a project's success, so identifying these roles correctly is not a formality—it determines whose concerns the design must privilege.

The roles for SHEO were fixed in the initial client session. The **Client is an IoT contractor**: it manufactures and supplies the smart devices, builds the underlying artificial-intelligence capability, and commissions this development team to build the application that ties the two together. The Client funds the work and sets the priorities but does not itself operate the running system; it is therefore a *business stakeholder outside the system boundary*, not a login actor. The **Customer** is the *building owner or landlord* that adopts SHEO and runs it for a single apartment building; in the running system the Customer is realised as the **Administrator** actor, who owns no unit of their own but oversees every unit—viewing the resident roster, each unit's live power and usage, and the building totals—sells and onboards units to residents (and may offboard a resident, returning the unit to vacant while keeping its devices and history), and sets the building-wide tariff and billing cycle. The Administrator never operates a unit's devices and authors no automation rules. The **End-users** are the **Residents** (tenants), each occupying one unit, who view their own unit's consumption and savings, control the operable devices in that unit, and author their own automation rules, with no visibility of any other unit. Two further actors complete the picture: a **Developer/Tester**, who prepares and reproduces the demonstration, and a non-human **System Scheduler**—a time actor that autonomously evaluates and fires rules and mines usage habits. The relationship between the external stakeholders and the system is shown in Figure 1.1.

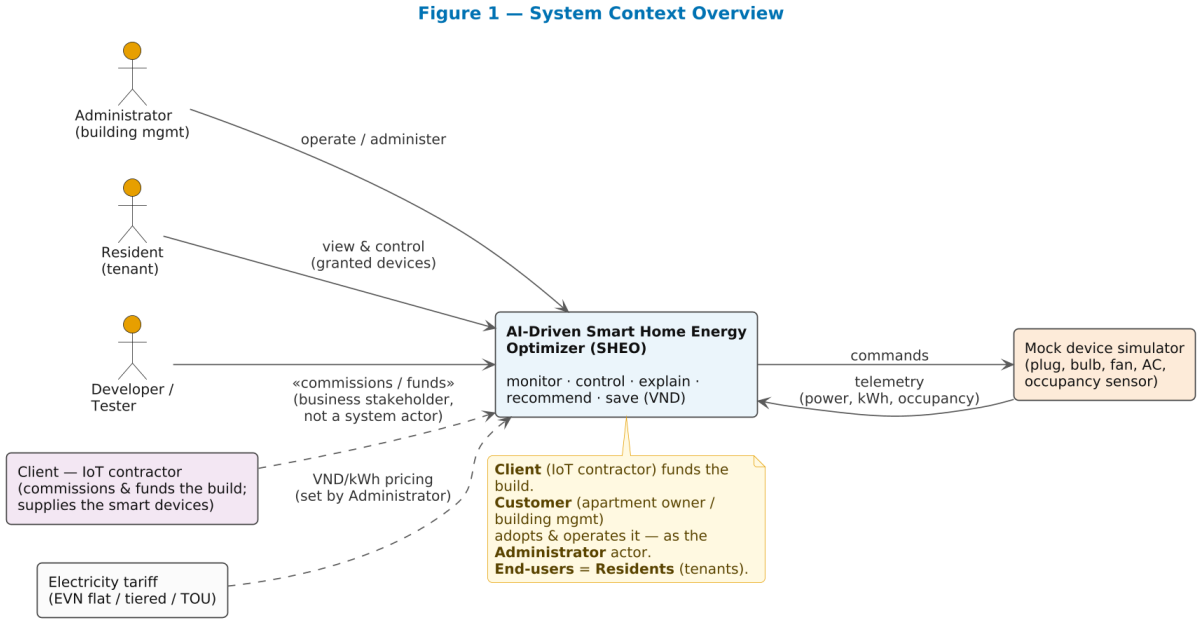

Figure 1.1. System context overview. The three human actors—Administrator (the Customer / building owner), Resident (tenant of a unit), and Developer/Tester—interact with SHEO. The Client (IoT contractor) is a business stakeholder *outside* the boundary that commissions and funds the build. A mock simulator stands in for physical devices, and the building-wide tariff is configured by the Administrator. The outputs returned to users are the live dashboard, notifications, explainable rules, and VND savings.

Table 1.1 summarises each stakeholder, the role it plays with respect to the system, and its principal concern. The most consequential entry is the Customer's: in the client session the contractor assigned *high priority* to the requirement that the Customer be able to *see the actual VND bill saved* by a suggested rule. That single priority shapes much of the rest of the report—it drives the bill-saving estimation engine, the explainability constraint, and the headline-figure layout of the dashboard.

Table 1.1. SHEO stakeholders, their roles with respect to the system, and their primary concerns.

Stakeholder	Role	Primary concern
Client (IoT contractor)	Commissions and funds the build; supplies devices and the underlying AI. Business stakeholder, *not* a system actor.	A demonstrable product that proves its value, especially the monetary saving the Customer can see.
Customer (building owner / landlord)	Adopts and operates SHEO; realised as the **Administrator** actor, who owns no unit and oversees all of them.	A building overview—the resident roster, every unit's usage, and building totals; selling and onboarding units; and setting the building-wide tariff.
End-user (Resident / tenant)	Day-to-day operator of one unit; the **Resident** actor.	A clear at-a-glance view of the unit's own consumption and savings, and simple control of that unit's operable devices.
Developer / Tester	Prepares and reproduces the demonstration.	A deterministic, reproducible system that is easy to exercise and test.
System Scheduler	Non-human time actor inside the system.	Reliable, periodic evaluation of rules and mining of habits.

1.4 Scope and Objectives

The objective of the project is to deliver, with full software-engineering rigour, the five capabilities the client requested. These define the *in-scope* functionality: (1) real-time monitoring of device power and energy; (2) capability-driven device control; (3) explainable WHEN–THEN automation rules with an opt-in auto-action; (4) habit-learning recommendations derived from observed usage; and (5) estimation and reporting of the VND bill saving of each rule. Each capability maps to a feature group in the requirements specification and is traced through design, construction, and test.

Two deliberate exclusions place the work *out of scope*. First, no physical IoT hardware is used: real plugs, bulbs, fans, air-conditioners, and sensors are unavailable to the team, so a **mock device simulator** stands in for them. The simulator implements the same device interface that a real vendor integration would, so the architecture remains hardware-ready, but the running product is demonstrable and testable entirely in software. Second, the "AI"—the habit detection the client (an AI contractor) supplies—is isolated behind a swappable **recommendation-provider port** (Strategy). The shipped default is an **explainable, rule-based** provider: every detected habit surfaces as a rule the user can read. A black-box machine-learning provider could be substituted behind the same port without touching the API, the UI, the rule engine, or—decisively—the VND saving estimator, which is deliberately kept as SHEO's own software so the saving always reduces to a documented formula. Shipping the explainable provider by default

keeps the product demonstrable in software and honours the client's high-priority demand that the saving be *visible and trustworthy*, while the design admits a learned model where one is warranted.

Both decisions follow directly from the *Function / Cost / Time* balance triangle that C01 places at the heart of project scoping. Procuring and integrating real hardware would inflate cost and time for no gain in the demonstrated function, whereas a deterministic simulator delivers the same observable behaviour within the course timeline. Likewise, building or training a model now would raise development cost and risk for no gain in demonstrated function, whereas the default explainable provider delivers it within the course timeline—and the provider port keeps a learned model available without re-architecting. In both cases the chosen scope maximises the function that matters while respecting the cost and time available, which is exactly the trade-off C01 asks an engineer to make explicit.

1.5 Report Structure

The remaining chapters follow the software-engineering lifecycle and are each anchored to a specific course lecture, so that the report doubles as a demonstration of the course material applied to a single coherent system. Table 1.2 maps each chapter to its governing lecture.

Table 1.2. Report roadmap: each chapter and the course lecture it applies to SHEO.

Chapter	Lecture	Topic
Chapter 2	C02 / C03	Software development process and Agile practice.
Chapter 3	C04	Software project management.
Chapter 4	C06	Requirements analysis and modelling.
Chapter 5	C07	Software (architectural and detailed) design.
Chapter 6	C07	User-interface and user-experience design.
Chapter 7	C08	Software construction and code quality.
Chapter 8	C05	Software configuration management.
Chapter 9	C09	Quality assurance: verification, validation, and maintenance.
Chapter 10	—	Conclusion and reflection.

Taken together, the chapters trace SHEO from the requirements that state *what* it must do, through the design and construction that determine *how* it does it, to the configuration management and quality assurance that keep it correct as it changes—the full arc the course defines as software engineering.

Chapter 2

Software Development Process

This chapter describes the software development process adopted for the AI-Driven Smart Home Energy Optimizer (SHEO) project. It grounds that description in the theoretical framework taught in the course: the software development life cycle (SDLC), the classical process models, the agile philosophy, and the ISO/IEC 9126 quality model. Each section moves from the general course concept to its concrete instantiation in SHEO.

2.1 The Software Development Life Cycle

A software development life cycle (SDLC) partitions the construction of a software system into a sequence of well-defined, measurable phases, each with assigned responsibilities and verifiable outputs. The course identifies two complementary views of the SDLC.

Phase-level view. Five core phases characterise most plan-driven processes: *Requirements Analysis* (understand and specify what the system must do), *Design* (determine how it will be structured), *Development / Coding* (realise the design in executable form), *Testing* (verify correctness and validate fitness for use), and *Maintenance* (correct, adapt, and extend the deployed system).

Generic project-step view. At the project level these phases are bounded by two additional activities: a *Feasibility Study* at the outset (to establish that the problem is worth solving and is tractable within the given constraints) and *Acceptance & Release / Operation* at the close. The full generic sequence is therefore:

Feasibility Study → *Requirements* → *UI Design* → *System Design* → *Program Development* → *Acceptance & Release* → *Operation & Maintenance*

The SHEO deliverable set mirrors this sequence directly. A feasibility study assessed technical and economic viability and fixed the scope decision to simulate hardware rather than depend on physical IoT devices. The Software Requirements Specification formalised the *what*; the Software Design Document (SDD) recorded the *how*; the backend codebase and React frontend constituted the *Development* phase; the automated test suite and test plan covered *Testing*; and a configuration management plan addressed ongoing *Maintenance* concerns. Crucially, the SDLC is not merely a checklist: quality control operated continuously across all phases rather than being deferred to a final testing stage, as the ISO/IEC 9126 standard prescribes.

2.2 Comparison of Process Models

The course presents six classical process models. Table 2.1 summarises the core idea and the conditions under which each model is most appropriate.

Table 2.1. Comparison of software process models

Model	Core idea	Best suited when
Waterfall	Sequential phases; each phase must be completed and signed off before the next begins. No iteration once a phase closes.	Requirements are fully understood and stable from the outset; domain is mature and well-documented; regulatory or safety standards mandate formal sign-off at each gate.
Modified Waterfall	Adds limited feedback arrows between adjacent phases so that a discovered defect may be corrected in the preceding phase without restarting the entire process.	Requirements are largely stable but minor clarifications are anticipated; a more disciplined team than pure Waterfall, but still primarily sequential.
Prototyping	A quick, often throw-away prototype is built to elicit or validate requirements before full development begins. Two variants: *throw-away* (discard after learning) and *evolutionary* (grow into the product).	Requirements are poorly understood or highly volatile; user interface expectations are hard to specify without tangible feedback; exploratory research context.
Incremental	The system is partitioned into increments, each of which delivers usable functionality. The core system is built first; subsequent increments add capability on a working base.	A working subset of the system can be deployed and used early; requirements are moderately understood; a small to medium team; value is best delivered in slices.
RAD	Rapid Application Development uses intensive, time-boxed cycles (typically 60–90 days) with heavy reuse of components and automated tooling to compress the schedule.	Time to market is critical; experienced team with strong tool knowledge; requirements can be scoped tightly enough for short cycles; reusable components are available.
Spiral	Each iteration traverses four quadrants: planning, risk analysis, engineering, and evaluation. Risk is the primary driver for deciding whether to proceed, prototype, or abandon.	Large, complex, high-risk systems; risk identification and mitigation are primary concerns; budget exists for repeated analysis cycles; safety-critical or mission-critical domains.

2.3 The Process Model Adopted for SHEO

2.3.1 Model selection

SHEO was developed using an **incremental, agile-style process**. This is closest to the Incremental model in the course taxonomy, augmented by agile working practices described in Section 2.4. The selection was justified by the following conditions, cross-referenced against the course's model-selection criteria.

(1) **Moderately understood requirements.** The functional scope—energy monitoring, capability-driven device control, rule-based automation, and AI-generated recommendations—was clear at a high level from the outset, but many interface details and edge-case safety constraints emerged during development. This ruled out pure Waterfall (which demands fully stable requirements) and favoured an iterative approach in which each increment surfaced and resolved ambiguities in a bounded scope.
(2) **Small team, short horizon.** A student team working over a single semester cannot sustain the overhead of formal Spiral risk-quadrant analysis, nor the intensive reuse infrastructure that RAD presupposes. An incremental model with lightweight process keeps coordination costs proportionate to team size.
(3) **Value delivered in working slices.** An early runnable system—even with only authentication, device registration, and a simulated sensor feed—provided a concrete foundation for demonstrating correctness and gathering feedback. Each subsequent increment extended a tested, running product rather than an accumulation of paper designs.
(4) **Reproducible demonstration requirement.** The project required a seeded demo that could be reliably executed at any point. This is best served by always maintaining a runnable product, which the incremental approach enforces naturally.

2.3.2 Incremental construction of SHEO

Development proceeded through five broad increments, each leaving the system in a passing, demonstrable state before the next began.

Increment 1. Runnable spine. Authentication (token-based), the user-role model (Administrator / Resident / Developer), device registration, and the database schema. The test suite was initialised; the seeded demo was executable from this point.

Increment 2. Capability schema and simulator. The declarative capability schema—which describes every device's controllable parameters, constraints, and UI contract—was introduced, together with the background simulator that drives synthetic sensor readings. All existing tests continued to pass.

Increment 3. Energy monitoring and safety rules. Real-time energy tracking endpoints, the safety-rule enforcement layer (NFR-SAF: rate limits, prohibition on autonomous power-off of safety-critical devices), and the corresponding automated tests.

Increment 4. Rule engine and recommendation engine. User-defined automation rules, the AI-backed recommendation generator, cost-savings estimation, and the acceptance test cases from the test plan.

Increment 5. UI and integration. React frontend pages, the capability-driven device control widget, the user flow, and final integration verification across all 65 test cases.

The team acknowledges that this was not a formally documented incremental plan with written increment specifications. The division above reflects the actual sequence of verified milestones rather than a pre-approved plan document, which is consistent with the agile-style nature of the process.

2.4 Agile Practices Applied

The course distinguishes between Agile as a *philosophy* (the 2001 Manifesto) and Scrum as a specific *framework*. SHEO embodied several Agile values and principles but did not run formal Scrum ceremonies. Each practised value is documented below.

Working software over documentation

The most consistently applied Agile value was the primacy of a runnable, tested product. At no point during development was the system reduced to a non-executing state; every increment was required to leave the full test suite passing before integration. The 54 automated test cases serve as the continuous evidence of this discipline. Documentation (the SDD, test plan, configuration management plan) was produced to capture design decisions, but it was never treated as a substitute for a working system.

Short increments and continuous delivery of value

Features were integrated in small, coherent units—a single endpoint with its test, a new capability handler, a frontend page—rather than in large, infrequent releases. This kept the feedback loop short and avoided the integration risk that accumulates when parallel development streams diverge over extended periods.

Continuous testing

Testing was not deferred to a distinct phase. Unit and integration tests were written alongside the corresponding implementation, and the full suite was re-executed after every non-trivial change. This is a direct application of the Agile principle that technical excellence and good design enhance agility.

Responding to change

Several design decisions were revised in response to feedback during development: the original role model was renamed to align with the course's stakeholder taxonomy; safety-rule NFRs were tightened after an adversarial review identified a cross-unit access vulnerability; and the capability schema was generalised to accommodate additional device types without modifying existing handlers. Each change was absorbed without disrupting the passing test suite, demonstrating the architectural support for change that Agile values require.

Definition-of-Done analogue

Although no formal Scrum Definition of Done was documented, the team operated with an implicit and consistently applied criterion: *a feature is complete when (a) it has at least one passing automated test that exercises its primary execution path, and (b) the seeded demonstration database reflects its behaviour such that an evaluator can observe it without additional setup.* This criterion operationalises two Scrum concepts— the Increment and the Definition of Done—in a lightweight form appropriate to a small student project.

Honest scope statement

Formal Scrum roles (Product Owner, Scrum Master), artefacts (Product Backlog, Sprint Backlog), and events (Sprint Planning, Daily Standup, Sprint Review, Retrospective) were not adopted. The team was too small to benefit from the full Scrum overhead, and the academic calendar imposed a fixed delivery date rather than an open-ended product roadmap. The process is therefore characterised as *agile-style incremental* rather than Scrum.

2.5 Quality Throughout the Process

The course cites ISO/IEC 9126 [6] as the quality standard for software products and emphasises that quality assurance must run *throughout* the development process, not only at the end.

Table 2.2 maps each ISO/IEC 9126 quality characteristic to the concrete mechanism or non-functional requirement (NFR) that addressed it in SHEO, together with the earliest phase at which that mechanism was active.

Table 2.2. ISO/IEC 9126 quality attributes mapped to SHEO mechanisms

Quality attribute	**Definition (ISO/IEC 9126)**	**SHEO mechanism / NFR**	**Active from**
Functionality	The set of features present and their suitability for intended tasks.	Functional requirements REQ-4.1 through REQ-4.5 (monitoring, control, rules, recommendations, savings); validated by 65 automated acceptance and integration tests; traced in the Requirements Traceability Matrix.	Requirements
Reliability	Ability to maintain performance under stated conditions over time; fault tolerance.	NFR-SAF: rate limits on device commands, prohibition on autonomous power-off of safety-critical appliances, input validation on all public endpoints. An adversarial review identified and closed a cross-unit data-isolation defect prior to release.	Design
Usability	Ease of learning and use; appropriateness of the interface for target users.	Capability-driven UI widget: device controls are rendered from the capability schema, eliminating hard-coded per-device pages and ensuring consistent constraint enforcement (e.g. AC target temperature bounded to $16 \leq T \leq 30\,^{\circ}\mathrm{C}$). The seven UI golden rules were applied throughout UI design; a usability heuristic walkthrough was conducted before the frontend was built.	Design

Quality attribute	Definition (ISO/IEC 9126)	SHEO mechanism / NFR	Active from
Efficiency	Resource consumption relative to performance under stated conditions.	NFR-PER: API response time target $\leq$ 200 ms under simulated concurrent load; in-process database avoids network round-trips in the demo; the background simulator runs as a non-blocking asynchronous task to avoid blocking the request loop.	Design
Maintainability	Ease of modifying the system to correct faults, improve performance, or adapt to a changed environment.	Capability schema as the single extension point for new device types (Open–Closed Principle); Strategy pattern in the adapter layer isolates device-specific logic; Repository pattern decouples persistence from business logic; all public interfaces documented with type hints and docstrings.	Design
Portability	Ability to transfer the system to different environments.	Standard Python packaging, environment-variable configuration, and a containerisation-ready architecture (no hard-coded paths or OS-specific system calls); hardware abstracted behind the simulator interface so real IoT drivers can be substituted without changing application logic.	Design

The critical point is that none of these mechanisms was applied solely at the end of the project. Reliability constraints were encoded as safety rules before the first device endpoint was written. Maintainability was designed in from the outset through the capability schema and layered architecture. Efficiency targets were stated as NFRs and informed technology choices. Usability was evaluated iteratively through wireframes and heuristic review before the frontend was built. This continuous, cross-phase attention to quality is what both the course and the Agile philosophy prescribe, and it is what enabled SHEO to reach a state of 65 passing tests, a fully seeded and demonstrable product, and no known open defects at the point of submission.

Chapter 3

Software Project Management

This chapter documents the project management artifacts for SHEO following the framework taught in Lecture C04: the four Ps, stakeholder analysis, Work Breakdown Structure (WBS), effort estimation, scheduling, and risk management. These artifacts are constructed to satisfy the course deliverable requirements and to reflect the actual scope and constraints of a small student team building a software-only system.

3.1 The Four Ps

3.1.1 People

The development team consists of six undergraduate students, each taking on overlapping roles across the project lifecycle. No dedicated project manager was appointed; instead, the team operated with shared responsibility and peer review. External stakeholders include the IoT contractor (Client), the building management organisation (Customer), and the tenants who use the system daily (End-users / Residents). Skill coverage spans backend engineering (FastAPI, Python), frontend development (React/TypeScript), systems integration, and documentation.

3.1.2 Product

SHEO (AI-Driven Smart Home Energy Optimizer) is a web-based platform that monitors household energy consumption through IoT device telemetry, supports rule-based and AI-assisted device control, surfaces personalised energy-saving recommendations, and estimates resulting cost savings. The product scope is bounded to a software application that communicates with a hardware abstraction layer via a capability schema; no physical hardware is procured or modified.

3.1.3 Process

The team adopted an *incremental* development approach: a functional backend spine was delivered first, followed by successive increments adding device control, the rules engine, the recommendation module, and the web UI. Each increment closed with automated regression testing (65 test cases) and a peer code-review pass. This mirrors the Incremental model described in C02 and aligns with Agile values of working software and customer collaboration.

3.1.4 Project

The project is a temporary, unique academic endeavour with a fixed delivery date. Key constraints are: no physical IoT hardware available (mitigated by a deterministic simulator), a six-person team with limited availability, and a requirement to satisfy course deliverables in parallel with development. Success is measured against the functional requirements (REQ-4.1–REQ-4.5), non-functional requirements (NFR-PER, NFR-SAF, NFR-SEC), and the acceptance criteria in the test plan.

3.2 Stakeholder Analysis

Table 3.1 identifies all stakeholders, their role in the C01 framework, and their principal interests and influence over the project.

Table 3.1. SHEO Stakeholder Analysis

Stakeholder	C01 Role	Interest	Influence
IoT Contractor	Client	Commission a reliable, extensible energy-management platform; protect investment in IoT hardware	High — funds the project and defines the device capability schema
Building Management / Apartment Owner	Customer (Administrator)	Reduce building-wide energy costs; maintain tenant satisfaction; oversee device onboarding and user accounts	High — primary adopter; defines operational policy
Resident (Tenant)	End-user	Convenient, safe device control; transparent energy feedback; actionable saving recommendations	Medium — directly operates the system; acceptance drives adoption
Development Team	Developer / Tester	Deliver a correct, maintainable product within course constraints	High (internal) — executes all design, build, and test activities
System Scheduler	Non-human actor	Execute time-triggered rules and telemetry aggregation reliably	Low (external input) — a system component, not a human stakeholder

3.3 Work Breakdown Structure

Table 3.2 decomposes SHEO into ten deliverable groups and their constituent work packages. Each leaf work package (WP) is assigned an identifier, a brief description, and an acceptance criterion that can be verified at handover.

Table 3.2. SHEO Work Breakdown Structure with Acceptance Criteria

WP	Work Package	Acceptance Criterion
1	**Monitoring**	
1.1	Real-time dashboard	Dashboard displays live telemetry for all registered devices within 2 s of event receipt.
1.2	Telemetry ingestion pipeline	Device readings are persisted to the database; simulator and live sources produce identical schema.
1.3	Top-consumer identification	API returns the top-5 energy-consuming devices, ranked correctly, for any selected time window.
2	**Device Control & Capability Schema**	

WP	Work Package	Acceptance Criterion
2.1	Capability schema registry	All device types expose a valid JSON capability document; unrecognised types are rejected at onboarding.
2.2	Device command execution	Control commands update device state and are reflected in the UI within 1 s.
3	**Rules Engine**	
3.1	Rule authoring (CRUD)	Residents can create, read, update, and delete automation rules scoped to their own unit.
3.2	Rule evaluation & triggering	Time- and condition-based rules fire deterministically; conflicting rules are resolved by priority.
4	**AI Recommendations (Habit Mining)**	
4.1	Usage-pattern analysis	The recommendation engine produces at least one suggestion per device with 7 days of history.
4.2	Recommendation lifecycle management	Residents can accept or dismiss recommendations; state transitions follow the documented FSM.
5	**Savings Estimation**	
5.1	Tariff model & cost computation	Estimated monthly savings are within ±5% of a manually computed reference for the demo dataset; costs are stated in VND.
5.2	Savings history & reporting	Residents can view a 30-day savings trend with correct aggregation.
6	**Web UI**	
6.1	Resident portal	All Resident use cases (view, control, rules, recommendations) are reachable within 3 navigation steps.
6.2	Administrator console	Administrators can onboard devices, manage accounts, and view building-wide analytics.
6.3	Responsive layout & accessibility	UI renders correctly at 1280 px and 375 px viewports; contrast ratio meets WCAG AA.
7	**Simulator & Scheduler**	
7.1	Deterministic device simulator	Simulator replays the seeded dataset identically on each run (fixed random seed).
7.2	Background task scheduler	Scheduled rules fire at the configured time (±5 s); missed triggers are logged.
8	**Authentication & RBAC**	
8.1	Token-based authentication	Login succeeds with valid credentials; invalid tokens are rejected as unauthenticated.
8.2	Role-based access control	Resident endpoints are denied as unauthorised when accessed with an Administrator token and vice versa.
9	**Testing**	
9.1	Unit and integration test suite	Automated suite covers all REQ-4.x items; all 65 test cases pass on a clean checkout.
9.2	Acceptance test mapping	Each functional requirement is traced to at least one passing test case in the test-plan RTM.

WP	Work Package	Acceptance Criterion
10	**Documentation**	
10.1	Software Requirements Specification	SRS is reviewed and signed off; all functional and non-functional requirements are enumerated.
10.2	Software Design Document	SDD covers architecture, detailed design, UI/UX, and traceability; all diagrams are consistent with code.
10.3	Project management report	This chapter is produced and included in the course deliverable package.

3.4 Effort Estimation

Effort was estimated using a *complexity-based sizing* method: each work package was classified as Small (S), Medium (M), or Large (L) based on the number of components involved, interface dependencies, and uncertainty. A baseline productivity rate was then applied (S = 2 person-days, M = 4 person-days, L = 7 person-days), which is consistent with the effort-table approach taught in C04. Table 3.3 summarises the result. Total estimated effort is approximately **68 person-days** across the six-member team over a twelve-week period (approximately 11 person-days per team member).

Table 3.3. Effort Estimation by Work Package

WP	Work Package	Complexity	Est. (person-days)
1	Monitoring (dashboard + telemetry + top-consumers)	L	7
2	Device control & capability schema	M	4
3	Rules engine (CRUD + evaluation)	L	7
4	AI recommendations (habit mining + lifecycle)	L	7
5	Savings estimation (tariff model + reporting)	M	4
6	Web UI (Resident portal + Admin console + layout)	L	7
7	Simulator & scheduler	M	4
8	Authentication & RBAC	M	4
9	Testing (suite + acceptance mapping)	L	7
10	Documentation (SRS + SDD + PM report)	L	7
–	Integration, code review, and contingency (15%)	–	10
		Total estimated effort	**68**

The integration and contingency buffer of approximately 15% accounts for merge conflicts, rework identified during peer review, and unforeseen dependencies between the rules engine and the recommendation module.

3.5 Project Scheduling

3.5.1 CPM Task Network

Table 3.4 lists the eight planning tasks, their predecessors, durations (in calendar days), and the Early Start (ES) and Early Finish (EF) values computed by the Critical Path Method. All durations assume a parallel team of six with partial time availability (estimated effective capacity of approximately 40% per day per member during semester).

Table 3.4. CPM Task Schedule (calendar days from project start)

ID	Task	Predecessor(s)	Duration	ES	EF
T1	Requirements analysis & SRS	—	7	0	7
T2	Architecture & database design	T1	10	7	17
T3	Authentication & RBAC	T2	5	17	22
T4	Device control & capability schema	T3	8	22	30
T5	Monitoring & telemetry	T4	10	30	40
T6	Rules engine & scheduler	T4	12	30	42
T7	Recommendations & savings	T5, T6	12	42	54
T8	Web UI, testing & documentation	T7	14	54	68

The **critical path** is T1 → T2 → T3 → T4 → T6 → T7 → T8, with a total duration of **68 calendar days** (approximately 10 weeks). Task T5 (Monitoring) carries two days of float relative to T6 and is therefore not on the critical path.

3.5.2 Timeline Overview

Figure 3.1 provides a high-level Gantt-style view of the schedule. Dark bars represent tasks on the critical path; the lighter bar represents the non-critical Monitoring task (T5).

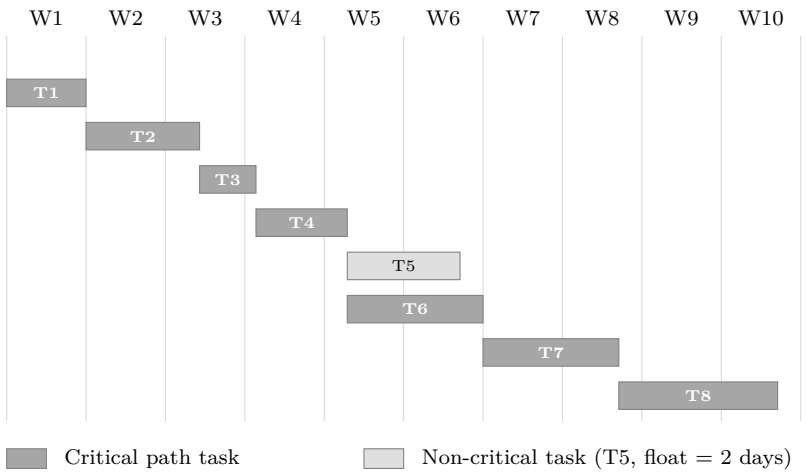

Figure 3.1. High-level project schedule (10-week view)

3.6 Risk Management

Risk management followed the five-step process from C04: plan, identify, analyse (Risk level = Probability × Impact), respond (Avoid / Transfer / Mitigate / Accept), and monitor. Table 3.5 presents the SHEO risk register. Probability is coded L = Low, M = Medium, H = High;

impact is Minor, Moderate, or Major. The composite risk level is derived as Probability × Impact on a 3×3 ordinal scale (L/M/H = 1/2/3; Minor/Moderate/Major = 1/2/3), giving integer values from 1 to 9.

Table 3.5. SHEO Risk Register

Risk	**Category**	**Prob.**	**Impact**	**Level**	**Response**	**Monitoring / Contingency**
No physical IoT hardware available for development or demonstration	Technical	H	Major	9	Mitigate	Replace with a deterministic software simulator (fixed seed); capability schema abstracts the hardware boundary so real devices can be integrated post-delivery without code changes.
Unsafe automatic actions on safety-critical devices (e.g., gas valves, fire suppression)	Safety	M	Major	6	Mitigate	Automatic action is *off by default* (NFR-SAF); the rules engine enforces a safety guard that prevents auto power-off of flagged devices; Residents must explicitly enable automation per device class.
Scope creep: feature additions beyond the agreed requirements baseline	Management	M	Moderate	4	Mitigate	Requirements are frozen in the SRS after T1; all change requests are impact-assessed before implementation; team reviews backlog scope at the start of each increment.
Tariff-model complexity: energy pricing structures vary by region and time-of-use period	Technical	M	Minor	2	Accept / Mitigate	A simplified flat-rate and two-band time-of-use model is implemented; tariff parameters are externalised to a configuration file so Administrators can update rates in VND without code changes.
Demo non-reproducibility: randomised simulation produces different outputs on each run	Technical	H	Moderate	6	Mitigate	All simulation components use a fixed random seed committed to the repository; the demo startup script re-applies the seed automatically, ensuring identical output on every execution.

Risk	Category	Prob.	Impact	Level	Response	Monitoring / Contingency
Time and skills shortfall: team members lack experience with selected technologies	Management	M	Moderate	4	Mitigate	The chosen web framework and client library are mature and widely adopted; a two-day spike was budgeted at T2 for upskilling; pair programming is used on unfamiliar modules.
Cross-unit data leakage: a Resident accesses another unit's data via the API (IDOR)	Security	L	Major	3	Mitigate	All database queries are scoped by the unit identifier carried in the verified session token; an adversarial code-review pass specifically targeted cross-unit access vulnerabilities; all findings were corrected and re-tested before delivery.
Frontend–backend integration failure after independent parallel development	Technical	M	Moderate	4	Mitigate	An interface contract is agreed and frozen at T2, from which the front-end's typed access layer is generated; integration is exercised by the automated test suite from T5 onward.

The two highest-priority risks — *no physical hardware* (level 9) and *unsafe automatic actions* (level 6) — are both mitigated by architectural decisions embedded in the core design. The remaining risks are re-evaluated at the handover between each development increment.

Chapter 4

Requirements Engineering

This chapter establishes *what* the AI-Driven Smart Home Energy Optimizer (SHEO) must do, deliberately kept separate from *how* it is built. Following the discipline taught in Lecture C06 (Requirement Analysis), it first walks through the requirements engineering process, then reconstructs the client elicitation session from which the requirements were drawn, and finally records the catalogue of functional and non-functional requirements, the use-case model, the use-case specifications, and the behavioural and data models. The artefacts produced here form the agreed basis against which the system is later designed, constructed, and acceptance-tested.

4.1 The Requirements Engineering Process (C06)

Lecture C06 frames requirements engineering as a five-step pipeline whose output is a Software Requirements Specification (SRS) [3]. SHEO followed this pipeline faithfully:

1. **Feasibility study.** Before committing, the team checked that the product was technically, operationally, and economically achievable within a single course timeline. The decisive constraint — no physical Internet-of-Things (IoT) hardware — was resolved by deciding to substitute a deterministic mock device simulator, which keeps the whole product demonstrable end-to-end while remaining hardware-ready. The conclusion was that a small, fully working system was feasible and preferable to a large half-built one.
2. **Requirements collection (elicitation).** Requirements were gathered directly from the client in an offline discussion (Section 4.2). The principal elicitation techniques applied were the *structured interview* (asking the client to describe the devices, the desired controls, and the business priority) and *observation / scenario walk-through* (reasoning about how a resident would inspect consumption, control a device, and act on a suggestion). The course's own client-discussion practice — in which each group posed sharp scoping questions to the instructor acting as client — supplied the model for this step.
3. **Analysis.** The raw notes were decoded, ambiguities were surfaced, and conflicts and gaps were negotiated into testable statements. For example, the client's loose phrase "find habit and apply that rule" was analysed into two separable obligations: *mine* a habit and *recommend* a rule, with the user retaining explicit control over whether it is ever applied.
4. **Modelling.** The analysed requirements were captured in the standard C06 models: a use-case diagram and use-case specification tables for the functional view (Sections 4.5–4.6), and an entity-relationship diagram together with finite-state machines for the data and behavioural views (Section 4.7).
5. **Definition (SRS).** Finally the requirements were written up as a structured catalogue of functional requirements (**REQ-4.x**, **REQ-6.1**), non-functional requirements (**NFR-***), and business rules (**BR-1**–**BR-5**), each given a unique identifier so that it can be traced forward into design, code, and test. C06's quality criteria for an SRS — that it be *clear, accurate, relevant,* and *complete* — guided this write-up, and the requirement identifiers used throughout this chapter are exactly those carried into the traceability matrix.

A central rule from C01 and C06 underpins the entire chapter: requirements ("what") are kept strictly separate from design ("how"). The statements below describe the system's externally observable obligations; the architecture and source modules that satisfy them are the subject of later chapters.

4.2 Client Elicitation Session

The requirements originate in an offline discussion with the project's client, an IoT contractor. Reconstructed and decoded, the session ran as follows.

Who the client is. Per the C01 stakeholder taxonomy, the client is the party that *commissions and funds* the build. Here that is an IoT contractor who manufactures a range of AI capabilities and a range of IoT devices, and who hires the team to deliver an application on top of them. Crucially, the contractor is a *business stakeholder, not a system actor*: it pays for and supplies the system but does not operate the running software day to day. The parties who do operate it — the building management (the *customer*) and the residents (the *end-users*) — are distinguished from the client throughout, exactly as C01 requires.

What the client asked for. The contractor "has all kinds of IoT devices" and "wants to provide an application." Each device type exposes its own controls: a fan can change speed and set a mode; an air-conditioner exposes a target temperature and an on/off switch. The client did not want the application to hard-code one screen per device. Instead it asked for an *application programming interface that decides which feature each device exposes*, so that the appropriate controls for any given device are determined dynamically. Because no real hardware was available for the build, the client agreed that the team should "create some kind of a mock device" standing in for the physical ones.

Automation and recommendations. The client described automation in terms of *preconditions and postconditions* attached to a device's behaviour: a user *may* set these conditions if they wish, but — in the client's words — "if they don't," the system should "find a habit and apply that rule." Analysis turned this into the habit-recommendation requirement: when the user has not authored a rule, the system mines the resident's usage pattern and *recommends* an automation rule, which the user can then read, accept, or dismiss.

The high-priority requirement. The client stated one priority explicitly and emphatically: *energy saving*. The contractor wants the customer "to see the result of the actual electricity bill they saved when using the suggested rule." This single sentence fixes the product's centre of gravity: every suggested rule must carry a concrete, monetary estimate of the saving it would achieve, expressed in Vietnamese đồng (VND), and shown *before* the rule is accepted. The estimation must therefore be transparent and explainable rather than a black box — a constraint that shapes the whole optimization design.

Relation to the course's client-Q&A norm. The course runs a recurring practice in which each group, having received a terse client brief, must return to the client with sharp questions that pin down scope, roles, and edge cases before designing anything — for instance, questions on who may publish a public ticket, how delivery is handled, or whether an offline mode is required. SHEO's elicitation mirrors this norm. The team's clarifying questions resolved exactly the points the client's notes left open: *scope* (mock devices instead of real hardware, an explainable rule engine instead of opaque machine learning), *roles* (which party is client, customer, and end-user, and which of these becomes a login actor), and *edge cases* (what happens when a device is offline, when a command is out of range, or when a safety-critical device would be powered off automatically). Those answers are precisely the obligations catalogued below.

4.3 Functional Requirements

The functional requirements are organised into six families, each tracing back to a point raised in the elicitation session. Table 4.1 lists the most important sub-requirements; every one carries a stable identifier (**REQ-4.x.y**, **REQ-6.1**) and a priority (H = high, M = medium) used in the traceability matrix.

Table 4.1. Functional requirements catalogue (the principal sub-requirements).

ID	Family	Requirement (the system shall ...)	Pri.
REQ-4.1 — Real-time monitoring			
REQ-4.1.1	Monitoring	display instantaneous power per device, refreshed within 5 s.	H
REQ-4.1.2	Monitoring	report cumulative energy in kWh for today, the billing cycle, and a chosen range.	H
REQ-4.1.3	Monitoring	retain one-minute telemetry aggregates for at least 12 months.	H
REQ-4.1.4	Monitoring	mark a device unreachable after more than 60 s of silence and exclude it from the live total.	H
REQ-4.1.5	Monitoring	rank the top consumers by day, week, and month.	H
REQ-4.2 — Capability-driven control			
REQ-4.2.1	Control	expose, per device, a capability description that the front-end fetches *before* rendering controls.	H
REQ-4.2.2	Control	provide on/off and per-type controls for plug, bulb, fan, and air-conditioner.	H
REQ-4.2.3	Control	validate each command against the device's capability range and reject anything out of range.	H
REQ-4.2.4	Control	return a command outcome — SUCCESS, REJECTED, or TIMEOUT — within 5 s.	H
REQ-4.2.5	Control	provide a mock simulator for plug, bulb, fan, air-conditioner, and sensor device types.	H
REQ-4.3 — Explainable rules & auto-actions			
REQ-4.3.1	Rules	support conditions over time, day, tariff, device state, and sensor readings.	H
REQ-4.3.2	Rules	require a rule's action to match the target device's capability.	H
REQ-4.3.3	Rules	detect conflicts between enabled rules whose time windows overlap.	H
REQ-4.3.4	Rules	let the user enable, disable, edit, and delete rules.	H
REQ-4.3.5	Rules	log every rule execution in an immutable audit record.	H
REQ-4.3.6	Rules	keep automatic action opt-in (off by default) and offer a 2-minute undo.	M
REQ-4.4 — Habit recommendations			
REQ-4.4.1	Recommend	generate recommendations only after at least 7 days of telemetry.	H
REQ-4.4.2	Recommend	express each recommendation as a readable `WHEN--THEN` rule with a rationale and data window.	H
REQ-4.4.3	Recommend	rank recommendations by VND saving and keep at most five active.	H
REQ-4.4.4	Recommend	convert a recommendation into a real rule only on explicit acceptance.	H
REQ-4.4.5	Recommend	suppress a dismissed recommendation for at least 30 days.	H
REQ-4.5 — VND bill-saving estimation (the client's high priority)			
REQ-4.5.1	Savings	compute a 14-day hourly baseline energy profile per device.	H
REQ-4.5.2	Savings	estimate a saving as Σ ((baseline − expected) × tariff).	H
REQ-4.5.3	Savings	show the VND estimate *before* the user saves a rule or accepts a recommendation.	H
REQ-4.5.4	Savings	display the saving actually accrued so far in the current billing cycle (VND).	H
REQ-4.5.5	Savings	flag a rule for recalculation when measured saving drifts beyond ±20% of the estimate.	M
REQ-6.1 — Reporting			

ID	Family	Requirement (the system shall ...)	Pri.
REQ-6.1	Reporting	export readings, rules, and savings as CSV.	M

Two features deserve emphasis because they encode the client's explicit wishes. First, **REQ-4.2.1** captures the requirement for an interface "deciding which feature" a device exposes: controls are rendered from a per-type capability description rather than hard-coded, so a new device type is supported without new control code. Second, **REQ-4.5.3** encodes the high-priority demand that the customer *see* the saving of a suggested rule before committing to it — the estimate must be visible at decision time, not merely after the fact.

4.4 Non-Functional Requirements and Business Rules

Following C06, non-functional requirements are classified as *product*, *organisational*, or *external* requirements. SHEO's product NFRs cover performance (**NFR-PER**), safety (**NFR-SAF**), and security (**NFR-SEC**); the external requirements are the deployment-level transport-security obligations; and the business rules (**BR-1**–**BR-5**) are organisational constraints that the business places on the system's behaviour. Tables 4.2 and 4.3 record them.

Table 4.2. Non-functional requirements, classified per C06 (product / external).

ID	Class	Requirement	Pri.
NFR-PER-1	Product (performance)	The dashboard shall refresh within 5 s for up to 30 devices.	H
NFR-PER-2	Product (performance)	A control command shall return user-visible feedback within 2 s (95th percentile).	H
NFR-PER-3	Product (performance)	The system shall sustain at least 100 concurrent simulated homes on one node.	H
NFR-SAF-1	Product (safety)	At most one air-conditioner compressor command shall be issued per 3 minutes.	H
NFR-SAF-2	Product (safety)	The system shall never automatically power off a safety-critical device.	H
NFR-SAF-3	Product (safety)	Authoring shall warn on temperature, safety-critical, and unattended-off actions.	H
NFR-SEC-1	External (deployment)	All traffic shall use TLS 1.2+ (HTTPS / WSS).	H
NFR-SEC-2	Product (security)	The system shall enforce authentication and role-based access control.	H
NFR-SEC-3	Product (security)	No password or token shall be stored in plaintext.	H
NFR-SEC-4	Product (security)	Access to readings and logs shall be scoped to the user's own unit; the Administrator may view every unit in the building.	H
NFR-SEC-5	Product (security)	The building overview shall disclose only aggregate per-unit metrics (live load, energy, bill) and device *reachability*, never a resident's per-device on/off state — data minimisation, so oversight does not become behavioural surveillance.	M

The safety NFRs and **BR-2**/**BR-3** together express a deliberate caution principle that runs through the requirements: the system advises and estimates, but it acts on a unit's devices only when a human has explicitly authorised it, and never in a way that could compromise a safety-critical device.

Table 4.3. Business rules — organisational constraints (**BR-1**–**BR-5**).

ID	Business rule	Pri.
BR-1	Only the Administrator (building owner) may onboard a resident by selling them a unit (provisioned with its default device package) or offboard a resident, after which the unit becomes vacant while its devices and billing history are retained; a Resident cannot add or remove devices.	H
BR-2	A recommendation remains advisory and has no effect until it is explicitly accepted.	H
BR-3	Automatic action is disabled by default; the user must opt in per rule.	H
BR-4	Deleting a rule shall preserve its execution history.	H
BR-5	The tariff is manually configurable and defaults to VND.	M

4.5 Use-Case Model

The functional requirements above are organised into a use-case model that views the system as a single black box used by a small set of external actors. In keeping with the C06 convention, each actor is named with a *singular role noun* (not an individual or a job title), and each use case is named *verb + noun*.

The model has four actors:

- **Administrator** — the building owner, realised by the customer (building management). Owns no unit and never operates individual devices: monitors a read-only building overview across all units (resident roster, per-unit usage, building totals), configures the building-wide tariff, onboards residents by selling them a unit (provisioned with its default device package), and offboards a resident — a soft removal that returns the unit to vacant while retaining its devices and history.
- **Resident** — the primary end-user (a tenant of one unit). Views their own unit's consumption and savings, controls every operable device in that unit, and may author automation rules and accept or dismiss recommendations for it; no other unit is visible to them.
- **Developer/Tester** — a maintenance role that may operate any device for diagnostics, exercises the mock devices, and uses the offline-test hook to prepare and reproduce demos.
- **System Scheduler** — a non-human (time) actor that autonomously fires due rules and runs the periodic habit-mining and savings-accrual jobs.

The model uses the ⟨⟨include⟩⟩ relationship for mandatory sub-behaviour: *Review & accept recommendation* includes *Author automation rule* (an accepted recommendation becomes a rule), which itself includes the common command-validation and VND-estimation steps. As C01 requires, the Client and Customer are drawn *outside* the system boundary as business stakeholders, since neither the funding contractor nor the adopting management is itself a login actor — the customer interacts with the running system only in its Administrator role.

4.6 Use-Case Specifications (C06)

C06 requires that the principal use cases be detailed in specification tables giving preconditions, the main flow, alternate and error-handling flows, and postconditions. Three core use cases are specified below: controlling a device (UC-02), reviewing and accepting a recommendation (UC-05), and authoring an automation rule (UC-04).

4.7 Behavioural and Data Models

The modelling step of the requirements process also produces a data model and a set of behavioural models, which together fix the structure of the system's state and the lifecycles of its

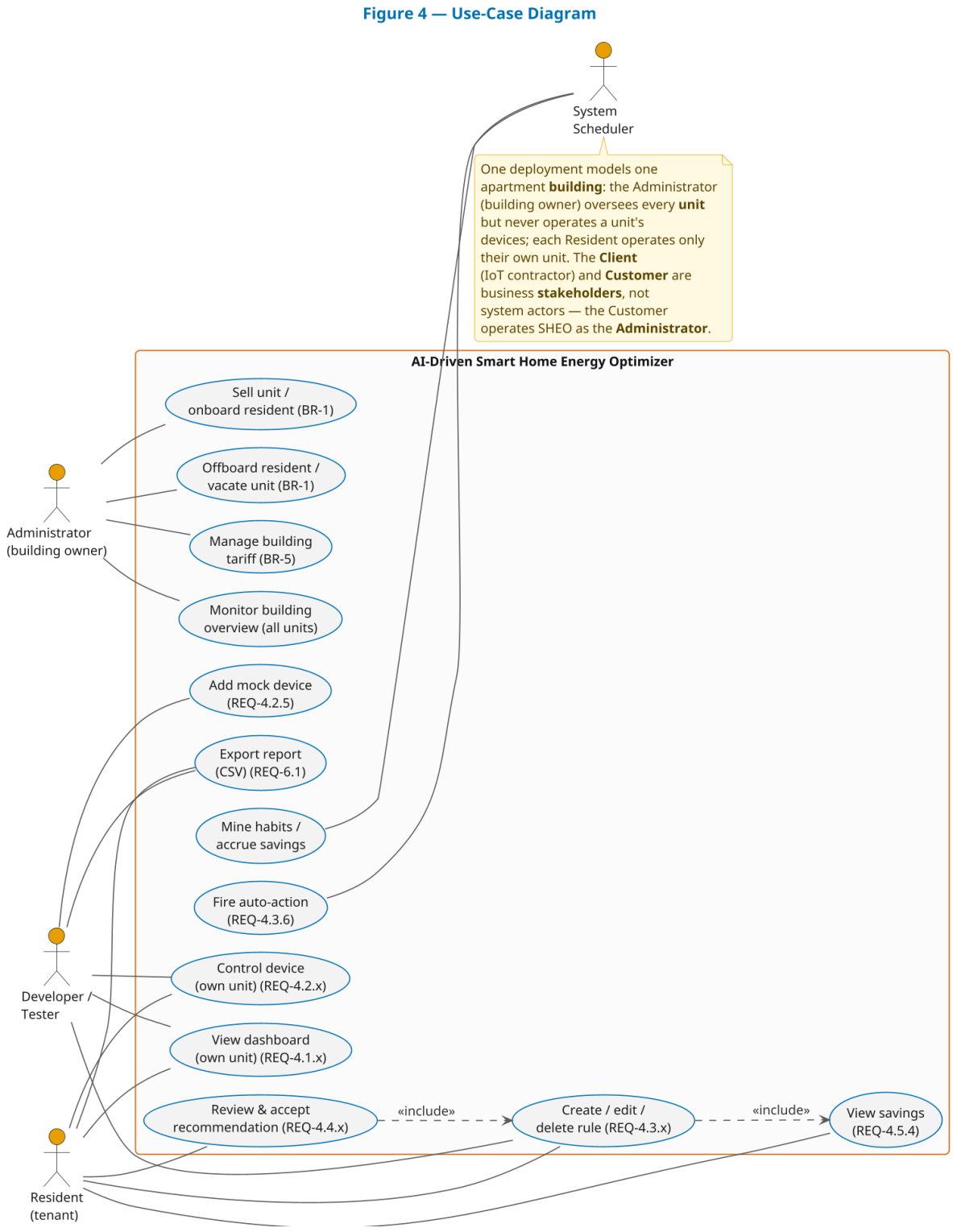

Figure 4.1. Use-case model. The deployment models one apartment building. The Administrator (building owner) monitors a building overview across every unit, configures the building-wide tariff, and onboards or offboards residents by selling a unit or returning it to vacant — but never manually operates a unit's devices and authors no automation rules; the Resident operates only their own unit's devices and may author rules and accept recommendations for that unit; the Developer/Tester exercises mock devices on a maintenance unit; the System Scheduler autonomously fires rules and mines habits. The Client (IoT contractor) and Customer (building management) are business stakeholders outside the boundary — the Customer operates the system *as* the Administrator. Accepting a recommendation ⟨⟨include⟩⟩ authoring a rule, which in turn ⟨⟨include⟩⟩ command validation and a VND estimate.

Table 4.4. Use-case specification — UC-02 Control device.

Field	Description
UC code	UC-02
Name	Control device
Primary actor	Resident
Secondary actor	— (the targeted device; the simulator stands in for hardware)
Preconditions	The actor is authenticated; the device exists in the actor's own unit; the actor holds control permission for it.
Main flow	1. The actor opens the device's detail view. 2. The system fetches the device's capability description (**REQ-4.2.1**). 3. The system renders the controls generically from each control's kind (toggle / range / enumeration). 4. The actor issues a command (e.g. set fan speed, set air-conditioner target temperature). 5. The system validates the command against the capability range and the safety guard (**REQ-4.2.3**). 6. The system applies the state change and records telemetry. 7. The system returns the outcome SUCCESS with the new state within 5 s (**REQ-4.2.4**).
Alternate flow	*A1 — Out-of-range or schema-mismatched command:* at step 5 validation fails; the system makes no state change and returns REJECTED with a clear message; flow ends. *A2 — Safety guard:* a safety-breaching command (e.g. exceeding the air-conditioner compressor rate limit) is returned as REJECTED / SKIPPED and nothing changes (**NFR-SAF-1**).
Error-handling flow	*E1 — Device offline:* if the device has been silent beyond the timeout, the system returns TIMEOUT, changes nothing, and surfaces an offline indication (**REQ-4.2.4**, **REQ-4.1.4**).
Postcondition	On SUCCESS the device is in the commanded state and a reading is logged; on REJECTED/TIMEOUT the device state is unchanged.

key objects. These are requirements-level models; their realisation in the database schema and the running services is treated in later chapters.

4.7.1 Data model

Figure 4.2 gives the entity-relationship model. The deployment is one apartment building; each *home* is a *unit* occupied by one Resident, and almost every entity is scoped to a unit, which is the basis of the unit-scoped access control demanded by **NFR-SEC-4**: a resident can never read another unit's readings or logs, while the Administrator (building owner, with no unit of their own) may view every unit. A tariff with no unit is the building-wide tariff. The model reifies the many-to-many control grant between users and devices as a device-permission entity (the basis of **BR-1**), and it deliberately keeps a rule's execution log independent of the rule so that deleting a rule preserves its history, as **BR-4** requires.

Table 4.5. Use-case specification — UC-05 Review & accept recommendation.

Field	Description
UC code	UC-05
Name	Review & accept recommendation
Primary actor	Resident
Secondary actor	System Scheduler (mines the habits that produce recommendations)
Preconditions	The actor is authenticated; at least one active recommendation exists, derived from ≥ 7 days of telemetry (**REQ-4.4.1**).
Main flow	1. The actor opens the Recommendations view. 2. The system lists active recommendations, ranked by VND saving, at most five (**REQ-4.4.3**). 3. For each, the system shows the readable `WHEN--THEN` rule, a rationale, the data window, and the monthly VND saving (**REQ-4.4.2**). 4. The actor selects *Accept*. 5. The system converts the recommendation into a real automation rule (⟨⟨include⟩⟩ UC-04) and marks the recommendation accepted (**REQ-4.4.4**).
Alternate flow	*A1 — Dismiss:* at step 4 the actor selects *Dismiss*; the system suppresses the same recommendation signature for at least 30 days and creates no rule (**REQ-4.4.5**). *A2 — Cap exceeded:* when more than five recommendations would be active, the lowest-saving surplus is expired to honour the cap.
Error-handling flow	*E1 — Insufficient data:* if fewer than 7 days of telemetry exist for a device, no recommendation is offered for it and the view explains why.
Postcondition	On accept, a new automation rule exists (auto-action off by default) and the recommendation is accepted; on dismiss, the recommendation is suppressed and no rule is created.

4.7.2 Behavioural models

Three finite-state machines specify the lifecycles of the system's most important objects, modelling the requirements as observable state transitions.

The rule lifecycle (Figure 4.3) captures **REQ-4.3.4** and **REQ-4.3.6**: a rule moves between Enabled and Disabled, is fired by the scheduler, and an auto-applied firing can be undone within two minutes.

The recommendation lifecycle (Figure 4.4) models **REQ-4.4.3**–**REQ-4.4.5**: a recommendation is advisory while Active and leaves that state only on an explicit user decision, becoming a rule on acceptance or suppressed for 30 days on dismissal.

The connectivity lifecycle (Figure 4.5) realises **REQ-4.1.4**: a silent device becomes Offline and is excluded from the live total until telemetry resumes. A complementary activity-style model of the end-to-end user journey — signing in, then choosing to control a device, review a recommendation, or author a rule — is presented with the user-interface design in the later chapter, since it is most naturally read alongside the screens it traverses.

Table 4.6. Use-case specification — UC-04 Author automation rule.

Field	**Description**
UC code	UC-04
Name	Author automation rule
Primary actor	Resident
Secondary actor	— (the Optimization service supplies the VND estimate)
Preconditions	The actor is authenticated and holds control permission for the target device.
Main flow	1. The actor opens the rule editor and chooses a target device. 2. The actor specifies the WHEN condition (time, day, tariff, state, or sensor) and the THEN action, optionally with an UNTIL stop condition (**REQ-4.3.1**). 3. The system validates the action against the device capability (**REQ-4.3.2**) and checks for conflicts with overlapping enabled rules (**REQ-4.3.3**). 4. The system computes and displays the VND saving estimate *before* the rule is saved (**REQ-4.5.3**). 5. The actor leaves auto-action off (default) or opts in, then saves. 6. The system persists the rule in the Enabled state.
Alternate flow	*A1 — Conflict detected:* at step 3 the system reports the conflicting rule and the time overlap; the rule is not silently saved until the actor resolves it. *A2 — Opt-in to auto-action:* the actor enables automatic application; the system records the opt-in and later fires the rule subject to the safety guard.
Error-handling flow	*E1 — Action exceeds capability:* a THEN action outside the device's capability is rejected at validation with a clear message; the rule is not saved.
Postcondition	A valid rule is stored (Enabled), carrying its VND estimate; on validation failure no rule is created.

Figure 8 — Entity-Relationship Diagram

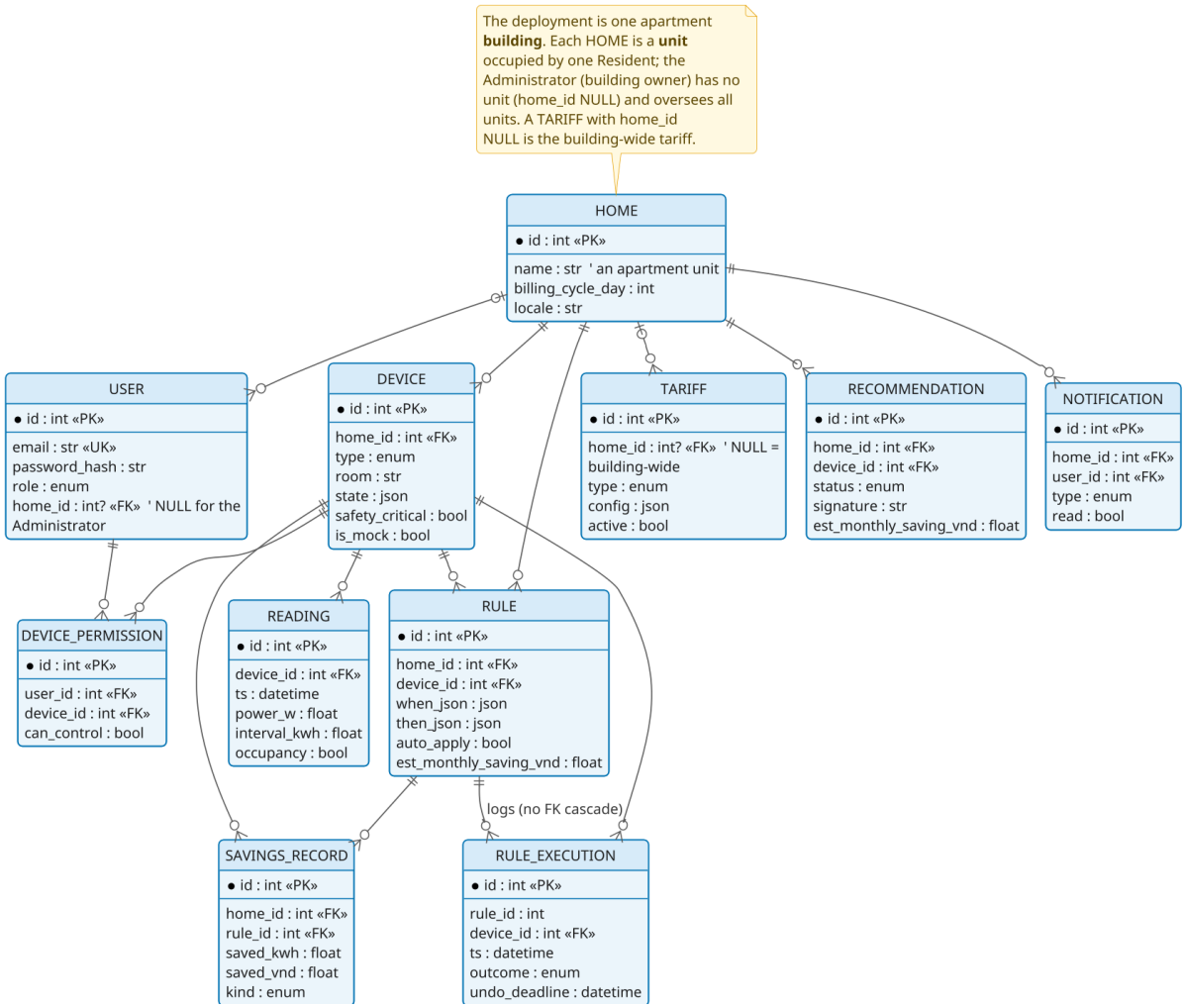

Figure 4.2. Entity-relationship diagram. The deployment models one building; each Home is a unit, and almost every entity is scoped to a unit (the basis of unit-scoped access control, **NFR-SEC-4**). The Administrator has no unit (nullable `home_id`) and oversees all units; a tariff with no unit is building-wide. The device-permission entity reifies the user–device control grant, and the rule-to-execution link carries no cascade so the execution log survives rule deletion (**BR-4**).

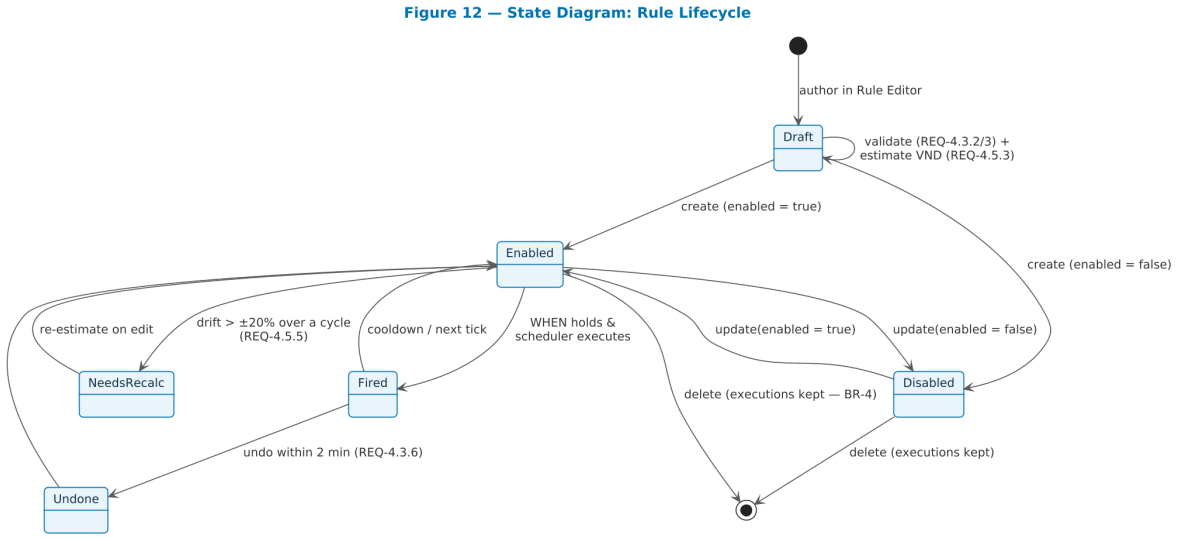

Figure 4.3. State machine — rule lifecycle. A rule is authored as a Draft, becomes Enabled or Disabled, transits a transient Fired state (auto-applied firings may be Undone within the 2-minute window), enters NeedsRecalc on ±20% drift, and on deletion is terminal yet preserves its execution history (**REQ-4.3**, **BR-4**).

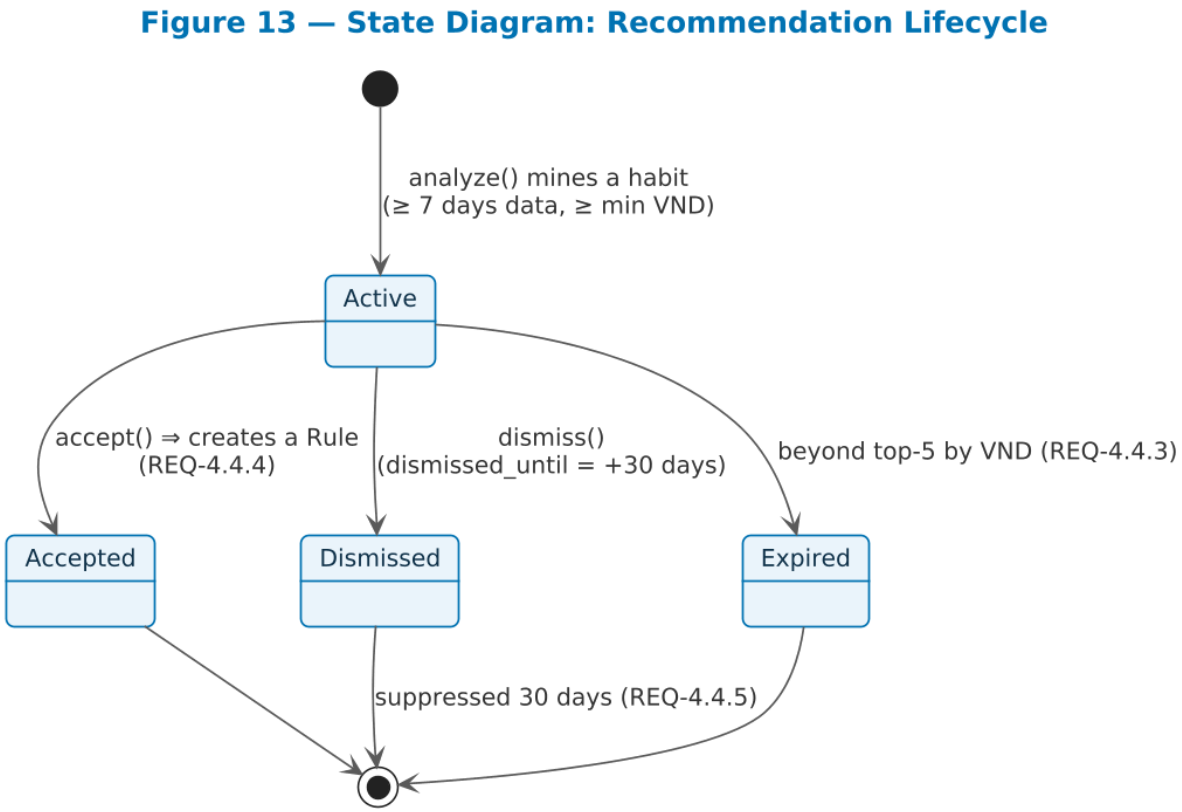

Figure 4.4. State machine — recommendation lifecycle. A mined recommendation becomes Active, then is Accepted (becoming a rule), Dismissed (suppressed for 30 days), or Expired when it falls outside the top-five cap (**REQ-4.4**).

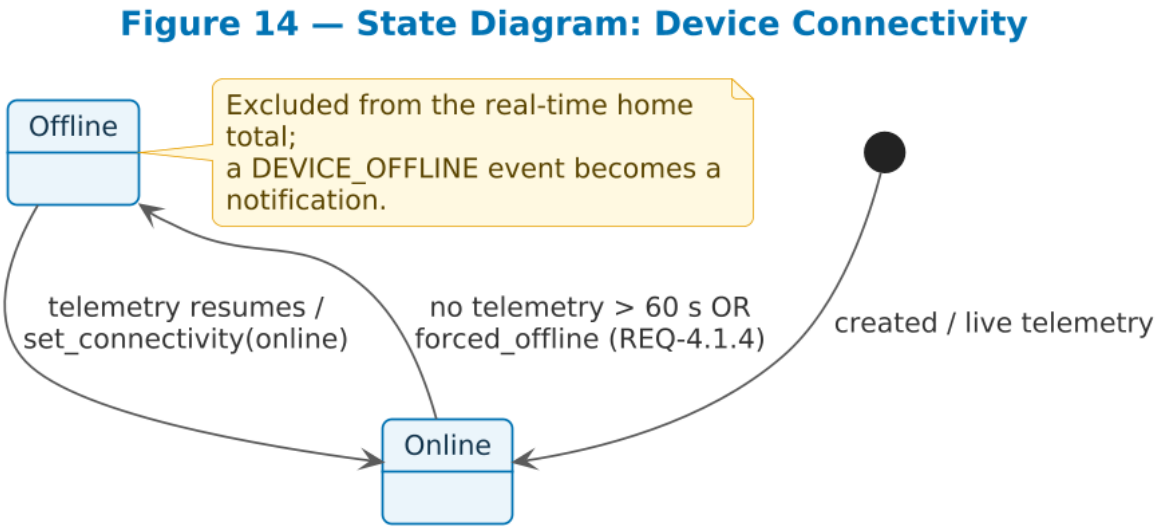

Figure 4.5. State machine — device connectivity. A device flips to Offline after more than 60 s of silence (or a forced offline) and back to Online when telemetry resumes; while offline it is excluded from the live home total (**REQ-4.1.4**).

Chapter 5

Software Architecture and Design

This chapter realises the first half of Lecture C07 (Software Design), and documents the structure as a software design description in the sense of IEEE 1016 [2]. Where the requirements chapter states *what* the AI-Driven Smart Home Energy Optimizer (SHEO) must do, this chapter states *how* the system is structured to deliver it: the design goals that govern every decision (§5.1), the architectural style and its alternatives (§5.2), the component and class decomposition (§5.3–§5.4), the design patterns that give the structure its flexibility (§5.5), the detailed behaviour of the two central control paths (§5.6), and finally the two subsystems that constitute the intellectual heart of the product (§5.7). Throughout, the design is presented in the conceptual vocabulary of the course, transforming the analysis model into a design model in the sense of ISO/IEC/IEEE 12207.

5.1 Design goals

The design is governed by the four fundamental concepts taught in C07: abstraction, modularity, the cohesion/coupling balance, and information hiding. These are not abstract ideals here; each is realised by a concrete mechanism in SHEO. The system is decomposed into modules that each own a single capability (high cohesion) and communicate through narrow, intention-revealing interfaces (low coupling), so that a change to one capability does not ripple across the others. Abstraction is applied twice over: device behaviour is abstracted behind a uniform adapter interface, and device *description* is abstracted into a declarative capability schema, so that neither the user interface nor the validation logic ever names a concrete device model. Information hiding is the organising principle of the data tier: every detail of persistence is sealed behind repository interfaces, so that business logic remains ignorant of, and therefore independent of, the storage technology.

Table 5.1 maps each design concept to the SHEO mechanism that embodies it. This mapping is what makes the qualities verifiable rather than merely asserted.

5.2 Architectural design

5.2.1 The architectural style

SHEO is a **three-tier client–server** application whose server interior composes three further architectural styles taught in C07, each selected to satisfy a specific quality goal:

1. **Three-tier client–server.** Presentation (a browser-based single-page application), application logic (a Python application server), and data (a relational store) are cleanly separated, giving independent deployability and a single, well-defined transport boundary over HTTPS and secure WebSockets.
2. **Layered architecture** inside the server. Dependencies point *strictly downward*: API routers depend on services, services on repositories, repositories on the domain model and the object-relational mapping. No router issues a database query and no service imports the web framework, so each layer is replaceable and unit-testable in isolation.
3. **Repository (data-centred) style** at the persistence boundary. Query construction is hidden behind intention-revealing methods, which also enforce unit-scoping for access control;

Table 5.1. Each C07 design concept and the SHEO mechanism that realises it.

Design concept	What it requires	SHEO mechanism
Abstraction	Suppress per-instance detail behind a general interface.	The device-adapter interface abstracts device behaviour; the capability schema abstracts a device type into data, so callers never branch on a concrete model.
Modularity	Partition the system into independently buildable, testable parts.	A layered backend (API → services → repositories → domain) plus one cohesive service per capability (devices, rules, optimisation, recommendations, telemetry).
High cohesion / low coupling	Each module has one reason to change; modules interact narrowly.	Each service owns exactly one capability; producers and consumers are decoupled by an in-process event bus, so a new consumer is added without touching producers.
Information hiding	Hide volatile design decisions behind stable interfaces.	Repositories hide all query construction and the storage engine; the capability schema hides device-type variance from both the front-end and the capability-validation routine.

a tariff repository additionally falls back to the building-wide tariff when a unit has no override.

4. **Event-driven (Observer) integration** inside the server. An in-process event bus decouples event producers (the simulator, the rule engine, the device service) from consumers (the live-feed broadcaster and the notification service), so new consumers can be added without modifying producers — a direct application of the open/closed principle.

The client itself follows an **MVC** flavour: the REST and WebSocket payloads are the Model, the page components are the View, and the authentication context together with the event handlers act as the Controller. The capability-driven control component is a pure View rendered over the capability schema, which serves as its Model. Figure 5.1 shows the resulting three-tier and layered structure, and the strict downward dependency rule that governs it.

5.2.2 The strict downward-dependency rule

The single invariant that keeps the layered design honest is that a layer may depend only on the layers beneath it, never on a peer or a layer above. The presentation tier knows the application tier but not the database; the application tier knows the repository abstraction but not the SQL dialect behind it; the data tier knows nothing of the web. This invariant is what makes the headline maintainability claim true: a change of storage engine is localised to the data tier, and a change of transport is localised to the presentation tier, because no business rule has been allowed to leak into either.

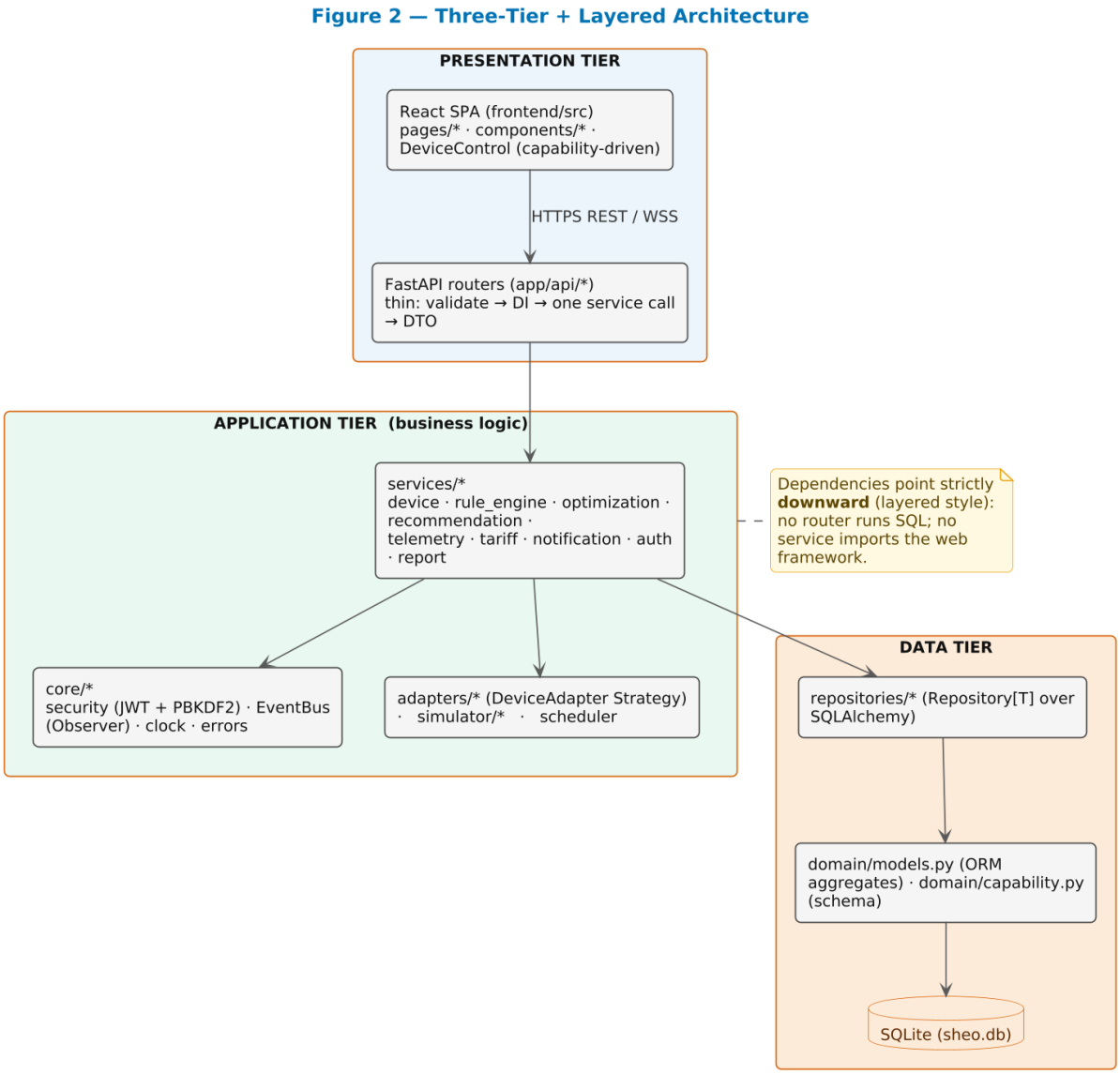

Figure 5.1. Three-tier and layered architecture. Dependencies point strictly downward (presentation → application → data); no router runs a database query and no service imports the web framework.

5.2.3 Alternatives considered

Two architectural decisions deserve an explicit record of the alternatives weighed, in the spirit of the C07 candidate-assessment step.

- *In-process event bus versus an external message broker.* A broker (for example a message queue or an MQTT bus) would permit horizontal scaling but would force an external infrastructure dependency that a single-node course demo cannot assume. SHEO instead runs its simulator and scheduler as in-process asynchronous tasks that communicate over an in-process publish/subscribe bus. Because producers and consumers already talk through that publish/subscribe seam, a future move to a real broker would replace only the transport, not the producers or consumers.
- *Layered monolith versus microservices.* Microservices were rejected as over-engineering for the scope. A layered monolith delivers most of the modularity benefit — clear seams, independent testability, localised change — at a small fraction of the operational cost, which is the correct trade-off on the function/cost/time triangle for a system of this size.

5.2.4 Deployment

The demonstration deployment is a **single application node** hosting the application server, together with a single embedded database file on a local volume; the browser is the only client node. The simulator and the scheduler are not separate processes but background tasks within the application's lifespan — which is exactly why the simulator can publish telemetry straight onto the same in-process bus that the WebSocket broadcaster listens on, with no broker in between. A production variant would swap the embedded database for a networked relational server and run several worker processes behind a TLS-terminating reverse proxy; the layered design localises that change to the data tier and the deployment topology, leaving the application logic untouched. Figure 5.2 shows this topology.

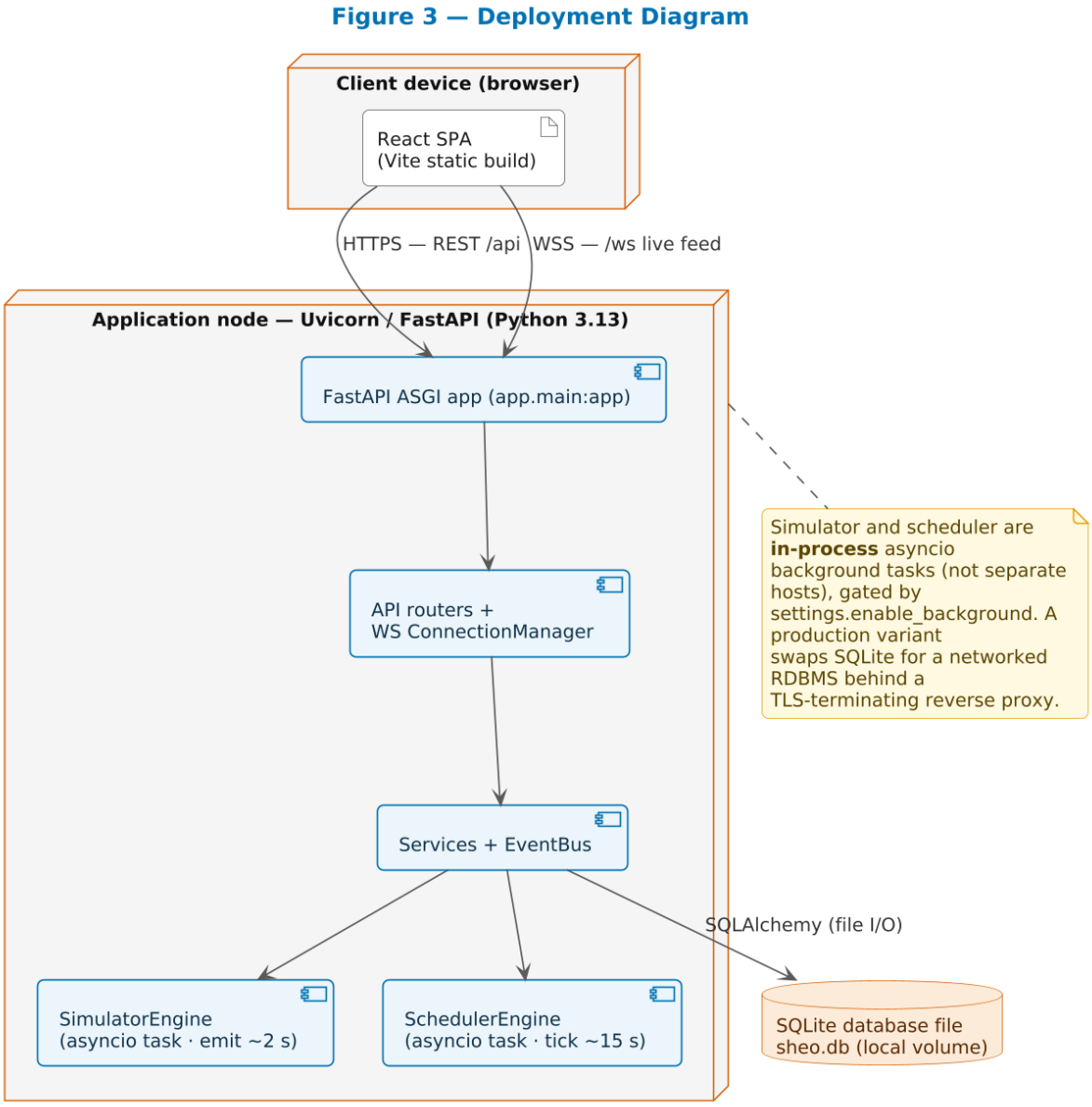

Figure 5.2. Deployment diagram. A single application node serves the browser-based front-end over HTTPS and secure WebSockets and persists to a local embedded database; the simulator and scheduler are in-process background tasks, not separate hosts.

5.3 Component decomposition

The decomposition opens the black box of the use-case view and divides the system along two orthogonal axes: *by tier/layer* (presentation, application, data), which yields the strict downward dependency flow, and *by cohesion*, so that each application-tier service owns exactly one capability and therefore has a single reason to change. Figure 5.3 shows the resulting package structure, listed in dependency order so that each package depends only on those below it.

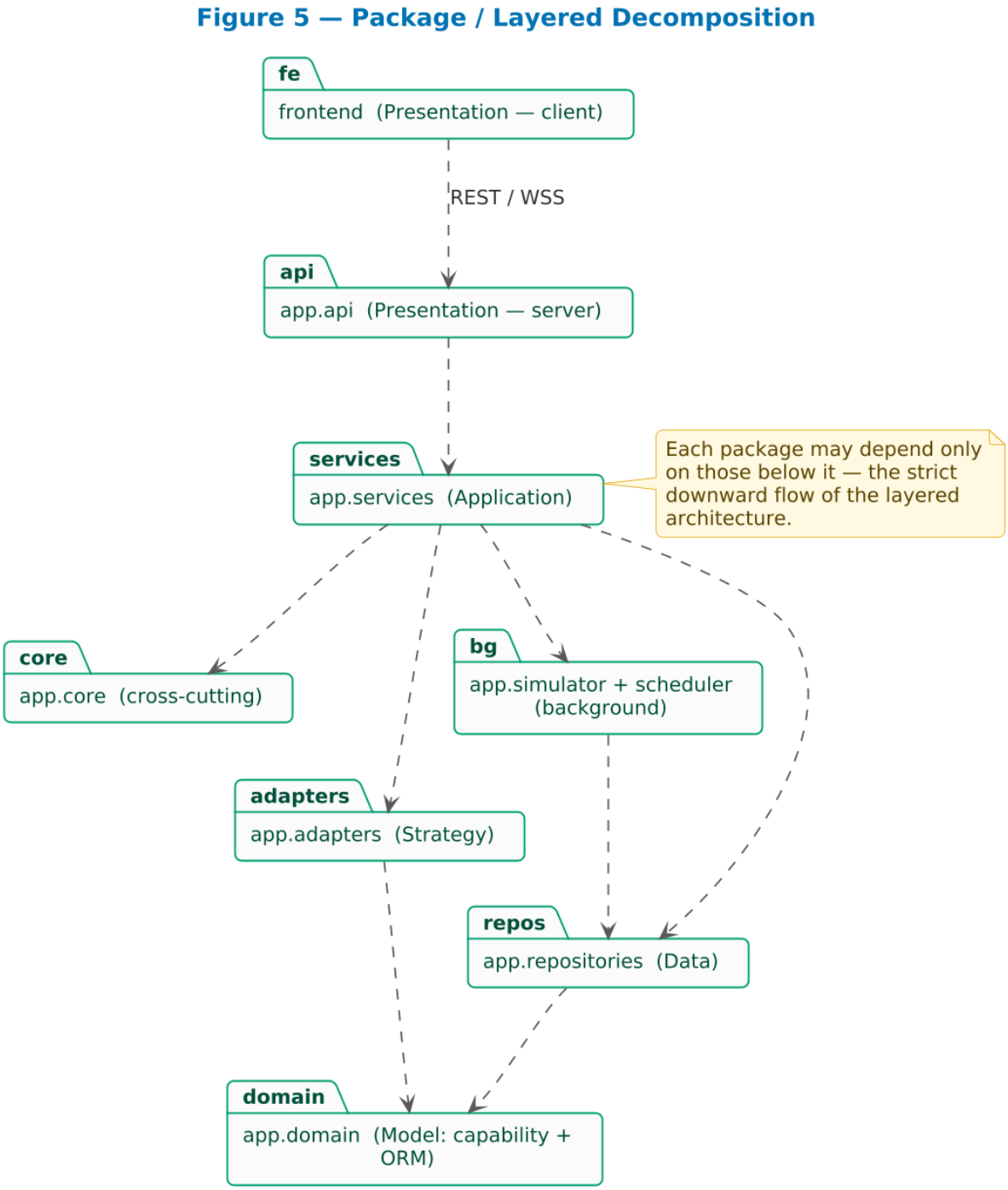

Figure 5.3. Package and layered decomposition. Each package may depend only on those below it — the strict downward flow of the layered architecture.

The principal packages are as follows. The **presentation layer** holds the thin routers and

the dependency-injection helpers; each router validates a request, resolves the authenticated user, calls exactly one service method, and returns a data-transfer object. A small set of Administrator-only, read-only routes (`GET /api/admin/overview` and the per-unit `/api/admin/units/{id}` drill-in routes) sits alongside the per-unit routers and is guarded by the role dependency. The **services layer** holds all business logic, one cohesive service per capability, including the Administrator service that assembles the building overview by reusing the per-unit telemetry service. The **cross-cutting layer** holds cross-cutting concerns: the event bus, the security primitives, a single clock source, and a framework-free domain-error hierarchy. The **device-adapter package** holds the device-adapter interface hierarchy that encapsulates per-device-type behaviour. The **simulator** and the **scheduler** are the two background workers — the former generating telemetry, the latter firing rules and accruing savings. The **persistence layer** is the only layer that issues database queries, and the **domain layer** holds the capability schema, the enumerations, and the object-relational aggregates that form the structural foundation.

Crucially, the front-end reaches the server through exactly **two ports** and no others: a **REST** port for request/response interaction, and a **WebSocket** port for the live feed of telemetry and notifications. This narrow contract is what allows the entire front-end to be developed and tested against a stable, documented surface. Figure 5.4 shows the component wiring: routers depend on services through dependency injection, services depend on repositories and adapters, and only the data tier touches the database. The event bus is the integration hub — the simulator and the device-control, rule, and recommendation services publish to it, while the notification service and the WebSocket connection manager subscribe.

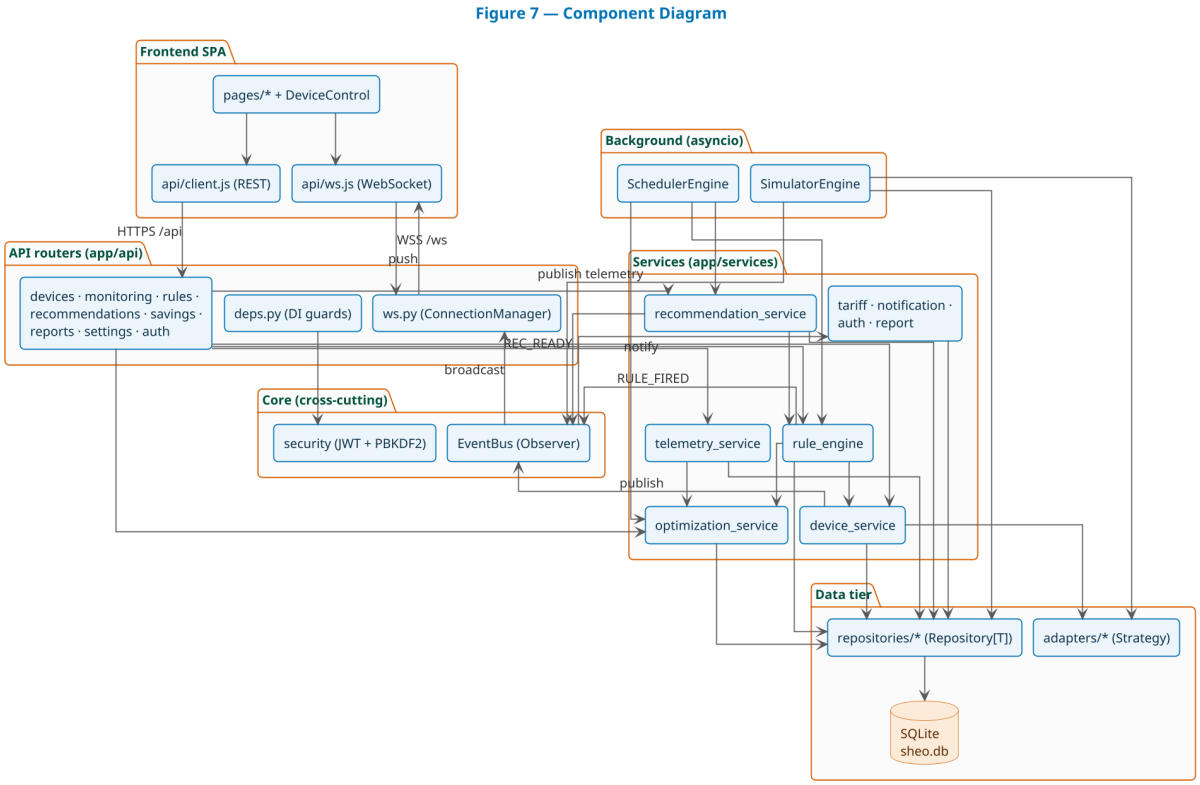

Figure 5.4. Component diagram. The front-end reaches the server only through the REST and WebSocket ports. The event bus is the integration hub: the simulator and the device-control, rule, and recommendation services publish, while the notification service and the WebSocket connection manager subscribe.

The component diagram above treats the front-end as a single box behind those two ports. Figure 5.5 opens that box and shows the front-end's own internal structure, which realises the

MVC flavour described in §5.2. The *Controller* is `App.jsx` (routing and the role gate) together with `auth/AuthContext.jsx` (the session, the bearer token, and the current role); the *View* is the navigation shell (`Layout.jsx`), the page components, the capability-driven `DeviceControl` component, and the shared UI primitives; and the *Model* is the data the client holds — the REST and WebSocket payloads, the capability schema that `DeviceControl` renders itself over, and the `lib/format.js` helpers for currency, palette, and time. Crucially, the only two arrows that leave the front-end are the same two ports as before: `api/client.js` (REST) and `api/ws.js` (WebSocket). This keeps the client's structure answerable to the same maintainability question as the server's class diagram — "to change behaviour X, which component do I touch?" — without widening the contract to the server.

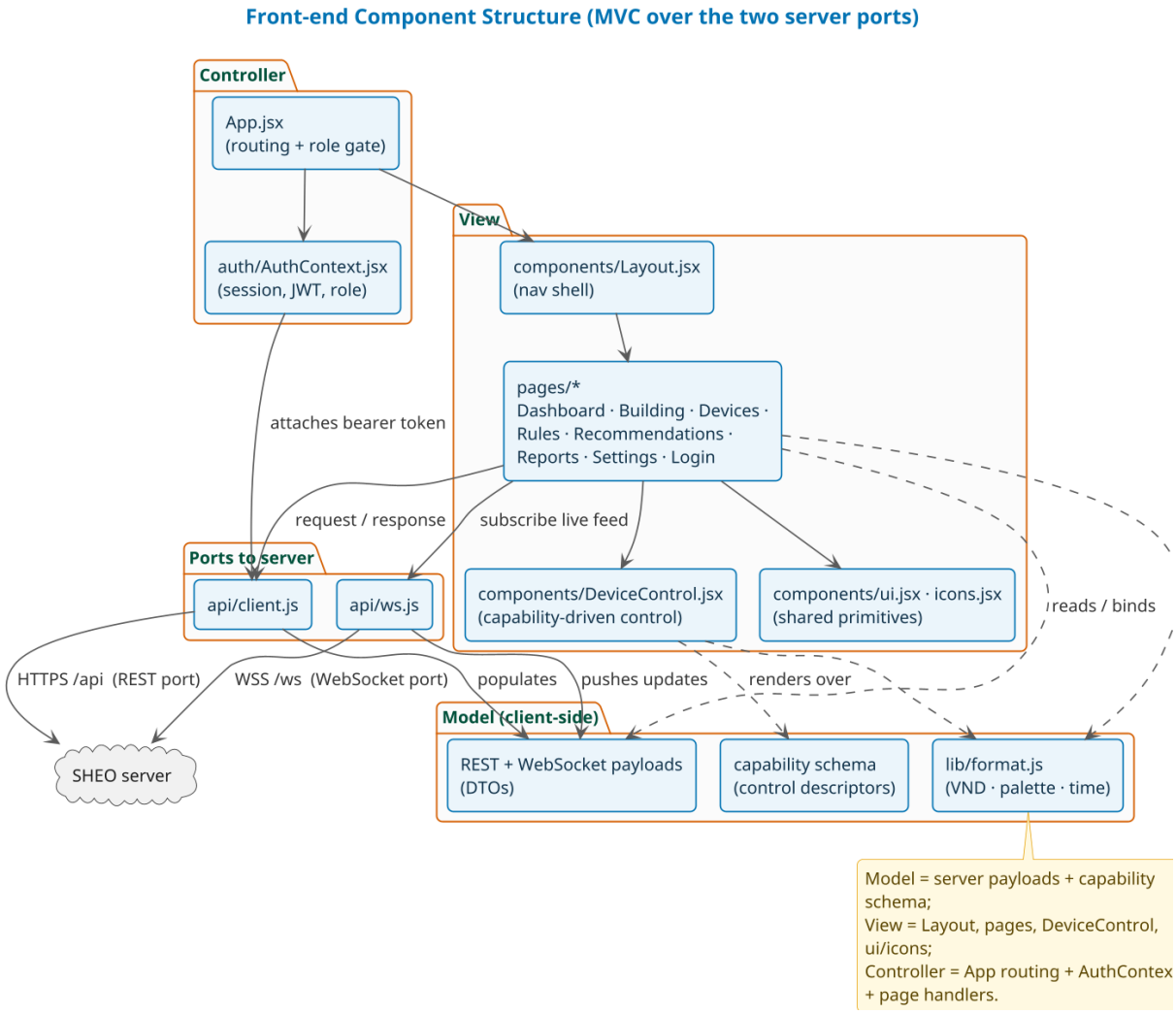

Figure 5.5. Front-end component structure. The single-page application follows an MVC split — `App.jsx` and `AuthContext` as Controller, `Layout`, the pages, `DeviceControl`, and the shared UI as View, and the server payloads plus capability schema as Model — and reaches the server only through the same REST and WebSocket ports as Figure 5.4.

5.4 Class design

The static model, shown in Figure 5.6, is organised around the **entity/boundary/control** separation taught in C07. *Entity* classes (the object-relational aggregates, with the unit (a Home occupied by one Resident) as the aggregate root) hold persistent state; *boundary* classes

(the routers and the capability-driven control component) mediate interaction with the outside world; and *control* classes (the services) coordinate the work. This separation keeps presentation, coordination, and state from contaminating one another.

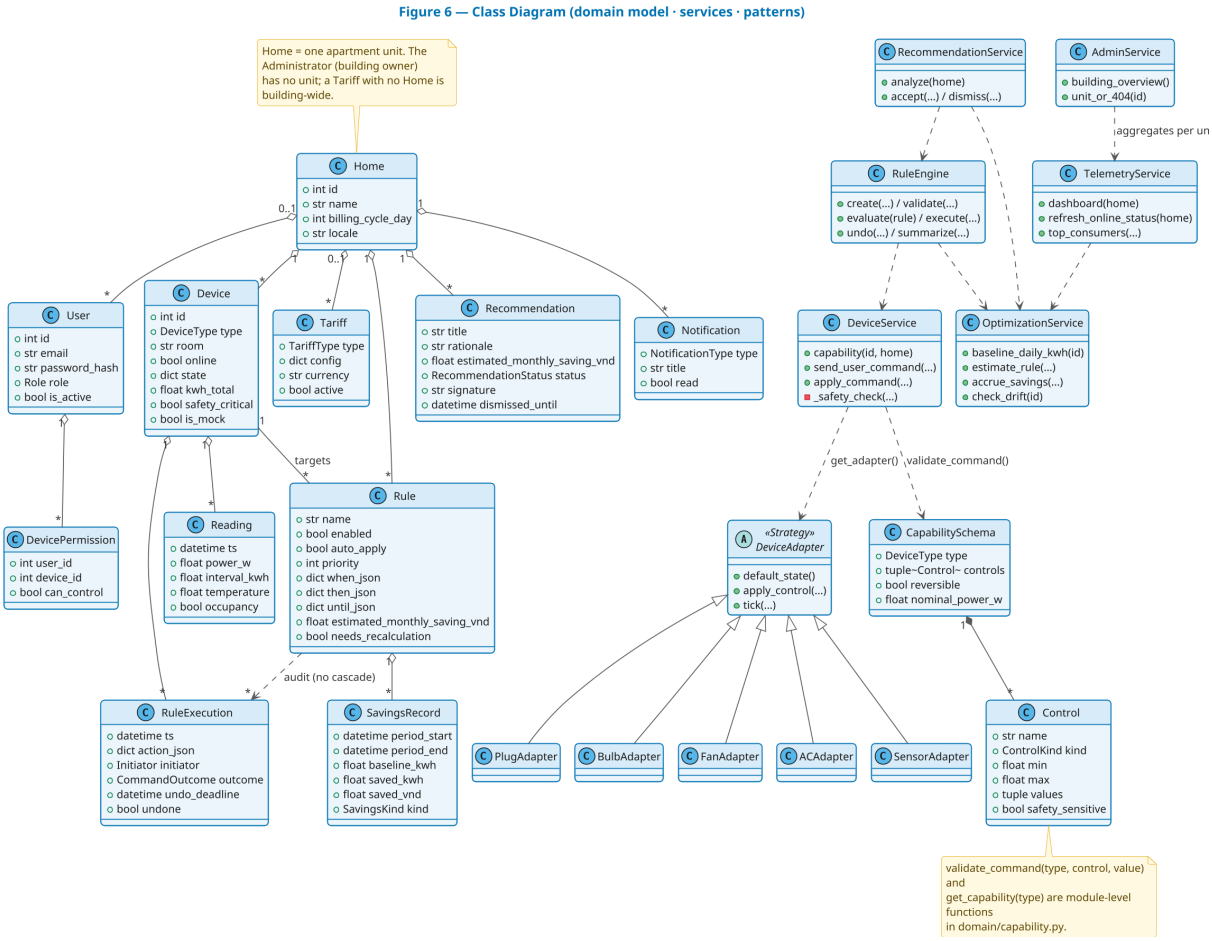

Figure 5.6. Class diagram. The unit (a Home occupied by one Resident) is the aggregate root; the device-adapter interface hierarchy and the capability-schema value objects drive device behaviour and command validation. The user–device control grant is reified by an explicit permission class, and the rule-to-execution dependency is deliberately non-cascading so that deleting a rule preserves its audit trail.

The control responsibilities are partitioned across six key services, each cohesive and singly responsible:

- **Device-control service** — the single, uniform pipeline for device control and the *single safety choke point*. Manual commands, rule-fired commands, and recommendation acceptances all flow through one command-application entry point, so the offline check, capability validation, and safety check cannot be bypassed by any caller. It returns a structured result indicating success, rejection, timeout, or a skipped no-op.
- **Rule engine** — authoring, validation, conflict detection, evaluation, execution, and undo of the WHEN–THEN–UNTIL automation rules. Validation checks an action against the device capability schema and detects time-overlapping opposing rules; execution is idempotent and rate-limited, dispatches through the device-control service only when auto-action is enabled, logs an immutable execution record, and supports a short undo window.
- **Optimisation/bill-saving service** — the deterministic, explainable estimation engine. It computes the historical baseline, estimates a rule's monthly saving in Vietnamese đồng (VND), accrues measured savings into the current billing cycle, and flags rules whose measured saving drifts from the estimate.

- **Recommendation service** — orchestrates the recommendation lifecycle: it delegates habit *detection* to a swappable **recommendation-provider port** (the client's "AI", defaulting to a deterministic, explainable miner), then performs the parts that are SHEO's own software — pricing each candidate in VND via the optimisation service, ranking and capping, turning an accepted suggestion into a rule, and suppressing a dismissed one by signature. A black-box model can replace the default provider behind the same port without affecting the VND estimator or any other layer.
- **Telemetry service** — produces the dashboard aggregate, the unit-scoped consumption time-series, the top-consumers ranking, and the online/offline refresh that marks a device unreachable after a silence threshold.
- **Administrator service** — serves the building owner's read-only oversight. It builds the *building overview* (the resident roster with each unit's live power, cycle kWh, and bill, plus building-wide totals) and resolves a unit for the Administrator-only per-unit drill-in, raising a not-found error for an unknown unit. It reuses the per-unit telemetry service rather than duplicating its aggregation (DRY), and authors no rules and operates no devices.

5.5 Design patterns

Five design patterns give the structure its flexibility; each is realised by a concrete component and, where natural, advances a SOLID principle. Strategy, Observer, and State are classical Gang-of-Four (GoF) behavioural patterns; Repository and Dependency Injection are architectural patterns from Fowler's *Patterns of Enterprise Application Architecture* and the Dependency-Inversion Principle respectively. Table 5.2 records the pattern, its intent, and its realisation in SHEO.

5.6 Detailed behaviour

Two sequence diagrams capture the dynamic behaviour of the system's two principal control paths. Figure 5.7 traces the **capability-driven control** path. When a user operates a device, the front-end first fetches the device's capability schema and renders the appropriate control widget from it; the resulting command is sent to the server, where the device-control service authorises the caller, validates the command against the same capability schema, runs the safety check, asks the device's strategy adapter to apply the state change, publishes a *command-applied* event, and returns a structured result. The WebSocket broadcaster, subscribed to the bus, then pushes the new state to every connected front-end. The same path serves manual, rule-fired, and recommendation-driven commands, which is what makes the safety guarantee non-bypassable.

Figure 5.8 traces the **automatic rule-firing** path driven by the scheduler. On each tick the scheduler asks the rule engine to evaluate each enabled rule; when a rule's WHEN condition holds and the device is not already in the target state, the engine dispatches the action through the device-control service (only if auto-action is enabled for that rule), writes an immutable execution record, and publishes a *rule-fired* event. The notification service, observing that event, persists a notification and re-publishes it for the broadcaster, so the user sees the automated action and retains a short window in which to undo it.

5.7 The two heart subsystems

Two subsystems carry the product's distinctive value and therefore warrant detailed treatment: the capability schema, which makes the system extensible without per-model code, and the bill-saving estimation algorithm, which makes every saving figure transparent and verifiable.

Table 5.2. Design patterns realised in SHEO, with the SOLID principle each advances. Strategy, Observer, and State are GoF behavioural patterns; Repository is an architectural pattern; Dependency Injection follows the Dependency-Inversion Principle.

Pattern	Intent	Realisation in SHEO
Strategy (GoF)	Encapsulate a family of interchangeable behaviours behind one interface.	The device-adapter interface hierarchy (one strategy per device type) encapsulates each type's command semantics and power model, so no code branches on type. Adding a device type adds a strategy without editing callers — the open/-closed principle (OCP). The same pattern isolates the "AI": habit detection sits behind a *recommendation-provider* port, so the default explainable miner and a future black-box ML provider are interchangeable strategies, swapped at the composition root.
Observer (GoF)	Notify dependents of state changes without coupling them to the source.	The in-process event bus: producers publish typed events; the notification service and the WebSocket broadcaster subscribe. New observers are added without touching publishers.
Repository (Fowler)	Mediate between the domain and the data source via a collection-like interface.	A generic repository plus one repository per aggregate hides all query construction and the storage engine, keeping business logic persistence-agnostic.
Dependency Injection (DIP)	Supply a component's collaborators from outside rather than constructing them within.	The dependency-injection helpers provide a ready database session, the current user, and role guards to each router. Routers depend on abstractions rather than concrete constructions — a direct application of the Dependency-Inversion Principle, and the structural basis of role-based access control.
State (GoF)	Let an object alter its behaviour as its internal state changes.	The rule and execution lifecycles are modelled as explicit states (for example a rule moving through draft, enabled, and disabled; an execution through applied, undone, and expired), so behaviour follows the lifecycle rather than scattered conditionals.

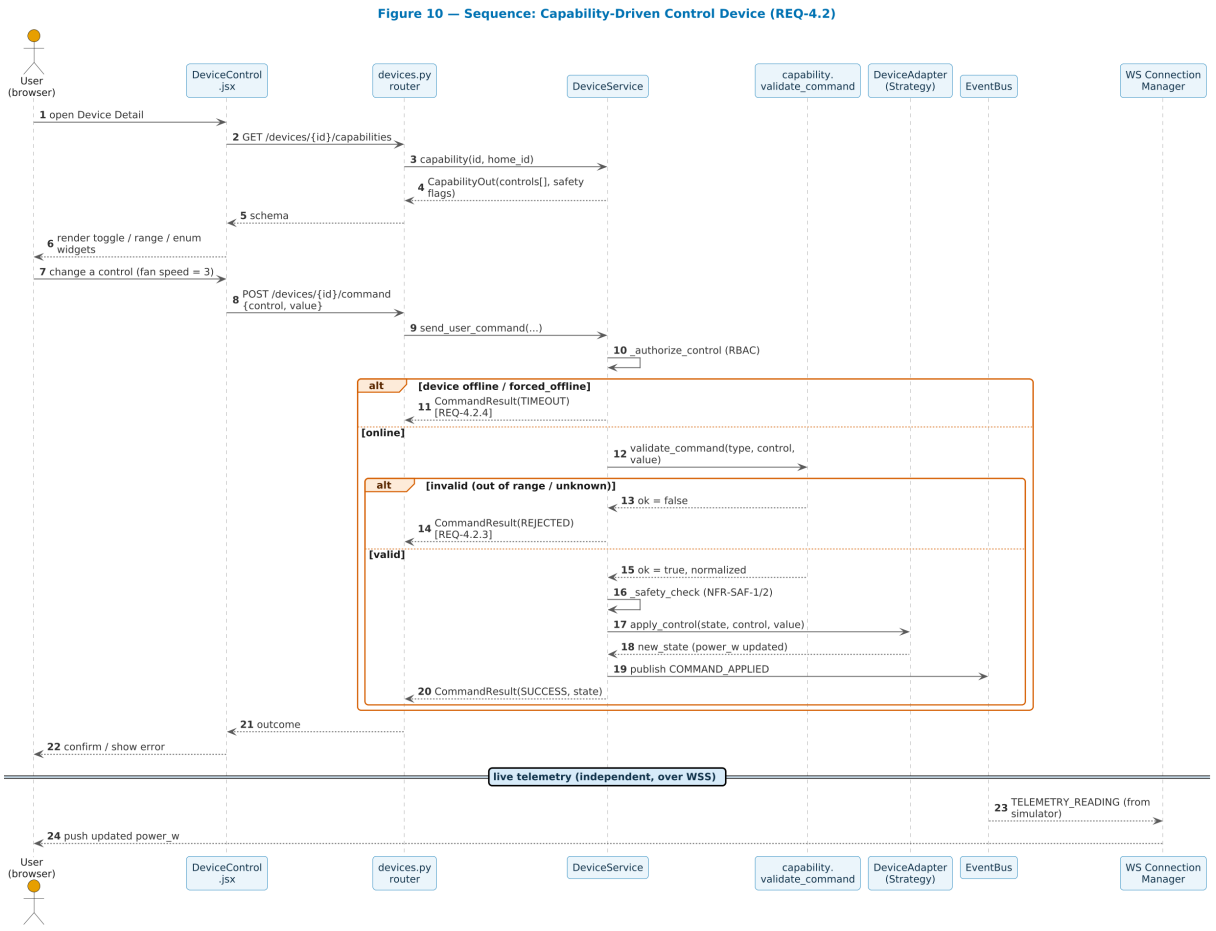

Figure 5.7. Sequence: capability-driven device control. The command is validated against the same capability schema used to render the control, then applied through the device-adapter interface, and the new state is broadcast over the event bus to all connected front-ends.

5.7.1 The capability schema: a single source of truth

The capability schema is the design device that lets SHEO support an open-ended set of device types without writing a screen or a validator for each. A device type is described declaratively as data, not as code branches. A *control* value object describes one controllable feature by its name, label, and *kind* — one of toggle, range, enumeration, or read-only — together with the bounds (minimum, maximum, step), the permitted values, the default, and a flag marking the control as safety-sensitive. A *capability schema* lists a device type's telemetry channels and its controls, plus whether the type is reversible, whether it is safety-critical by default, and its nominal power draw. A registry holds exactly one schema per device type.

The decisive property is that this single description **drives two consumers at once**. The front-end renders its control widgets directly from the schema — a toggle for a toggle control, a slider for a range, a segmented selector for an enumeration — so the user interface never names a concrete device model. The server validates every incoming command against the *same* schema before dispatching it, using the same capability-validation routine on both the manual and the rule-fired path. Because rendering and validation read one definition, they can never disagree: a control the interface offers is exactly a control the server accepts, with exactly the bounds the interface enforced. Adding a new device type is therefore a two-element change — one schema entry and one device adapter — with no edit to any router, service, or screen.

Table 5.3 illustrates the structure of a schema entry for a representative device type (a room air-conditioner), showing the three controllable features it exposes.

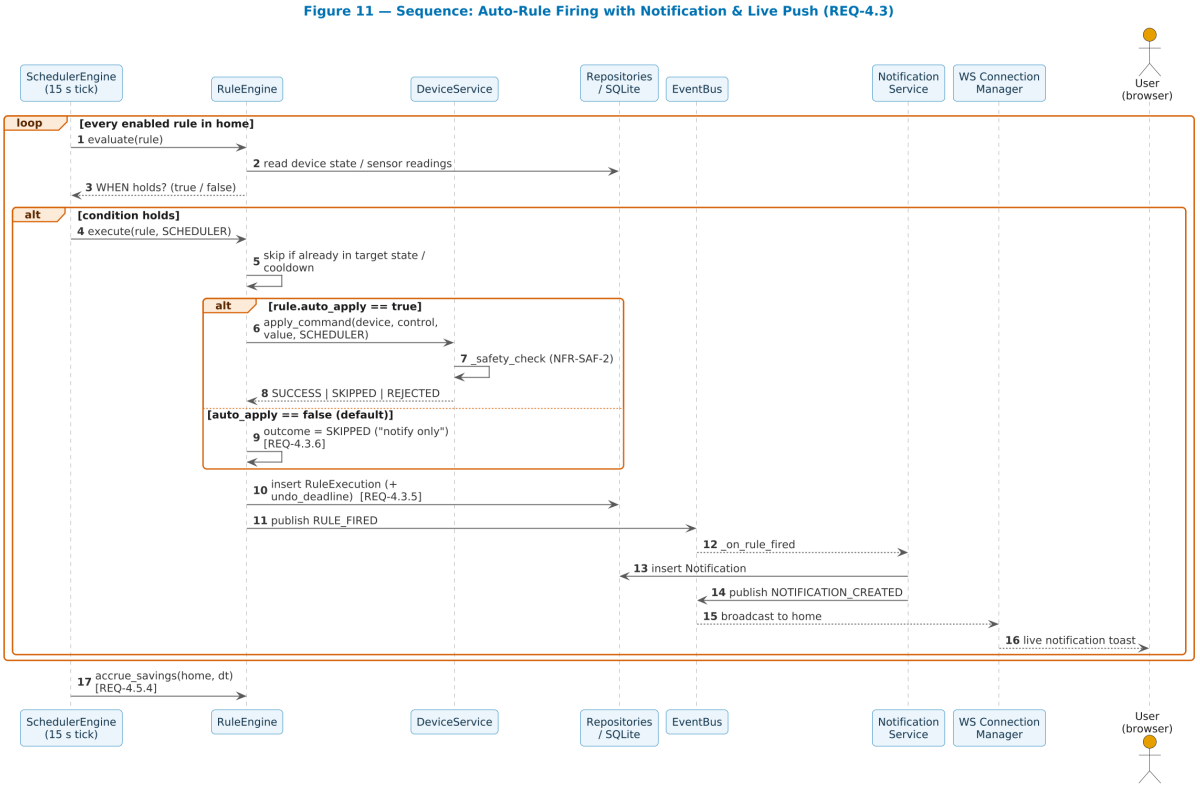

Figure 5.8. Sequence: automatic rule firing. The scheduler evaluates an enabled rule; when its condition holds, the engine dispatches through the same control pipeline, logs an immutable execution, and the firing is observed by the notification service and broadcast to all connected front-ends.

Table 5.3. Capability schema entry for a room air-conditioner. Each row describes one controllable feature: its kind determines how the front-end renders the widget; its bounds constrain the accepted values; the safety flag causes the capability-validation routine to apply an additional safety check before dispatch.

Control	Kind	Bounds / permitted values	Safety-sensitive
Power	Toggle	{on, off}; default: off	No
Target temperature	Range	16–30 °C, step 1 °C; default: 26	Yes
Operating mode	Enumeration	{cool, fan, dry, auto}; default: cool	No

5.7.2 The bill-saving estimation algorithm

The second heart subsystem is the explainable estimation of a rule's monetary saving. It is deliberately kept inside SHEO — never behind the recommendation-provider's black box — so the figure the client values most stays transparent. Every estimate reduces to one formula, summed over the hours a rule affects:

$$\text{saved\_vnd} = \sum_{h \in \text{affected hours}} \left(\text{baseline\_kWh}_h - \text{expected\_kWh\_with\_rule}_h \right) \times \text{tariff\_VND\_per\_kWh}.$$

The estimate is computed in three transparent steps.

1. **Baseline.** For the device in question, the system computes an *hourly* profile of average energy use — the mean kilowatt-hours consumed in each hour of the day — over the previous **14 days**. Summing the profile over the hours the rule affects gives the baseline energy that the rule will act upon.
2. **Expected use with the rule.** A transparent per-action model derives what the device would draw *if the rule were in force*. Switching a device off yields zero expected energy; raising an air-conditioner target by a degree applies a fixed saving fraction per degree (capped); reducing a fan speed or a bulb brightness scales the energy by the ratio of the new setting to the current one. Each rule of thumb is documented, so a reader can reproduce the figure by hand.
3. **Pricing.** The avoided energy — the baseline minus the expected use, floored at zero — is multiplied by the effective tariff price in VND per kilowatt-hour, resolved from the building-wide tariff set by the Administrator (flat, tiered, or time-of-use), falling back to it when a unit has no override. The daily saving is projected to a month for display.

The design property that matters most is *when* the figure is shown: the estimated VND saving is presented to the user **before a rule is accepted**, so that a saving is a promise the user can weigh, not a surprise discovered later. Each estimate is accompanied by a plain-language explanation naming the baseline, the expected use, and the price, closing the loop on the explainability constraint. After a rule is in force, the scheduler accrues the *measured* saving per billing cycle and flags any rule whose measured saving drifts materially from its estimate, so the promise is subsequently checked against reality.

Chapter 6

User Interface Design

This chapter describes the user interface design of SHEO, following the five-step process taught in C07 Part 2: identify the *context of use* → derive *UI requirements* → produce a *design* (information architecture and screen layouts) → build a *prototype* → conduct *usability evaluation*. Each section below corresponds to one step and maps its findings to the system's screens, navigation structures, and interaction patterns.

6.1 UI Design Process

The interface design process followed in SHEO is the five-step model from the C07 curriculum:

1. **Identify context of use.** Record who the users are, what environment they work in, and on what platform — before any screen is drawn (Section 6.2).
2. **Derive UI requirements.** Translate the context, user goals, and the seven golden rules into measurable usability targets and interaction constraints (Section 6.3).
3. **Design.** Produce an information architecture (sitemap and user flow) and low-fidelity wireframes for each screen (Sections 6.4 and 6.5).
4. **Prototype.** Realise the wireframes as a working React single-page application; the live implementation is the executable prototype.
5. **Evaluate.** Assess the prototype against the golden rules (heuristic walkthrough) and against measurable task criteria (task-based test), then feed findings back into the design (Section 6.6).

The process is intentionally iterative: the heuristic pass (step 5 applied to the initial design) surfaced two concrete improvements that were folded back into step 3 — surfacing the VND saving estimate *before* the Save button is enabled, and adding a safety pill on safety-sensitive controls.

6.2 Context of Use and User Profiles

6.2.1 Context of use

SHEO is deployed for a single apartment building, whose units are each occupied by one resident. Residents interact with it through a personal computer or a mobile-phone browser during ordinary domestic hours rather than during a fixed office shift, while the building owner uses it occasionally to oversee the building. The platform is a **browser-hosted single-page application** (SPA): there is no native install, and interaction is by mouse and keyboard on desktop or by touch on mobile. The information handled during a typical session includes live device power and energy figures, estimated and measured savings in Vietnamese dong (VND), automation rules, AI-derived recommendations, and the building-wide electricity tariff.

Following the C01 stakeholder distinction, the project separates *business stakeholders* who commission and fund the build from *system actors* who operate the running software. The Client (IoT contractor) falls in the first category and does not log in. The Customer (building owner or landlord) operates SHEO as the Administrator actor, who owns no unit of their own; the tenants of the individual units operate it as Resident actors.

6.2.2 User profiles

Table 6.1 summarises the three operative user classes, their characteristics, their primary interaction goals, and the consequence of an error for each class. The consequence column drives the protection mechanisms described in Section 6.3.

Table 6.1. User profiles for SHEO.

User class	Characteristics	Goals / interaction intent	Consequence of an error
Administrator (Customer)	Building owner or landlord; owns no unit of their own; non-technical; occasional user; motivated by the building's electricity bill.	Oversee all units through a read-only building overview; set the building-wide tariff and billing cycle; sell, onboard, and offboard units. Never operates devices and authors no rules.	**High** — sets the building-wide tariff and onboards/offboards units. Safeguarded by explicit confirmations and by read-only oversight of units.
Resident (end-user / tenant)	Tenant of one unit; mixed ages and abilities; views and controls only their own unit's operable devices; sets up personal automation rules.	Check own unit's consumption and savings; operate own unit's devices; author automation rules; accept or dismiss recommendations.	**Low–Medium** — can author rules (auto-action off by default) and control own unit's devices; cannot change the tariff or access another unit (**NFR-SEC-4**).
Developer / Tester	Technical; prepares and reproduces demos.	Instantiate mock devices, exercise edge cases, force a device offline.	**Contained** — changes are limited to the mock world.

6.3 UI Requirements and the Seven Golden Rules

6.3.1 The seven golden rules applied to SHEO

The C07 UI curriculum specifies seven golden rules of interface design. Table 6.2 lists each rule and its concrete realisation in SHEO.

Table 6.2. The seven UI golden rules and their realisation in SHEO.

Golden rule	Realisation in SHEO
Easy to learn	The left navigation exposes every primary task from a single persistent menu; the dashboard lands on the most important information (live power and cycle savings) without any configuration. A first-time Resident can operate a device in their unit and see live telemetry within a few minutes.

(Table 6.2 continued)

Golden rule	Realisation in SHEO
Familiar terms	Domain language mirrors what residents already know: "kWh today", "estimated bill", and "savings so far this cycle" are expressed in the user's billing vocabulary rather than in software abstractions. VND is used throughout for monetary figures rather than a generic currency symbol.
Consistency	The **capability-driven control widget** is the central consistency benefit. Because one generic component renders every device type from the capability schema (**REQ-4.2.1**), the toggle, range slider, and segmented-enum interactions behave identically regardless of whether the user is controlling a plug, a bulb, a fan, or an air-conditioner. There is no per-device-type UI code to learn or to diverge from each other.
Minimal surprise	All destructive and automated actions are gated: auto-action on automation rules is *off by default* (**BR-3**); the rule editor shows the VND estimate and any conflict warning *before* the Save button is enabled; the Settings screen is read-only for non-Administrators.
Recoverable	Two explicit recovery mechanisms are provided. First, automation rules require the user to opt in to auto-apply; absent that opt-in the system notifies but takes no action, leaving the user in control. Second, any rule firing that *is* auto-applied can be undone within a two-minute window (**REQ-4.3.6**). Out-of-range or offline commands are rejected cleanly without changing device state.
Feedback and help	Every command returns an explicit three-valued outcome: SUCCESS, REJECTED, or TIMEOUT. The rule editor provides live validation and conflict warnings as the user types. Offline devices are badged on the dashboard. The recommendations panel shows a rationale and a data-window label so users understand why each suggestion was generated. WebSocket push ensures the dashboard reflects reality within five seconds (**NFR-PER-1**).
Diverse users	The interface uses the Okabe–Ito colour-blind-safe palette throughout; the dashboard prominent headline-metric (large-numeral) pattern renders the key saving figure large enough to read at a glance; domain wording mirrors what residents already know; and the role-based access model ensures each actor sees only the controls relevant to their role, reducing cognitive load.

6.3.2 Measurable usability targets

In addition to the qualitative golden-rule analysis, the following measurable usability targets were established from the context-of-use analysis and serve as the acceptance criteria for the task-based evaluation in Section 6.6:

- A user shall be able to send a control command to a device in ≤ 3 clicks from the dashboard (**REQ-4.2.x**, **NFR-PER-2**).
- UI feedback on every command outcome shall appear within the two-second budget (**NFR-PER-2**).
- A user shall be able to review a recommendation's rationale and VND figure and act on it in ≤ 2 clicks.
- The rule editor shall present the VND estimate and any conflict warning before the Save button becomes active (**REQ-4.5.3**).
- A first-time Resident shall be able to operate a device in their unit and see live telemetry within a few minutes (**REQ-4.2.5**, **REQ-4.1.1**).

6.3.3 Colour and accessibility

The Okabe–Ito palette is applied to all charts and status indicators. The seven Okabe–Ito colours are distinguishable for the three most common types of colour-vision deficiency (deuteranopia, protanopia, and tritanopia), satisfying the "diverse users" golden rule at the perceptual level.

6.4 Information Architecture and Navigation

6.4.1 Sitemap

The application has a flat information architecture accessed through a persistent left navigation bar, with a device-detail sub-view opened from the devices screen. The navigation is **role-aware**: a Resident or Developer sees the unit-scoped Dashboard, Devices, Rules, Recommendations, and Reports screens, whereas the Administrator sees a role-specific **Building** page (the units roster with totals and a read-only drill-in) in place of the single-unit Dashboard. After authentication, every primary task for the actor's role is reachable in one click from the navigation, and every primary task other than Settings is reachable within one additional click from there. The flat hierarchy minimises navigation depth, directly supporting the "easy to learn" golden rule. Figure 6.1 shows the complete sitemap.

The Settings screen deserves special note: the building-wide tariff configuration and unit-onboarding panels are visible to all authenticated users but are editable only by the Administrator (**BR-1**). Non-Administrators see the same layout in a read-only state rather than a different screen, preserving layout consistency ("minimal surprise").

6.4.2 User flow

The sitemap describes the static screen hierarchy; the user flow in Figure 6.2 describes the dynamic paths a user takes to accomplish the three core journeys: controlling a device, acting on a recommendation, and authoring an automation rule. The diagram makes the key decision points explicit: the capability fetch that precedes rendering, the three-valued command outcome, the accept-or-dismiss branch on recommendations, and the live validation gate on the rule editor.

6.4.3 Three core journeys

Journey 1 — Monitor and control a device.
From the Dashboard the user selects a device tile, the front-end requests the device's capability description from the server, and the control widget is rendered generically. The user

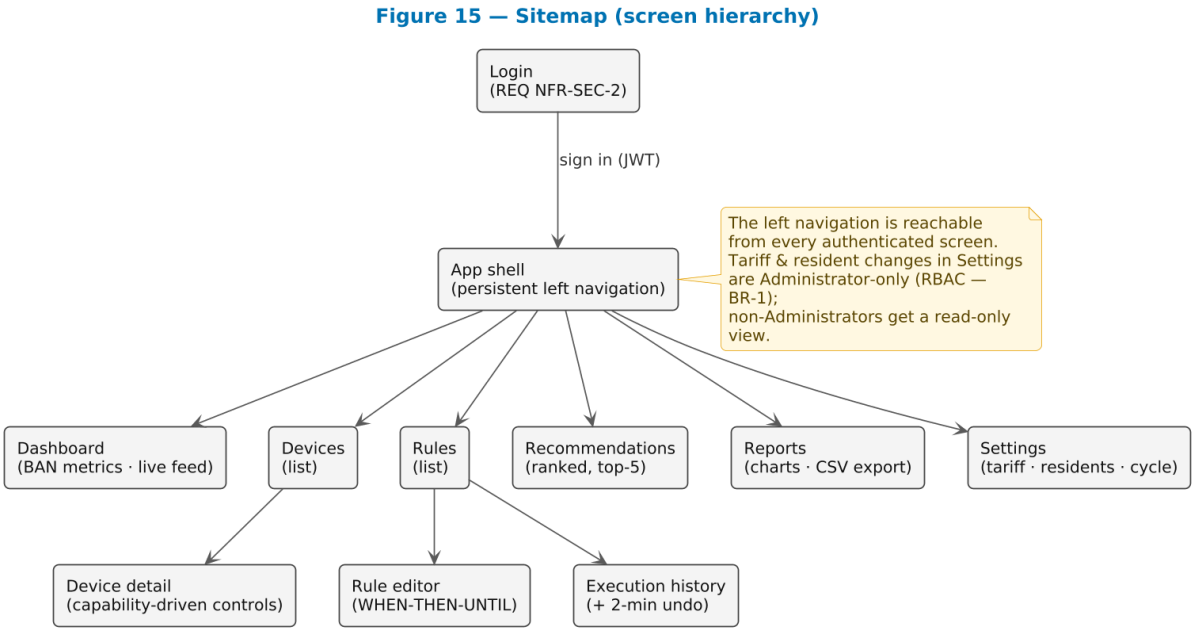

Figure 6.1. Sitemap of the SHEO single-page application. After sign-in the shell exposes six top-level screens through the left navigation, plus a device-detail sub-view opened from the devices screen. The rules screen opens a rule-editor overlay; the settings screen is read-only for non-Administrators (role-based access model, **BR-1**). Every primary task is at most one click from the dashboard.

adjusts a control and the command flows through the command-application pipeline, which returns SUCCESS, REJECTED, or TIMEOUT with a human-readable message.

Journey 2 — Review and act on a recommendation.
The Recommendations screen presents up to five ranked suggestions, each showing a human-readable *when…then…* rule expression, a rationale, the data window over which the habit was mined, and the estimated monthly saving in VND. Accepting a recommendation promotes it to a normal automation rule; dismissing it suppresses the same signature for 30 days (**REQ-4.4.5**).

Journey 3 — Author an automation rule.
The rule editor validates the rule in real time as the user constructs the *when…then…until…* expression. Conflicting enabled rules are reported before the user can save. The VND saving estimate is fetched from the server and displayed in the editor; the Save button is not enabled until a syntactically valid, conflict-free rule with a visible estimate is present.

6.5 Screen Design (Wireframes)

Six application screens were designed and prototyped as wireframes before implementation. The wireframes are presented in Figures 6.3–6.8. All six screens follow the same visual language: a left navigation rail, the page title and primary action button in the top bar, and content laid out with a consistent spacing grid. The Okabe–Ito palette is applied to all charts and status indicators.

A defining design constraint applies to the device-control screens: **every control widget is rendered from the capability schema** (**REQ-4.2.1**). The front-end never branches on device type. When the user navigates to a device's detail screen, the front-end requests the device's capability description from the server and receives a structured schema listing the device's controls — each classified as a toggle, a bounded range, an enumeration, or a read-only readout

Figure 16 — User Flow: monitor, control, and accept a saving

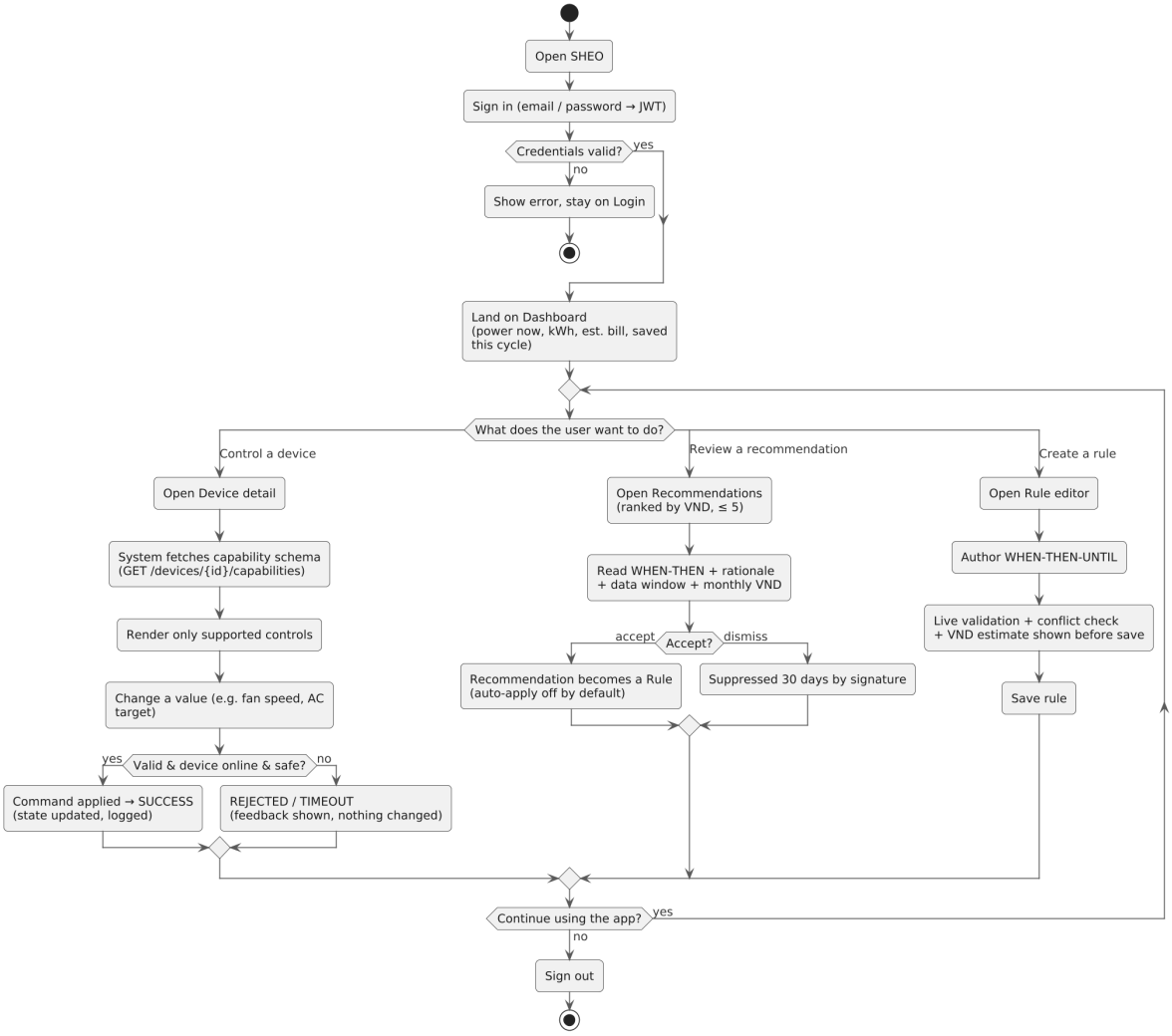

Figure 6.2. User flow for the three core journeys. After sign-in the user lands on the Dashboard and chooses a journey: controlling a device (capability fetch → validate → SUCCESS/REJECTED/TIMEOUT), acting on a recommendation (Accept ⇒ becomes a rule; Dismiss ⇒ 30-day suppression), or authoring a rule (live validation + conflict check + VND estimate before Save is enabled).

— and the generic control component renders the appropriate widget for each entry. Adding a new device type to the system requires only a new capability schema entry — no new UI code.

6.6 Usability Evaluation

Following the evaluation step of the UI design process, the interface is assessed in two passes: a formative heuristic walkthrough applied during development, and a summative task-based test applied at the milestone demonstration.

6.6.1 Heuristic walkthrough

Each screen is examined against the seven golden rules from Section 6.3. Rather than checking every screen independently, the walkthrough focuses on the *single interaction pattern that spans all device types* — the capability-driven control widget — because validating one pattern validates the interaction on every current and future device type simultaneously.

Figure 9e — Login

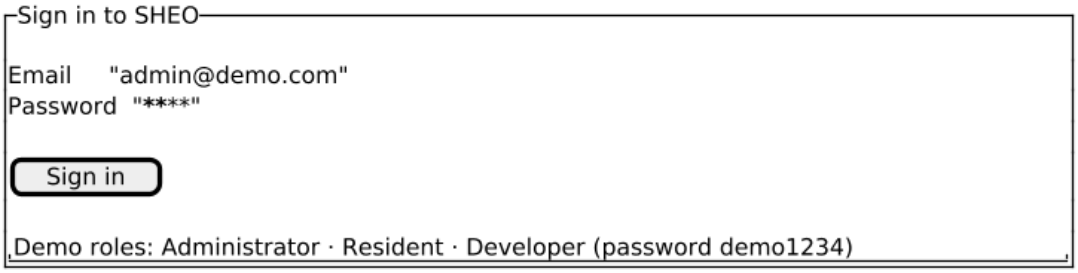

Figure 6.3. Login screen. Email and password sign-in establishes a session token held by the front-end for the duration of the session. Four demo accounts (all with password `demo1234`) are accessible via pre-filled quick-login buttons, supporting the task-based evaluation: `admin@demo.com` (Administrator — building owner with no unit, providing the building overview, tariff, and onboarding, and unable to operate devices), `resident@demo.com` (Resident of Unit 101, controlling that unit's devices), `resident2@demo.com` (Resident of Unit 102), and `dev@demo.com` (Developer/Tester on maintenance Unit 100).

Figure 9a — Dashboard: live energy overview (Resident, Unit 101)

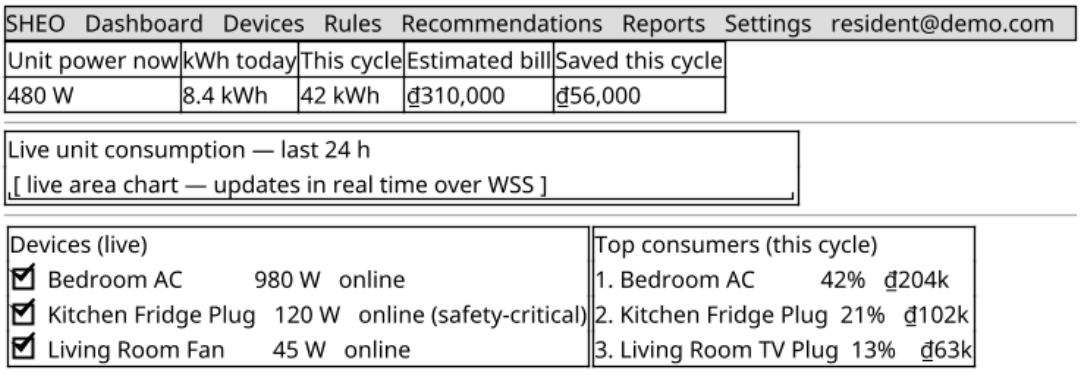

Figure 6.4. Dashboard. Prominent headline-metric (large-numeral) displays show the unit's total power (W), kWh consumed today and in the current billing cycle, estimated bill in VND, and **savings so far this cycle in VND**. A live WebSocket consumption chart, per-device tiles with offline badges, and a top-consumers panel complete the view (**REQ-4.1.1**–**REQ-4.1.5**, **REQ-4.5.4**).

Figure 9b — Device Detail (controls rendered from the capability schema)

SHEO Dashboard Devices Rules Recommendations Reports Settings

- Living room
  - Living Room TV Plug (online)
  - Living Room Fan (online)
  - Living Room Sensor (read-only)
- Hallway
  - Hallway Light (online)
- Bedroom
  - Bedroom AC (online)
- Kitchen
  - Kitchen Fridge Plug (safety-critical)

Bedroom AC — Bedroom

Power "Off ▾"
Target "[====O=] 24 °C (range 16-30)"
Mode "cool ▾"

⚠ safety: Target is safety-sensitive

Telemetry: 980 W · today 4.2 kWh

[Send command] last outcome: SUCCESS

Widgets above are generated from the device capability endpoint — zero per-type UI code.

Figure 6.5. Device detail screen. Controls (on/off toggle, target temperature range, fan-speed enumeration) are generated entirely from the structured capability schema returned by the server — there is no per-device-type UI code. Safety-sensitive controls carry a *safety* warning pill (**NFR-SAF-3**); the fridge is safety-critical and not switchable, and the sensor is read-only. The Resident operates only their own unit's operable devices, while the Developer/Tester can add or remove mock devices on the maintenance unit (**BR-1**).

Figure 9c — Rule Editor (explainable WHEN-THEN, estimate before save)

SHEO Dashboard Devices Rules Recommendations Reports Settings

New rule

Name "Turn off hallway light when room is empty"
WHEN "occupancy = false" on "Hallway Light ▾"
THEN "power = off"
UNTIL "occupancy = true"

☐ Auto-apply (off by default) Priority "normal ▾"

Estimated saving: ~ ₫38,000 / month = (baseline − expected) × tariff
Validation: OK — no conflicting rules

[Save rule] [Cancel]

Existing rules	State	Auto	Est. ₫/mo
Bedroom AC 26 °C at night	Enabled	on	61,000
Hallway light off when empty	Enabled	off	38,000
Idle TV plug off 01-05h	Disabled	off	12,000

Figure 6.6. Rule editor. A *when…then…until* rule form with live validation, conflict detection, and the VND saving estimate displayed before Save is enabled (**REQ-4.3.x**, **REQ-4.5.3**). Auto-apply is off by default (**BR-3**); the execution history shows past firings with a two-minute undo link (**REQ-4.3.6**).

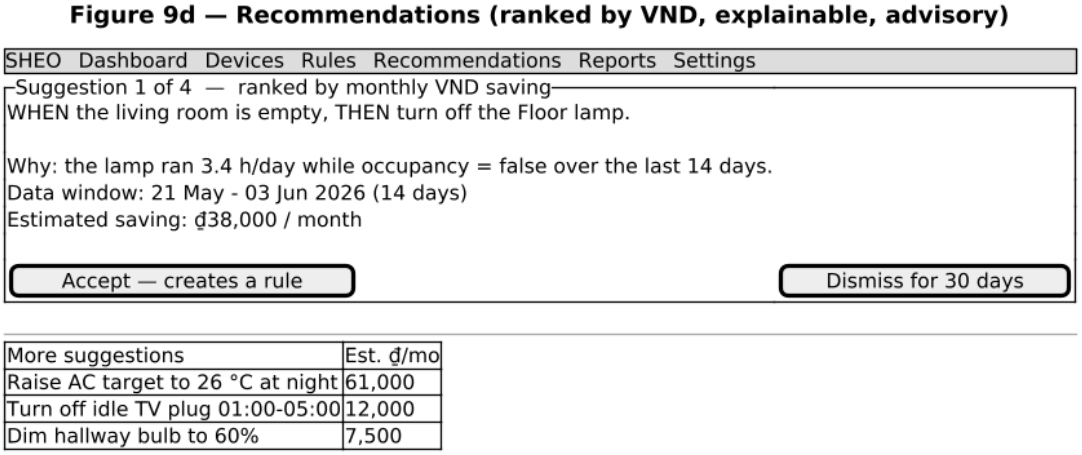

Figure 6.7. Recommendations screen. Up to five ranked, human-readable suggestions are shown, each with the *when…then* rule expression, a rationale, the data window, and the estimated monthly saving in VND. *Accept* converts the recommendation into a normal rule; *Dismiss* suppresses it for 30 days (**REQ-4.4.x**).

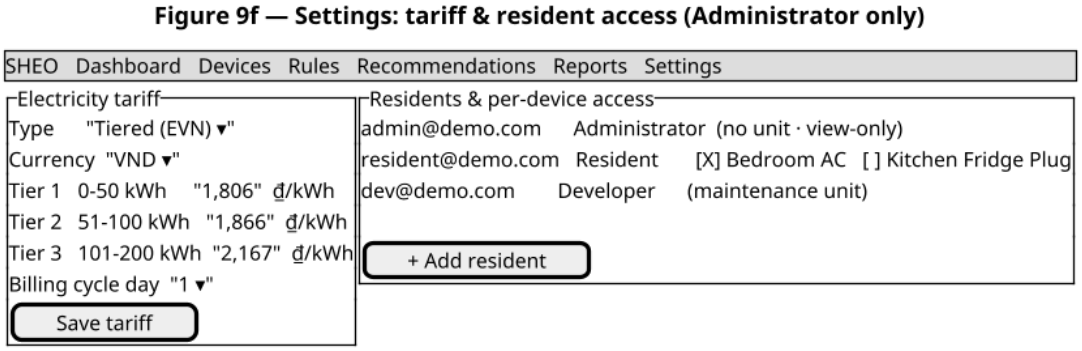

Figure 6.8. Settings screen. The building-wide tariff configuration (flat, tiered EVN, or time-of-use; amounts in VND) and the unit-onboarding panel are editable only by the Administrator; non-Administrators see this screen in a read-only state (**BR-1**, **BR-5**, **NFR-SEC-4**).

Two concrete design defects surfaced during this pass and were corrected before the implementation milestone:

1. **VND estimate placement.** An early iteration placed the saving estimate below the Save button, making it easy to miss. The corrected design disables the Save button until the estimate has been fetched and displayed, forcing the user to see the figure before committing.
2. **Safety visibility.** Safety-sensitive controls (such as the AC compressor toggle) were initially indistinguishable from ordinary controls. The corrected design adds a prominent *safety* warning pill adjacent to any control that the capability schema identifies as safety-sensitive (**NFR-SAF-3**).

The heuristic pass also confirmed that auto-action defaulting to off (**BR-3**) and the two-minute undo window (**REQ-4.3.6**) together satisfy the "recoverable" golden rule for all three core journeys.

6.6.2 Task-based usability test

At the milestone demonstration, evaluators perform the four tasks in Table 6.3 using the four demo accounts, spanning the Administrator, Resident, and Developer/Tester roles. For each task, successful completion is defined by a measurable criterion; any task that fails its criterion is treated as a defect and triggers a design revision before release.

Table 6.3. Task-based usability test: tasks and measurable success criteria.

Task (journey)	Measurable success criterion	Tied to
Control a device	Reach a device's controls and send a command in ≤ 3 clicks from the dashboard; the outcome (SUCCESS / REJECTED / TIMEOUT) is shown within the two-second UI-feedback budget.	**REQ-4.2.x**, **NFR-PER-2**
Review and act on a recommendation	Read the *when…then* rule expression, rationale, and VND figure, then Accept or Dismiss in ≤ 2 clicks; the VND figure is visible without scrolling.	**REQ-4.4.x**
Author a rule	Build a valid rule and see its VND estimate *before* Save is enabled; a conflicting enabled rule is reported inline, not silently overwritten.	**REQ-4.3.x**, **REQ-4.5.3**
First-run onboarding	A first-time Resident operates a device in their unit and observes live telemetry on the dashboard within a few minutes of signing in to the unit.	**REQ-4.2.5**, **REQ-4.1.1**

Error prevention and recoverability are tested explicitly as part of each task: an out-of-range or offline command must surface a clear rejection message and leave device state unchanged; the auto-action toggle must default to off; and any auto-applied rule firing must be undoable for two minutes. Evaluation results are written up in a brief usability report; any failing task loops back into the design (re-layout, clearer labels, or additional feedback wording) before the release build is produced.

Chapter 7

Software Construction

Software construction, as defined in Lecture C08, is the activity of converting a detailed design into working, maintainable code. It encompasses not only typing instructions into an editor but also the discipline that surrounds that act: a consistent coding style, a modular decomposition that hides complexity behind stable interfaces, a principled programme of refactoring to remove code smells as they accumulate, and a formal inspection that exposes defects before they reach users. This chapter traces each of those concerns through the SHEO implementation, grounding every point in the structure of the system as it was actually built.

7.1 Construction in the development lifecycle

In the SHEO development process, construction follows the detailed design established in Chapter 5: class responsibilities, sequence flows, and persistence schemas had been decided before implementation began. Construction therefore had a clear contract to satisfy, and every significant decision—which library, which language feature, how to structure an error return—could be evaluated against the design goals of high cohesion, low coupling, and information hiding that are documented in the System Design Document.

The technology stack was chosen to support those goals directly.

7.2 Coding style and conventions

Lecture C08 identifies six coding-style principles: *consistency*, *clarity*, *single-purpose methods*, *fewer than six parameters*, *naming* (verbs for functions, nouns for classes and variables), and *comments that describe what, not how*. SHEO adheres to each, enforced by convention rather than a separate lint pass.

7.2.1 Naming and intent

Every operation in the backend is named by a verb phrase that announces its effect: the capability-validation routine, the command-application entry point, the unit-scoped lookup that raises a not-found error when nothing matches, the capability-retrieval accessor, and the adapter-selection function all read as actions. Every abstraction is named by a noun that describes the responsibility it represents: the device-control service, the capability schema, the device-adapter interface, and the structured validation result. This consistency means that a reader surveying the list of components in a module can determine the responsibility of each one before examining its definition.

7.2.2 File-level docstrings

Every backend module opens with a brief descriptive header that states the module's purpose and its relationship to the requirements. The header of the device-control service module, for example, records that the module implements the capability-driven control path of **REQ-4.2.3** and the safety guards of **NFR-SAF**, and that both manual user commands and rule-initiated commands flow through the single command-application entry point so that validation and safety rules are enforced uniformly. Such a header names the requirement identifiers that the module

Table 7.1. SHEO technology stack and the construction rationale for each choice.

Component	Technology	Construction rationale
Language	A modern, statically hinted dynamic language	Its rich type-hint system, value-object facilities, and a mature standard library reduce the volume of boilerplate that obscures intent.
Web framework	A declarative Python web framework	Routes, dependency injection, and request and response schemas are declared together, making the interface layer self-documenting without a separate tool.
Persistence	A mature object-relational mapper	The repository pattern is natural to it; queries are isolated from service logic, realising the information-hiding goal at the persistence boundary.
Data validation	A type-driven validation library	Schema validation is declared as ordinary types; invalid inputs are rejected at the boundary with a machine-readable error before any business logic executes.
Relational store	A file-backed embedded database	Eliminating an external database server removes an infrastructure dependency during development and testing, keeping the test suite fast and reproducible.
User-interface framework	A component-based front-end framework	Component-based decomposition mirrors the backend service decomposition, giving each screen a single owner and a single reason to change.
Charting	A declarative charting library	Declarative, data-driven charts align with the capability-schema philosophy: the dashboard renders from data descriptors rather than hard-coded widget trees.

satisfies and states the architectural invariant—the single choke point for all commands—that reviewers should verify, in accordance with the "describe what, not how" rule.

7.2.3 Type hints and small, focused functions

Type annotations are declared on every public operation. Return types, parameter types, and the element types of collections are all stated, converting the type system into a machine-checked expression of each operation's contract. The capability-validation routine is representative of the single-purpose, narrow-parameter style the course advocates.

The routine accepts a device type, the name of a control, and a proposed value, and returns a structured validation result that reports either success—with the value normalised to its canonical form—or failure together with a human-readable explanation. It first resolves the capability schema for the device type, then locates the named control within that schema, and finally checks the proposed value against the control's kind: a numeric range control is bounds-checked, while toggle and enumeration controls are matched against their permitted sets. The routine takes exactly three parameters, well below the six-parameter threshold; it returns a structured result so that the error message is machine-readable rather than raised as a bare exception; and it has no side effects, leaving persistence and dispatch to its callers. Its descriptive header cites **REQ-4.2.3** rather than restating the algorithm that the code already expresses.

7.3 Information hiding and modularity in code

Information hiding is applied at three distinct boundaries in the SHEO codebase, each chosen to isolate a design decision that is likely to change.

7.3.1 Repositories hide persistence

The persistence layer is organised into repositories—one for devices and one for the per-device permission records—that encapsulate all query construction. Service code never assembles a query predicate directly; it invokes named operations such as a unit-scoped query that returns every device belonging to a given unit, or a membership check that confirms a particular device belongs to a particular unit, and receives domain objects in return. If the storage engine were replaced, only the repository implementations would need to change.

7.3.2 The capability schema hides device-type variance

The capability schema expresses everything the rest of the system needs to know about a device type as *data* rather than as *code branches*. Neither the interface layer nor the user interface ever tests for a specific device type with a conditional branch; both query the schema and act on what they find. Adding a new device type therefore requires changes in exactly two places—the capability registry and the corresponding adapter—and nowhere else.

7.3.3 Adapters hide per-type behaviour

The Strategy pattern is realised through an abstract device-adapter interface defined in the adapter base module. The interface is deliberately minimal: one abstract operation for advancing the simulation by one time step, and one default operation for applying a validated control change to a device's state. Callers obtain the correct strategy through the adapter-selection function, whose body is a single lookup in a registry that maps each device type to its corresponding per-type adapter; the concrete adapter classes are never referenced directly by callers.

The consequence of this design is that the entire call site in the service layer reduces to two steps—obtain the adapter for the device's type, then ask it to apply the validated control change—regardless of how complex the underlying simulation logic becomes. The air-conditioner adapter, for instance, models hysteresis, compressor cycling, and ambient-temperature drift in roughly thirty lines that remain entirely invisible to the service.

7.4 Refactoring and code smells

Refactoring, as defined in C08, is the transformation of the internal structure of code without changing its observable behaviour. Its purpose is to remove *code smells*—structural problems that make software harder to understand, test, or extend—prior to the addition of new capability. The canonical smells named in the course include duplicated code, long method, large class, too many parameters, divergent change, lazy class, speculative generality, and temporary field.

Two concrete refactoring decisions made during SHEO construction directly address smells from that list.

7.4.1 Centralising the safety check

An early version of the control path performed safety checks in multiple places: once in the interface handler that received a user command, and again in the rule engine that executed automated actions. This is the *duplicated code* smell: the same guard appeared in two locations, so a correction to the guard had to be applied twice and could silently diverge. The refactoring moved all safety logic into a single private safety-check step within the device-control service, and

introduced a single public command-application entry point through which every command—manual and automated alike—must pass.

Within that entry point the command-application logic is strictly linear. It first rejects commands addressed to a device that is offline or has been forced offline; it then delegates to the capability-validation routine and abandons the command if the proposed value is invalid, retaining the normalised value otherwise; it next runs the safety-check step and returns immediately if a safety constraint is violated; and only then does it obtain the per-type adapter and ask it to apply the validated change. The sequence—validate, safety-check, dispatch—is explicit and self-contained. A reader can verify the entire guard chain by following one operation, without tracing branches across several modules.

7.4.2 Removing duplicated validation

Before the capability schema was introduced, each interface operation that accepted a device command contained its own range check and enumeration check. These checks were written independently, carried slightly different error messages, and diverged over time as device types were added. This is simultaneously the *duplicated code* smell and a mild case of the *divergent change* smell: a new device type required edits across several interface operations. The refactoring introduced the capability-validation routine as the single authority on what constitutes a valid command for each device type, and removed all per-operation checks. Every operation now delegates to that routine by way of the command-application entry point; the capability schema is the single source of truth.

Both refactorings satisfy the C08 criterion: no behaviour changed—the automated test suite continued to pass at every intermediate step—but the internal structure became easier to reason about and to extend.

7.5 Code review

Lecture C08 distinguishes two review processes. A **walkthrough** is author-led and informal: the author guides peers through the code, gathering suggestions. A **formal inspection** is conducted by non-authors against a pre-defined checklist; the author is present but does not lead. The course identifies three review criteria: *maintainability* (can the code be changed safely?), *reusability* (can components be used in other contexts?), and *extensibility* (can the system accommodate new requirements without pervasive edits?).

7.5.1 The adversarial inspection process

SHEO underwent a structured review that functioned as a formal inspection. A non-author reviewing agent examined the codebase against a checklist derived from the requirements, the non-functional requirements, and security best practice. The review was adversarial in the sense that the inspector was explicitly tasked with finding defects, not confirming correctness—matching the spirit of the formal inspection model in which the non-author role is designed to surface problems that the author has normalised. The checklist covered five categories:

1. **Authorisation.** Every interface operation must enforce unit-scoped access control; cross-unit access must be impossible.
2. **Input validation.** Every external input must be rejected at the boundary before entering business logic; no operation may bypass the validation schema.
3. **Safety constraint enforcement.** The **NFR-SAF** rules must be applied on every command path—interactive and automated—not only when a human issues a command.
4. **Telemetry isolation.** One unit must not be able to read another unit's sensor data, even if device identifiers are predictable integers.

5. **Rule mutation.** A rule update must pass through the same authorisation and validation as rule creation.

7.5.2 Defects found and corrected

The inspection identified two defects, both verified by a failing test before the fix and a passing test after.

Cross-unit insecure direct object reference in the telemetry series operation. The telemetry-series retrieval operation accepted a device identifier without verifying that the device belonged to the requesting user's unit. An authenticated resident from unit A could therefore obtain the full telemetry history of any device in unit B by supplying its identifier. This is an insecure direct object reference (IDOR) vulnerability—a failure of the authorisation criterion. The fix introduced a unit-scoped lookup at the start of the operation, matching the pattern already established by the device-control service's unit-scoped lookup. A regression test was added asserting that a not-found response is returned when a device belonging to a different unit is requested.

Unvalidated rule update. The rule-update operation accepted an arbitrary request payload and wrote it to the database without passing it through the same input-validation schema that the rule-creation operation enforced. A malformed or adversarially crafted update could therefore insert a rule with missing required fields, causing the rule engine to raise an unhandled exception at execution time. The fix applied the existing rule-update schema to the operation, restoring the input-boundary guarantee established at creation time.

7.5.3 Review criteria mapped to SHEO mechanisms

Table 7.2 maps each C08 review criterion to the mechanism that satisfies it after the inspection.

The two defects found by the inspection were both failures of the *maintainability* and *security* criteria: the cross-unit IDOR arose because a defensive pattern established in one service had not been propagated to a newer interface operation, and the unvalidated rule update arose because a boundary guarantee had been applied inconsistently. Both are the kind of defect that is invisible to the original author—who knows the intent—and visible to a non-author reviewer who tests the code against the stated contract. This is precisely the rationale that C08 gives for requiring formal inspections rather than relying on walkthroughs alone.

Table 7.2. C08 code review criteria and the corresponding SHEO mechanisms post-inspection.

Review criterion	**What it requires**	**SHEO mechanism**
Maintainability	Internal structure can be changed without inadvertent side effects.	A single command-application choke point; the repository abstraction; one capability schema per device type; no duplicated guards.
Reusability	Components can be used in different contexts without modification.	The capability-validation routine is invoked by the interface layer, the rule engine, and the automated test suite without any context dependency; the per-type adapters are stateless and context-free.
Extensibility	New requirements can be accommodated with localised changes.	A new device type requires exactly one new capability-schema entry and one new per-type adapter; no existing code is modified (Open/Closed Principle).
Security	Access control is enforced at every boundary.	A unit-scoped lookup on every device and telemetry operation; input-validation on every write path; an administrator-authorisation guard on all mutation operations.

Chapter 8

Software Configuration Management

This chapter documents the Software Configuration Management (SCM) plan for the AI-Driven Smart Home Energy Optimizer (SHEO) project. SCM encompasses the principles, techniques, and tools used to identify, control, record, and audit changes to every work product throughout the project lifecycle. The chapter is organised around the five canonical SCM tasks drawn from **IEEE 828** [4] and **IEEE 1042**: configuration identification, version control, change control, configuration audit, and status reporting.

8.1 SCM Concepts and Standards

8.1.1 Foundational Terminology

Software Configuration Management rests on three foundational concepts that must be carefully distinguished.

Configuration Item (CI)
: Any artifact placed under version control and governed by the change-control process. A CI is identified by a unique name, a version number, and a designated owner. Examples range from source files and test suites to design documents and dependency manifests. Once a CI is placed under control, any modification to it requires a sanctioned change request.

Baseline
: A formally reviewed, approved, and version-locked set of CIs that serves as the stable foundation for subsequent work. As defined in **IEEE 828**, a baseline can be changed only through the formal change-control process; ad hoc modifications to baselined artifacts are prohibited. Each baseline corresponds to a project milestone and is recorded as an immutable, annotated marker in the version-control repository.

Version / Revision / Release
: These three terms are distinct and must not be conflated. A **version** denotes a particular state of a CI at a point in time, identified by a version number of the form `major.minor.patch` (e.g. `1.0.0`). A **revision** is a specific change to an existing version, typically addressing a defect or a minor update within the same major–minor line. A **release** is a version that has been formally approved, baselined, and made available for use or delivery; it implies a completed quality gate and is associated with a milestone tag.

8.1.2 SCM Tasks

Following **IEEE 1042**, the SHEO project organises its SCM activity around five tasks:

1. **Configuration identification** — naming and registering every CI under control (Section 8.2).
2. **Version control** — tracking changes to CIs over time using a distributed version-control system (Section 8.4).
3. **Change control** — evaluating, approving, and implementing changes to baselined CIs via a defined process (Section 8.5).

4. **Configuration audit** — verifying that a baseline is internally consistent and satisfies its requirements (Section 8.5.3).
5. **Status reporting** — recording and communicating the state of every CI and change request over time (Section 8.5.2).

8.2 Configuration Items in SHEO

The SHEO project identifies seven CI groups. Each group is assigned a short code used throughout the change-control and status-accounting logs. Table 8.1 lists the groups with their representative artifacts and designated owners.

Table 8.1. SHEO Software Configuration Items

Code	CI Group	Representative Artifacts	Owner
C1	Requirements baseline	The approved requirements specification (v1.0) together with its refinement note	Requirements lead
C2	Design & engineering docs	The software design document, the accompanying structural and behavioural diagrams, and this configuration management plan	Architect
C3	Backend source	The backend service, domain, core, adapter, simulation, persistence, and schema modules together with the data-seeding routine	Backend developers
C4	Frontend source	The client application root, its page and component modules, and its authentication and supporting library code	Frontend developers
C5	Test suite	The shared test fixtures and the nine automated test modules comprising sixty-five cases	QA lead
C6	Build & dependency manifests	The backend and client dependency and build manifests, with all versions pinned	Build owner
C7	Runtime configuration	The centralised configuration module exposing every requirement-mandated policy threshold, overridable through environment settings	Backend lead

Several aspects of this identification scheme merit elaboration.

Requirements (C1). The requirements specification is the project's single source of truth. Functional and non-functional requirements each carry a structured identifier formed from their category and number. These identifiers serve as the anchor points for the traceability matrix and are referenced in the recorded description of every change, maintaining a complete forward and backward trail between requirements, implementation artifacts, and tests.

Runtime configuration (C7). Every policy threshold mandated by the requirements is centralised in a single configuration module rather than scattered as literal constants throughout the source code. The named thresholds — among them the offline-detection timeout, the savings-baseline window, the drift tolerance, and the undo window, each traceable to its governing requirement — are declared as named fields whose defaults can be overridden through environment configuration. Treating policy thresholds as configuration data means that a policy change is a single, narrowly scoped, reviewable, and baseline-able edit.

Build manifests (C6). The exact version of every backend dependency is pinned, and the client dependencies are likewise locked to fixed versions. Pinning is what makes a baseline reproducible: each retrieval of the configuration installs an identical set of dependency versions.

Derived artifacts (non-CIs). Generated outputs are listed in an ignore manifest and are not baselined, as they are fully reproducible from the CIs above. These include the backend virtual environment, compiled-bytecode caches, the generated runtime database, the installed client dependency tree, and the client build output.

8.3 Baselines

A baseline freezes a coherent snapshot of all relevant CIs at a milestone boundary. IEEE 828 requires that once a baseline is established, it may be modified only through the formal change-control process. In the SHEO project each baseline corresponds to an immutable, annotated milestone marker placed on the integration line; the marker records the precise revision it identifies, the date, and the milestone name.

Table 8.2 summarises the four project baselines.

The M1 baseline constitutes the **Requirements Baseline**: it freezes the requirements specification and makes subsequent design and implementation decisions traceable back to an approved set of requirements. The M2 baseline constitutes the **Design Baseline**: the software design document and all architectural figures are locked and serve as the authoritative specification for coding. The M3 and M4 baselines together constitute the **Implementation Baseline**: M3 is the first fully-running system and M4 is the release-ready, fully-audited delivery. Each baseline approval is recorded in the status-accounting log together with the milestone marker, the precise revision it identifies, the date, and the list of change requests incorporated since the previous baseline.

8.4 Version Control and Branch Workflow

8.4.1 Tooling

The project uses a single distributed version-control system (Git) to track changes to every CI over time. The entire project — source code, tests, documents, and dependency manifests — resides in one repository, ensuring that a single baseline marker captures every CI at once. No CI is managed outside the repository.

8.4.2 Branch Model

A trunk-plus-feature model is adopted, suited to a small, collocated course team where the integration branch must remain continuously releasable.

Table 8.2. SHEO Project Baselines

Milestone	Marker	CIs Baselined	Entry Criteria
M1 — Requirements	M1	C1 (the requirements specification v1.0 and its refinement note), the initial documentation scaffold, and the repository with its ignore manifest	Requirements specification approved by instructor; all requirement identifiers stable
M2 — Design & skeleton	M2	C2 (the architectural and structural design documents); the backend module skeleton; C6 with pinned dependencies; C7 with all policy thresholds declared	Design reviewed; the skeleton loads cleanly and the backend service starts
M3 — Implementation	M3	C3 (the complete backend services), C4 (all client pages and components), C5 (the full test suite), C7 (all thresholds wired)	All sixty-five automated tests pass; the demonstration seeds twenty-one days of data and the application runs end-to-end
M4 — Release / submission	M4	The entire project: C1–C7 consistent end-to-end; the traceability matrix complete; this configuration management plan	Test gate passing; traceability matrix fully populated; configuration audit passed

Integration line
: The sole long-lived line of development. It must always be buildable and pass the full suite of sixty-five automated tests. Changes are never applied to it directly; they arrive exclusively through reviewed merge proposals.

Feature branch
: A short-lived line of work that diverges from the integration line for each approved change request or new capability. It is discarded once its work has been merged.

Fix branch
: A short-lived line with the same lifecycle as a feature branch, used specifically for defect corrections.

Figure 8.1 illustrates the branch flow schematically.

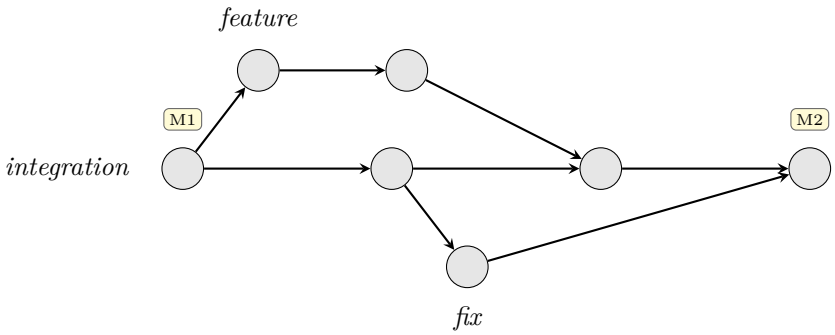

Figure 8.1. Schematic branch model: feature and fix branches diverge from the integration line and are merged back after peer review and a passing test gate. Milestone markers (M1, M2, …) denote baseline revisions on the integration line.

8.4.3 Commit Conventions

The recorded description of each change follows a structured convention so that the project history remains scannable and every change stays traceable to its origin. Each description pairs a change-type — distinguishing, for instance, a new capability, a defect correction, an addition to the test suite, a documentation update, an internal restructuring, or routine maintenance — with the affected subsystem and, where applicable, the identifier of the requirement it implements or modifies.[1] Naming the governing requirement identifier is mandatory whenever a change implements or modifies a requirement, as this is what keeps automated status accounting tractable.

8.4.4 Version Numbering

Software artifacts are versioned according to a three-part *major.minor.patch* scheme, with version `1.0.0` designating the M4 release. The **major** digit increments on incompatible interface or data-model changes; the **minor** digit increments on backward-compatible new functionality; the **patch** digit increments on backward-compatible defect fixes. The backend declares its own version number, which is advanced as part of the same change that establishes a new baseline.

8.5 Change Control and Status Reporting

8.5.1 Change Control Process

Any modification to a baselined CI — whether it addresses a new requirement, a discovered defect, an adjustment to a policy threshold, or a document revision — must pass through a three-stage change-control process. For a course-sized team this process is intentionally lightweight, but the evaluation and review gates are real and enforced.

Stage 1: Request, Evaluate, and Approve. A team member raises a tracked request describing the proposed change, the CIs affected (by code C1–C7), and the requirement identifiers involved. A designated reviewer assesses the impact: which requirements are affected, which modules must change, which tests must be updated, and — critically — whether a baselined CI is implicated (changes to a baseline require stronger justification than changes to in-progress work). The configuration-control authority approves, defers, or rejects the request and records the decision against it.

[1] For example, a change introducing conflict detection among enabled rules would be recorded as a new capability scoped to the rules subsystem and annotated with the functional requirement it satisfies.

Stage 2: Implement on Branch and Peer-Review. An approved request is implemented on a dedicated short-lived feature or fix branch that diverges from the integration line. The implementing developer records the code change together with the tests that demonstrate its correctness, naming the relevant requirement identifier in each change description. When implementation is complete, a merge proposal is opened and a second team member performs a formal peer review, verifying correctness, adherence to the three-tier layered architecture, and that every change is traceable to an approved requirement. Self-merges are not permitted.

Stage 3: Test Gate, Merge, and Release. The full automated test suite of sixty-five cases must pass on the branch before merge. This is the non-negotiable merge gate. On passing, the branch is merged into the integration line and then discarded. If the merged change completes a milestone, the configuration manager establishes the corresponding annotated baseline marker (M1–M4) and updates the status-accounting log and traceability matrix.

8.5.2 Configuration Status Reporting

Configuration status accounting answers three questions for every CI at any point in time: *what* changed, *who* made the change, and *when* it was made. In the SHEO project, status accounting is maintained through two artefacts.

The **change register** records every change request and defect with a unique identifier, the affected CI codes, the requirement identifiers involved, the current state as it advances from raised, through in progress and under review, to merged and finally released, and the line of work and merge proposal that resolved it. The change identifier is carried through to the line of work and the recorded change descriptions, providing a complete forward and backward audit trail.

The **baseline log** records, for each milestone, its marker, the precise revision it identifies, the calendar date, and the list of change identifiers incorporated since the preceding baseline. This log is the canonical record of what the project delivered at each stage and forms the primary input to the configuration audit.

8.5.3 Configuration Audit

Two complementary audits are performed at each milestone, and most rigorously at the M4 release baseline:

Functional Configuration Audit (FCA)
: Verifies that the baseline satisfies all requirements assigned to it. Evidence is the requirements-to-implementation traceability matrix, which maps each requirement identifier to the implementing module and routine and to the test case that exercises it, combined with a fully passing run of the test suite.

Physical Configuration Audit (PCA)
: Verifies that the baseline contains exactly the expected CIs (C1–C7) at their declared versions, and that all derived artifacts are absent (governed by the ignore manifest). The auditor checks the contents identified by the baseline marker against the CI catalogue in Section 8.2.

Together these two audits provide the assurance required by IEEE 828 that the released configuration is both correct and complete, closing the SCM lifecycle for each milestone.

Chapter 9

Quality Assurance: Verification and Validation

This chapter documents the quality-assurance programme of the Smart Home Energy Optimizer (SHEO). It establishes the conceptual vocabulary of software quality, positions the project's testing effort within the classical V-model, and then demonstrates *both* families of dynamic testing required of a rigorous verification campaign: black-box (specification-based) testing and white-box (structure-based) testing. The discussion is grounded in the project's **65 automated tests** and concludes with a requirements-to-tests traceability summary and a forward-looking classification of anticipated maintenance.

9.1 Software Quality, Verification and Validation

Following the IEEE Standard Glossary of Software Engineering Terminology (IEEE 610.12) [5], *software quality* is the degree to which a system meets specified requirements and, in turn, the degree to which it meets stated and implied user needs and expectations. Quality is therefore not a single attribute but a composite of functional correctness together with non-functional properties such as reliability, security, safety and maintainability. For SHEO the load-bearing non-functional families are safety (NFR-SAF, protecting safety-critical appliances and rate-limiting the air-conditioner compressor) and security (NFR-SEC, authentication, role-based access control and never storing plaintext secrets).

Two complementary activities establish quality. *Verification* asks "are we building the product right?" — it is the developer-facing check that each artefact conforms to its specification and to the artefact that preceded it (requirements, design, code). *Validation* asks "are we building the right product?" — it is the user-facing check that the delivered system actually satisfies the stakeholder's real needs. In SHEO, verification is realised by the unit and integration tests that confirm the implementation honours its contracts, while validation is realised by the system and acceptance tests that exercise the application through its public interface against the acceptance criteria agreed with the customer — here the SHEO stakeholder in the IEEE sense, namely the Administrator who commissions and accepts the system. The two are not interchangeable: a system can be verified (faithful to its specification) yet invalid (the specification was wrong), which is precisely why both arms of the V-model below are required.

9.2 The V-model

The *V-model* arranges the software life-cycle so that every constructive (development) phase on the descending left arm is paired with a corresponding destructive (test) phase on the ascending right arm. The horizontal links express the principle that the test cases for a level are derived from, and validate against, the artefact produced by the phase opposite it: acceptance tests are written against the requirements, system tests against the architecture, and integration tests against the detailed design, and so on. Figure 9.1 draws the model and maps SHEO's concrete test levels onto it.

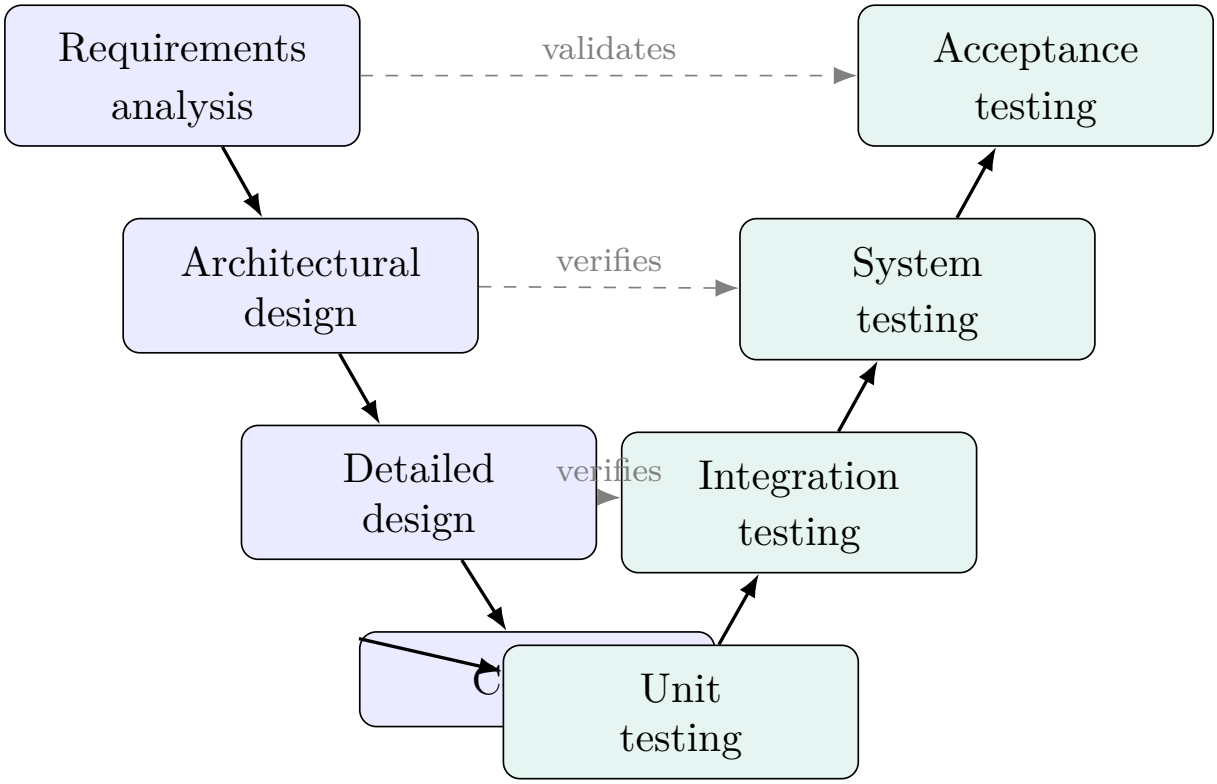

Figure 9.1. The V-model mapped onto SHEO. Each development phase (left, shaded blue) is checked by the test level opposite it (right, shaded green): the topmost link *validates* the requirements against the stakeholder's real needs, while the lower links *verify* each design artefact against its specification. Dashed links denote the artefact each test level is derived from.

SHEO populates every level of the right arm. *Unit testing* exercises the pure domain functions in isolation; *integration testing* drives the wired stack (API router → service → repository → persistence) across real boundaries; *system testing* asserts aggregate, user-visible outputs end-to-end on a fully seeded application; and *acceptance testing* demonstrates the customer's acceptance focus items. The sequence-level interactions that integration tests confirm are illustrated in the design chapter (Figure 5.7).

9.3 Test Strategy and Levels

The automated suite comprises **65 tests**. It runs in-process against the application's public interface using an in-memory test harness, requiring no live server, network or hardware; a disposable database is recreated for each run, so the suite is fully deterministic. The split between levels is physical: pure-logic unit tests touch no transport layer and no database, whereas every other suite boots a fully seeded application and drives it through its public endpoints.

Each test is expressed in the canonical test-case form ⟨input, expected outcome⟩: an input (or input sequence) is applied and the actual result is compared against an *expected output that is defined in advance*, never inferred after observing the program. A collection of such cases that share a fixture and a target constitutes a *test suite* — the project organises its tests into nine such suites, each devoted to one cross-cutting concern. The strategy observes three classical testing principles:

- the **tester ≠ developer** separation of concern: tests are written from the specification (the requirements and the capability contract) rather than to ratify whatever the code happens to do;

- the **expected output is fixed in advance**, so a test can only pass by the implementation being correct, not by redefining correctness;
- **regression**: because the suite is automated and deterministic, it is re-run on every change to guard previously verified behaviour against accidental breakage.

Table 9.1 summarises how the 65 tests distribute across the four levels.

Table 9.1. Distribution of the 65 automated tests across V-model levels.

Level	What it checks	Representative target
Unit	Pure domain functions in isolation: capability validation, tariff pricing, billing-cycle and time utilities, simulator energy model, password hashing.	the pure-domain logic suite
Integration	Multiple layers cooperating across a real boundary: the wired request path through service, repository and persistence, plus the event bus and adapter wiring.	the device-capability and rules suites
System	End-to-end behaviour of a fully seeded application through public endpoints: dashboards, reports, recommendations, savings.	the monitoring and savings suites
Acceptance	The customer's acceptance focus items, realised by the union of the system and integration tests against the demonstration seed.	(see Section 9.6)

9.4 Black-box Testing

Black-box (specification-based) testing derives test cases from the external specification of a component without reference to its internal structure. The suite deliberately applies three systematic black-box design techniques rather than choosing inputs ad hoc.

9.4.1 Equivalence Partitioning

Equivalence partitioning divides the input domain into classes within which the program is expected to behave identically, so that one representative per class suffices. The air-conditioner target-temperature control accepts an integer temperature in the closed range $[16, 30]$ degrees Celsius. Its domain partitions into three classes:

- a **valid** class — any value inside $[16, 30]$ (representative: 23), expected to be accepted;
- an **invalid-low** class — values below 16, expected to be rejected;
- an **invalid-high** class — values above 30, expected to be rejected.

The principle generalises beyond numeric ranges. The enumerated-and-toggle partitioning test divides such controls into valid representatives (a valid mode and power setting) and invalid ones (an invalid mode; an invalid power value), while a companion test treats an unsupported control (for instance a brightness setting directed at a device that has none) as its own invalid class.

9.4.2 Boundary Value Analysis

Boundary value analysis (BVA) exploits the empirical observation that faults cluster at the edges of equivalence classes. For the $[16, 30]$ range it prescribes testing each boundary, its immediate neighbour just outside, and a nominal interior value. Table 9.2 gives the canonical BVA set,

plus an out-of-domain negative value; all six cases are run as a single parametrised boundary test driven directly against the capability-validation routine.

Table 9.2. Boundary value analysis for the air-conditioner target range $[16, 30]$.

Input	Class	Expected
16	lower boundary (valid)	accept
30	upper boundary (valid)	accept
23	nominal interior	accept
15	just below lower boundary	reject
31	just above upper boundary	reject
-5	far invalid (negative)	reject

This realises the canonical "min, min $-$ 1, max, max $+$ 1, nominal" boundary set: the value 16 exercises the $\geq$ min guard, 15 its lower neighbour, 30 the $\leq$ max guard, and 31 its upper neighbour.

9.4.3 Decision-table Testing

When an outcome depends on a *combination* of conditions, a *decision table* enumerates the relevant condition combinations (rules) and their required actions, ensuring that no combination is overlooked. The outcome of a device command depends on whether the device is online, whether the value is in range, and whether the command is suppressed for safety (a safety-critical auto power-off, or a rate-limited compressor command). The resulting outcomes are SUCCESS, REJECTED, TIMEOUT and SKIPPED. Table 9.3 captures the rules and describes the test that exercises each.

Table 9.3. Decision table for command outcomes. "–" denotes a condition whose value is irrelevant once an earlier condition has determined the outcome.

Online?	In range?	Safety-blocked?	Outcome	Covering test
no	–	–	TIMEOUT	the offline-command timeout test
yes	no	–	REJECTED	the out-of-range rejection test
yes	yes	yes	SKIPPED	the safety-critical auto-power-off suppression test
yes	yes	yes	REJECTED	the compressor rate-limit test
yes	yes	no	SUCCESS	the in-range acceptance test

A related decision table over the conditions (temperature change?), (safety-critical tag?) and (unattended power-off?) governs the validation *warnings* surfaced to the user, and is exercised by the safety-warning validation test.

9.5 White-box Testing

White-box (structure-based) testing derives test cases from the internal control structure of the code, with the goal of executing its statements, branches and paths. We analyse the range-validation logic of the capability-validation routine — the procedure that enforces the $[16, 30]$ contract tested above. For a range-typed control it proceeds through a fixed sequence of guarded steps: it first coerces the supplied value to a number and rejects it if that is not possible; it then rejects any value below the declared minimum, and any value above the declared maximum; for

a control declared with an integer step it rounds the value to a whole number; and finally it accepts the normalised value. The corresponding *control flow graph* (CFG), in which decisions are drawn as diamonds and basic blocks as rectangles, is given in Figure 9.2; its numbered nodes correspond to these steps.

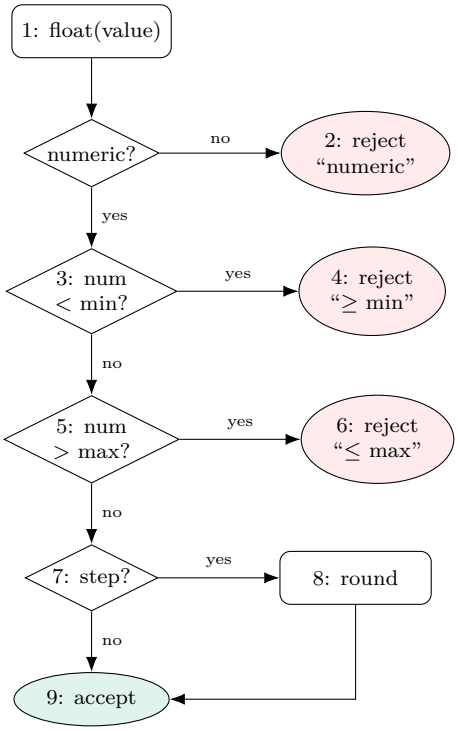

Figure 9.2. Control flow graph of the range-validation branch of the capability-validation routine. Diamonds are decisions; red terminals are rejections; the green terminal is the accepting return. Numbers correspond to the guarded steps described above.

The CFG exposes four decisions (nodes 1, 3, 5, 7) and therefore eight branch outcomes. Table 9.4 gives a small set of ⟨input → expected⟩ cases that, taken together, achieve *path* coverage of this subgraph (every feasible execution path), which subsumes both branch and statement coverage.

Table 9.4. White-box coverage cases for the range branch (air-conditioner target, with an integer step of 1). The node sequence shows the path taken.

Input	Expected	Path (nodes)	Exercises
non-numeric	reject (not numeric)	$1 \to 2$	node-1 false branch
15	reject ($<$ min)	$1 \to 3 \to 4$	node-3 true branch
31	reject ($>$ max)	$1 \to 3 \to 5 \to 6$	node-5 true branch
23	accept (rounded)	$1 \to 3 \to 5 \to 7 \to 8 \to 9$	node-7 true, normalising path

The four cases above execute all eight branch outcomes (node 7's false branch is taken by any range control declared without an integer step, e.g. a continuous control), and in doing so visit every statement node, so they satisfy branch and statement coverage as well. This illustrates the standard *subsumption* ordering of coverage strength:

$$\text{path coverage} \;\geq\; \text{branch coverage} \;\geq\; \text{statement coverage},$$

that is, achieving a stronger criterion guarantees the weaker ones but not conversely. The boundary inputs 15, 23 and 31 from Table 9.2 thus do double duty: chosen by black-box BVA, they also drive the most fault-prone branches of the white-box CFG.

9.6 Results and Traceability

The full suite passes: **65 tests, 65 passed**. Because tests are derived from the specification, every requirement family is traceable to at least one executable check, and conversely every test traces back to a requirement. Table 9.5 summarises the mapping from requirement families to representative covering tests.

Table 9.5. Requirements-to-tests traceability (representative tests).

Requirement family	Concern	Behaviour verified
REQ-4.1	Monitoring (live power, cumulative energy, offline detection)	that the home dashboard reports the expected metrics, and that a command to an offline device times out
REQ-4.2	Device control via capability schema (load, validate, command)	that the capability schema drives the available controls, and that an out-of-range command is rejected
REQ-4.3	Rules, conflict detection, auto-action opt-in and undo	that conflicting rules are detected, and that automatic actions are disabled by default
REQ-4.4	Recommendation generation, ranking and approval	that an accepted recommendation becomes a rule, and that recommendations are ranked and capped
REQ-4.5	Bill-saving estimation and cycle reporting	that the saving estimate follows the specified formula, and that the savings summary reports the billing cycle
NFR-SAF	Compressor rate limit, safety-critical protection, warnings	the compressor rate-limit test, and the safety-critical auto-power-off suppression test
NFR-SEC	Authentication, RBAC, no plaintext secrets	that an unauthenticated request is rejected, and that a password survives a hash round-trip without ever being stored in plaintext

Both styles of testing are present and traceable: pure-logic unit tests pin the algorithms (validation, pricing, the simulator energy differential), while seeded in-process tests driven through the public interface prove the wired system behaves correctly. Negative and boundary paths are first-class throughout: rejected, timeout and skipped command outcomes, out-of-range and unsupported controls, bad passwords, unauthenticated access and forbidden cross-role actions are all explicitly exercised. The principal residual gaps — direct assertion of notification delivery, frontend rendering, and non-functional load and transport security — are documented as known limitations to be closed with a notification endpoint test, manual UI acceptance, and a dedicated load and TLS harness respectively.

9.7 Maintenance

Verification and validation do not end at delivery; they continue through the Support phase of the life-cycle. Anticipated changes to SHEO classify into the four standard categories of software maintenance, each of which would re-enter the regression suite before release:

- **Corrective** — repairing a latent fault. For example, fixing an off-by-one error should the air-conditioner target guard ever admit 31; the corresponding boundary test in Table 9.2 would reproduce and then guard the fix.
- **Adaptive** — accommodating a changed environment. For example, updating the tariff math when the electricity utility revises its tiered price schedule, re-validated by the tariff-pricing unit tests.
- **Perfective** — improving function or quality without changing external behaviour. For example, adding a new device type (a water heater) by declaring one capability schema entry and one adapter, then extending the capability and command tests to cover it.
- **Preventive** — restructuring to forestall future defects. For example, hardening the rate-limiter against clock skew before any failure is observed, protected by the existing rate-limit regression test.

This classification realises the Support phase of the life-cycle introduced in the process chapter, and closes the verification-and-validation loop: each kind of change is admitted only once the relevant tests — old and new — pass, keeping the requirements-to-tests traceability of Table 9.5 honest as the system evolves.

Chapter 10

Conclusion and Future Work

10.1 Summary

This report has documented the full software-engineering lifecycle of the **Smart Home Energy Optimizer (SHEO)**, an AI-assisted monitoring and control platform designed to help residential occupants reduce electricity consumption and visualise the financial impact of their energy-saving decisions in Vietnamese Dong (VND).

SHEO is delivered as a three-tier web application: a FastAPI back-end organised into layered services and a capability-driven device abstraction, a React single-page front-end, and a mock device simulator that enables end-to-end demonstration without physical hardware. The platform's principal capabilities are:

- **Capability-driven monitoring and control.** Every device exposes a capability schema that drives both the API and the front-end widget automatically, so adding a new device type requires no UI code changes.
- **Explainable WHEN–THEN automation rules.** Residents author rules for their own unit in structured natural language; the rule engine interprets and audits them rather than executing opaque scripts.
- **Habit recommendations.** A recommendation engine identifies high-consumption patterns and proposes WHEN–THEN rules; residents review and accept each suggestion individually.
- **VND bill-saving estimation.** Before accepting a recommendation, the user sees an estimated monthly saving in VND. Accepted rules accumulate in a cycle-savings dashboard, closing the feedback loop between behaviour change and financial outcome.

The system is fully implemented and verified: **65 automated tests** (unit, integration, system, and acceptance levels) pass against the current codebase, and all functional requirement families listed in the SRS are represented in the test suite's traceability matrix.

10.2 Requirements Satisfaction

The client's single highest-priority acceptance criterion was:

> *The Customer can see the actual VND amount saved by a suggested automation rule before deciding whether to accept it.*

This criterion is satisfied through two complementary mechanisms. First, the *estimate-before-accept* flow computes a prospective monthly saving (in VND) at the point of recommendation review; the user sees the figure before committing. Second, the *cycle-savings dashboard* aggregates realised savings for each accepted rule over the current billing cycle, providing post-hoc validation that the estimate was meaningful.

Table 10.1 maps each functional requirement family to its implementation. Collectively, these families are exercised by the suite of 65 automated tests, whose per-requirement traceability is detailed in the appendix.

Non-functional requirements were addressed as follows: response-time constraints (NFR-PER) are enforced by asynchronous request handlers in the application server and in-memory caching; safety constraints (NFR-SAF) prevent the system from automatically powering off safety-critical devices; security constraints (NFR-SEC) are upheld by role-based access guards

Table 10.1. Functional requirement families and their satisfaction status

Req. family	Description	Key component
REQ-4.1	Real-time device monitoring	The device-listing and monitoring service with its capability widget
REQ-4.2	Remote device control	The device-control operation
REQ-4.3	WHEN–THEN automation rules	The rule engine
REQ-4.4	Habit recommendations	The recommendation service and its savings estimate
REQ-4.5	VND bill-saving dashboard	Cycle-savings aggregation in the bill-saving (optimisation) service

(the administrator-authorisation guards) and the IDOR vulnerability discovered during the adversarial review was patched and regression-tested.

10.3 Course-Concept Coverage

Table 10.2 provides a candid assessment of how each lecture topic in IT3180E is reflected in the SHEO project artifacts.

Table 10.2. IT3180E lecture-topic coverage by SHEO project

Lecture	Topic	Where addressed in this report	Coverage
C01	SE overview; stakeholder roles	Chapter 1: Client / Customer / Resident model, a general framing of the software-engineering lifecycle, quality focus, Function–Cost–Time triangle	Strong
C02	Software development process	Chapter 2: incremental model with agile-style iterations; ISO/IEC 9126 quality attributes mapped to NFRs	Partial
C03	Agile / Scrum	Chapter 2: working-software discipline maintained throughout; no formal Scrum roles or sprint artifacts documented	Light
C04	Software project management	Chapter 3: Work Breakdown Structure, effort-estimation table, CPM task network with critical path, Gantt schedule, and a risk register scored by Probability × Impact with Avoid/Transfer/Mitigate/Accept responses	Strong
C05	Software configuration management	Chapter 8: dedicated CM plan identifying CIs, baselines, and Git workflow	Partial

Lecture	Topic	Where addressed in this report	Coverage
C06	Requirements analysis and UML	Chapter 4: SRS, FR/NFR taxonomy, Use-Case Diagram, ERD, state-machine diagrams, activity-style user flow	Strong
C07	Software design and UI/UX	Chapter 5 (architecture, detailed design, and patterns) and Chapter 6 (UI/UX): context-of-use, user profiles, sitemap, user flow, wireframes, usability evaluation	Strong
C08	Software construction	Chapter 7: coding-style conventions, adversarial code review as formal inspection analogue, refactoring findings	Partial
C09	Quality assurance: testing and maintenance	Chapter 9: V-model mapping, black-box equivalence partitioning, boundary-value analysis, decision-table testing, white-box CFG examples, regression suite; maintenance classification	Partial

The project is strongest where the course emphasises artefact production. **C01** coverage is strong because the stakeholder model is rendered end-to-end from the glossary through to the role enumeration in the implementation. **C04** coverage is strong because Chapter 3 produced the full project-management suite — Work Breakdown Structure, effort estimation, a CPM task network with an identified critical path, a Gantt schedule, and a risk register scored by Probability × Impact with explicit Avoid/Transfer/Mitigate/Accept responses. **C06** coverage is strong because the project produced a full SRS together with a complementary suite of UML diagrams (use-case, ERD, state machines). **C07** coverage is strong across both the architectural-design half and the UI/UX half. Coverage is partial for **C02** (the incremental process model was followed in practice but not formally justified in a process document), **C05** (a dedicated CM plan exists but IEEE 828 compliance and the exact branching strategy are not fully cross-referenced from the main body), **C08** (the adversarial review activity occurred and its findings were corrected, but a formal inspection report was not produced as a standalone artefact), and **C09** (white-box CFG examples were added but a complete branch-coverage report was out of scope). Coverage is lightest for **C03**, which concerns the team-level Scrum process that is inherently difficult to demonstrate through a single design-and-build deliverable.

10.4 Lessons Learned

- **A single source of truth yields cumulative benefit.** Anchoring every API endpoint, front-end widget, and test assertion to the capability schema eliminated an entire class of synchronisation bugs. When the schema changed, the ripple effect was immediately visible rather than silently divergent.
- **Separating requirements from design is not bureaucracy.** Maintaining a discrete SRS before writing the SDD forced the team to confront ambiguities during elicitation rather than during coding. The *what* document also served as the primary input for the traceability matrix, making acceptance-test authorship straightforward.

- **An adversarial review surfaces defects that a cooperative walkthrough does not.** Running a structured correctness and security review from a non-author perspective (analogous to a formal inspection) surfaced a cross-unit IDOR vulnerability and an unvalidated rule-update path that had survived all prior testing. Both findings were patched and regression-tested before the final submission.
- **Oversight is not the same as surveillance — data minimisation is a design decision.** When the building owner gained a building-wide overview, the natural instinct was to surface, per unit, how many devices were switched on. We rejected that: a live per-unit count of running appliances would expose a resident's occupancy and behaviour to their landlord. Applying *data minimisation* (**NFR-SEC-5**), the overview discloses only aggregate load, energy, and bill — the data the owner legitimately needs for billing — plus device *reachability* (an operational, non-behavioural signal), and never per-device on/off state. Naming the privacy purpose turned an ambiguous label ("online") into a deliberate, defensible one ("reachable").
- **Client elicitation must be documented, not recalled.** The highest-priority acceptance criterion — VND savings visibility — was stated clearly in the initial elicitation session; because it was captured in the SRS, the team could verify against it mechanically. Requirements that were held only in memory tended to drift or be deprioritised under time pressure.
- **The process model should be named early, not inferred later.** SHEO was built incrementally — a working vertical slice first, then features fanned out — which maps naturally to the Incremental model. Not naming this decision explicitly in the process documentation made the C02 coverage weaker than the actual practice warranted.
- **Mock-before-hardware is a legitimate and traceable scope decision.** Decoupling the device-adapter interface from physical hardware from day one allowed the entire system to be demonstrated and tested without IoT equipment. The decision was justified in the feasibility study and respected throughout; it did not compromise the architectural validity of the design.

10.5 Future Work

Several directions would extend SHEO from a demonstration system to a production-grade platform.

Real-hardware integration via the device-adapter contract

The existing device-adapter interface and capability schema were designed with physical hardware in mind. Replacing the mock implementations with drivers for standard IoT messaging protocols would require no changes to the API or front-end layers. A priority integration target is the smart air-conditioner controller, which accounts for the majority of residential electricity consumption in the Vietnamese context.

Broader device and capability coverage

Current device types cover air conditioners, lighting, and socket plugs. Extending the capability registry to include water heaters, inverter washing machines, and EV chargers would increase the platform's addressable saving potential without requiring architectural changes, since all device behaviour is expressed through the existing capability schema.

Horizontal scalability

The present deployment runs as a single application-server process. Introducing a message broker between device adapters and the rule engine, combined with multiple stateless API workers behind a load balancer, would support multi-building deployments and eliminate the single point of failure identified in the NFR-SAF analysis.

Deeper project-management instrumentation

The project-management plan in Chapter 3 already establishes a Work Breakdown Structure, an effort-estimation table, a CPM schedule with an identified critical path, a Gantt chart, and a risk register. A follow-on phase could deepen these artefacts with earned-value tracking and periodic re-estimation as the project progresses. Adopting a formal Scrum process with a maintained Product Backlog, two-week sprints, and a Definition of Done would additionally close the C03 gap and improve team-level visibility during development.

White-box coverage gate

The current test suite emphasises black-box equivalence partitioning and boundary-value analysis. Adding an automated branch-coverage gate with a minimum-coverage threshold would provide a C09-compliant white-box verification gate and prevent coverage regression as the codebase grows.

Richer analytics and personalised insights

The cycle-savings dashboard currently aggregates savings by device and rule. Future work could incorporate time-series anomaly detection to flag unexpected consumption spikes, comparative benchmarking against similar household profiles, and personalised rule suggestions derived from appliance-usage clustering — moving SHEO from reactive reporting toward proactive energy intelligence.

Appendix A

Requirements Traceability Matrix

This appendix gives the consolidated requirements traceability matrix (RTM) for SHEO, linking each requirement to the design element that realises it and the activity that verifies it — an automated test, or design inspection where noted. It supports both forward (requirement → component → test) and backward (test → requirement) traceability. Policy thresholds (refresh interval, 60 s offline, 14-day baseline, 7-day minimum history, five active recommendations, 30-day dismissal, two-minute undo, AC compressor interval, ±20% drift) are centralised in the system configuration module. The full suite is **65 automated tests, all passing**.

A.1 Functional requirements

Req.	Requirement (short)	Realising component	Verified behaviour
REQ-4.1.1	Instantaneous power per device, live refresh	The device-simulation engine, the telemetry/-dashboard service, and the WebSocket push layer	Confirms that the dashboard endpoint returns live power metrics for all home devices
REQ-4.1.2	Cumulative kWh: today / cycle / range	The consumption-series method of the telemetry service, backed by the energy-aggregation query in the repository layer	Verifies that energy consumption is correctly bucketed by day, billing cycle, and arbitrary date range
REQ-4.1.4	Device silent > 60 s ⇒ offline	The online-status refresh routine of the telemetry service	Verifies that a command sent to a device silent for more than 60 seconds times out and that the device is marked offline
REQ-4.1.5	Top consumers ranking	The top-consumers query of the telemetry service	Verifies that devices are returned in descending order of energy consumption
REQ-4.2.1	Fetch capabilities before rendering	The capability-retrieval routine and the device-control API endpoint	Verifies that the capability endpoint drives control rendering for each device type

Req.	Requirement (short)	Realising component	Verified behaviour
REQ-4.2.2	On/off control for plug/bulb/fan/AC	The capability-validation routine together with the per-type device-adapter implementations	Verifies that power on and off commands are accepted and applied for all supported device classes
REQ-4.2.3	Validate command vs capability range	The capability-validation routine	Boundary test of the air-conditioner target range (accepts 16 and 30, rejects 15 and 31); verifies that out-of-range commands are rejected
REQ-4.2.4	Outcome success / rejected / timeout	The command-application step of the device-control service	Verifies that the service correctly reports success, rejection, and timeout outcomes for issued commands
REQ-4.2.5	Mock simulator (plug/bulb/fan/AC/sensor)	The device-simulation engine and the mock-device management method of the device-control service	Verifies that all mock device profiles are available and that mock devices can be added and subsequently deleted
REQ-4.3.2	Action must match device capability	The rule-validation routine of the rule engine	Verifies that a rule action referencing an unsupported device capability is rejected
REQ-4.3.3	Detect conflicting enabled rules	The conflict-detection routine of the rule engine	Verifies that enabling two mutually contradictory rules is detected and reported as a conflict
REQ-4.3.4	Enable/disable/edit/delete rules	The rule-update and rule-deletion operations of the rule engine	Verifies that a rule can be toggled, modified, and removed through the lifecycle management interface

Req.	Requirement (short)	Realising component	Verified behaviour
REQ-4.3.5	Log every execution	The rule-execution record model and the rule-execution step of the rule engine	Verifies that each triggered rule produces a persistent execution log entry, that an undo is recorded, and that the history is preserved after subsequent edits
REQ-4.3.6	Auto-action opt-in + 2-min undo	The auto-apply field of the rule model and the undo operation of the rule engine	Verifies that auto-action is disabled by default and that the undo window is correctly enforced
REQ-4.4.1	Recommend only after $\geq$ 7 days	The sufficient-history guard of the recommendation service	Verifies that no recommendation is generated for a device whose usage history is shorter than seven days
REQ-4.4.3	Rank by VND, $\leq$ 5 active	The active-recommendation cap enforcer of the recommendation service	Verifies that recommendations are ranked by projected monetary saving and that the active count is capped at five
REQ-4.4.4	Explicit accept $\Rightarrow$ becomes a rule	The acceptance handler of the recommendation service	Verifies that accepting a recommendation creates a corresponding automation rule
REQ-4.4.5	Dismissed $\Rightarrow$ suppressed 30 days	The dismissal handler of the recommendation service	Verifies that a dismissed recommendation does not reappear within the 30-day suppression window
REQ-4.5.1	14-day baseline profile per device	The hourly-baseline computation of the optimisation service	Verifies that the saving estimate uses the 14-day baseline formula before a rule is saved

Req.	Requirement (short)	Realising component	Verified behaviour
REQ-4.5.2	Saving = $\Sigma((\text{baseline}-\text{expected})\times\text{tariff})$	The saving-estimation method of the optimisation service, together with the tariff service	Verifies that the computed saving matches the expected value when the baseline-minus-forecast formula is applied with the configured tariff
REQ-4.5.3	Show estimate before save / accept	The saving-estimate endpoint of the savings API	Verifies that a monetary saving estimate is returned before the rule or recommendation is committed
REQ-4.5.4	Dashboard cycle savings (VND)	The accrual routine of the optimisation service	Verifies that the savings summary correctly aggregates realised savings over the current billing cycle
REQ-4.5.5	$\pm$20% drift $\Rightarrow$ needs recalculation	The drift-detection routine of the optimisation service	Verifies that the drift-check path is reachable and triggers a recalculation flag when consumption deviates by more than 20% from the baseline
REQ-6.1	Export readings/rules/savings as CSV	The CSV-export methods of the report service	Verifies that energy readings, automation rules, and saving summaries can each be exported as a well-formed CSV file

A.2 Non-functional requirements and business rules

ID	Requirement (short)	Realising component	Verified behaviour
NFR-SAF-1	$\leq$ 1 AC compressor command / 3 min	The safety-check routine of the device-control service	Verifies that a second air-conditioner compressor command issued within three minutes is blocked

ID	Requirement (short)	Realising component	Verified behaviour
NFR-SAF-2	No auto power-off of safety-critical device	The safety-check routine of the device-control service	Verifies that an automated power-off command targeting a safety-critical device is refused
NFR-SAF-3	Warn on temperature / unattended off	The rule-validation routine of the rule engine	Verifies that validation emits warnings when a rule would set a hazardous temperature or leave a device unattended in the off state
NFR-SEC-2	Authentication + RBAC	The authentication module, the administrator-authorisation guard and the building-overview service	Verifies that login succeeds with valid credentials and fails with invalid ones; that a resident-role user is prevented from adding a device and from reaching the Administrator's building overview; that the view-only Administrator (building owner) is forbidden from issuing a device-control command; and that a Resident may control only the operable devices of their own unit
NFR-SEC-3	No plaintext passwords	The salted password-hashing routine of the security module	Verifies that the stored credential is a salted hash and that the original plaintext is never stored or returned

ID	Requirement (short)	Realising component	Verified behaviour
NFR-SEC-4	Unit-scoped access to data	The unit-scoping filter of the device repository and the telemetry service	Rejection of a cross-unit device identifier: verifies that a Resident cannot retrieve consumption data for a device belonging to a different unit, while the Administrator may view every unit
NFR-SEC-5	Building-overview data minimisation	The building-overview aggregation service	By design inspection: the overview and unit drill-in expose only aggregate per-unit metrics (load, energy, bill) and device *reachability* — never a resident's per-device on/off state, so oversight cannot become behavioural surveillance
BR-1	Only the Administrator sells units / onboards & offboards residents	The administrator-authorisation guard and the resident onboarding/offboarding endpoints of the authentication and admin APIs	Verifies that a resident-role user cannot add a device; that the Administrator can sell a unit — provisioned with its default device package — to a new Resident who then controls that unit but cannot reach another; and that offboarding soft-removes the resident (login disabled) while the unit, its devices and history are retained
BR-2	Recommendations advisory until accepted	The acceptance handler of the recommendation service	Verifies that a recommendation has no effect on automation until it is explicitly accepted by the user

ID	Requirement (short)	Realising component	Verified behaviour
BR-3	Auto-action disabled by default	The auto-apply field of the rule model	Verifies that a newly created rule has automated execution disabled by default
BR-4	Deleting a rule keeps execution history	The rule-execution record model, which retains history without cascading deletes	Verifies that execution log entries remain accessible after the parent rule is deleted
BR-5	Tariff manually configurable; VND default	The tariff service and the settings API endpoint	Verifies that the tiered tariff structure is applied progressively, and that a resident-role user cannot modify tariff settings

A.3 Design-pattern traceability

Pattern	Realisation in SHEO	Evidence
Strategy (device adapters)	Per-device-type behaviour and power model encapsulated in interchangeable adapters	The device-adapter interface and its per-type implementations
Strategy (recommendation provider)	The "AI" habit-miner sits behind a swappable provider port; the shipped default is a deterministic, explainable miner, and a black-box ML provider could replace it	The recommendation-provider port and its heuristic implementation; a test verifies the provider is swappable (default is the heuristic provider, and an injected provider is the one consulted for detection)
Observer	Decoupled event fan-out (telemetry, notifications)	The event bus (Observer pattern)
Repository	Persistence hidden behind cohesive accessors	The repository layer
Dependency Injection	Request-scoped database session, user identity, and role guards	The dependency-injection provider for API handlers
State	Device and rule lifecycles	The domain model and rule engine

Bibliography

[1] Introduction to Software Engineering Teaching Group, *IT3180E — Introduction to Software Engineering*, Lecture slides C01–C09 (Overview of SE; Software Development Process; Agile Software Development; Software Project Management; Software Configuration Management; Requirement Analysis; Software Design and User Interface Design; Software Construction; Quality Assurance). Hanoi University of Science and Technology, SoICT, 2025.

[2] IEEE, *IEEE Std 1016 — IEEE Standard for Information Technology — Systems Design — Software Design Descriptions*. Institute of Electrical and Electronics Engineers.

[3] ISO/IEC/IEEE, *ISO/IEC/IEEE 29148 — Systems and Software Engineering — Life Cycle Processes — Requirements Engineering* (superseding IEEE Std 830).

[4] IEEE, *IEEE Std 828 — IEEE Standard for Configuration Management in Systems and Software Engineering*.

[5] IEEE, *IEEE Std 610.12-1990 — IEEE Standard Glossary of Software Engineering Terminology*.

[6] ISO/IEC, *ISO/IEC 9126 — Software Engineering — Product Quality*.

[7] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th ed. McGraw-Hill.

[8] I. Sommerville, *Software Engineering*, 10th ed. Pearson.

[9] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

[10] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. Addison-Wesley.

[11] K. Beck et al., *Manifesto for Agile Software Development*, 2001. https://agilemanifesto.org

[12] S. Ramírez, *FastAPI Documentation*. https://fastapi.tiangolo.com

[13] *SQLAlchemy 2.0 Documentation*. https://docs.sqlalchemy.org

[14] *React Documentation*. https://react.dev